ChronoScope: Time-Series Diagnostics and Forecast Evaluation

Felipe Santibáñez-Leal
CAOS open-research programme, Santiago, Chile
ORCID: 0000-0002-0150-3246

Working revision; not a new published version.

Technical report v0.26.000 draft, 5 November 2026 Licence CC BY 4.0
CAOS open-research programme; ChronoScope forecasting and diagnostic atlas.
Published predecessor DOI: 10.5281/zenodo.23884372. Stable concept DOI: 10.5281/zenodo.23874816.
Not peer reviewed. Source, data and computational records: `https://github.com/fsantibanezleal/CAOS_RES_ChronoScope`.

Abstract

ChronoScope combines source-bound time-series diagnostics with separate forecast evaluations. Exact Matrix Profile and FLUSS annotations, multiscale entropy and seeded nonlinear estimates describe observed sequences without establishing intrinsic predictability. A disjoint-series M4 Daily experiment retains an unfavorable method-selection result, while an external combination study finds lower mean MASE for temporally validated mixtures than for a recency-only control on eligible M4 Weekly and M3 Monthly series. AutoTheta remains better on monthly point error and both collections' interval score. Paired series resampling quantifies those bounded comparisons. On the fifteen selected blocks, the planned Timestral-209M minus AutoTheta mean MASE difference is -0.44740, with a descriptive case-reweighting interval from -0.79356 to -0.15991. A separate prospectively specified synthetic experiment finds that validation-selected ridge does not improve the browser point forecast. An independent confirmation then preserves that point map and evaluates prefix-stratified residual calibration on 800 fresh series. Mean normalized interval score falls from 4.749003 to 2.393376, with a paired difference of -2.355627 and exploratory 95 percent interval [-2.437258,-2.273077]. Overall marginal coverage is 82.73 percent at the nominal 80 percent level, with remaining family-specific differences disclosed. A source-bound browser walkthrough separates observed prefixes, expert paths, sequence geometry, held-out errors, historical information timing and descriptive comparisons. Shared source ownership prevents a late case load from acquiring another case identity. These views expose the admitted evidence and its availability; they do not create additional forecasting trials. Complete forecasts, partition identities, source hashes and independent scoring expose both favorable and unfavorable findings. These experiments establish neither operational benefit nor calibration for arbitrary authentic series.

1 Introduction

Inventory, staffing and capacity commitments depend on forecasts made before demand is observed. A forecast revision can alter those commitments even when its statistical benefit has not been measured. The relevant engineering question is therefore broader than selecting the smallest displayed error: which observations were available, what algorithm produced the path, which future outcomes were scored, and what uncertainty remains before an action is taken?

Fildes, Goodwin and De Baets analyzed 147,131 one-step demand forecasts from six field studies, pairing each system forecast and final planner forecast with realized demand [29]. The median

share of stock-keeping units with improved accuracy was 51.5% across business units. The separate median share with improved bias was 55.6%. The variability across units motivates preserving the original forecast and the revision as separate paths, scoring them on the same targets and reporting the distributions of gains and losses. ChronoScope's case experiments use different sources and do not estimate the value of those planners' adjustments.

ChronoScope supports inspection of these questions through selected real excerpts, generated mechanisms and explicit controls. Generated cases explain mechanisms and expose defects; they are not customer observations. The real excerpts represent electricity demand, air quality and M4 series. Air quality is a forecasting example rather than a demand-planning measurement. No monetary benefit, service-level improvement, laboratory result or customer deployment is inferred from these examples.

The browser offers series structure, forecasts, interval boundaries, individual errors and historical streaming diagnostics. Documentation separates a sourced executive problem framing from technical method descriptions. The product is a research and teaching instrument. Acceptance of a production policy would additionally require an operational objective, representative data, governance of revisions, latency constraints and prospective evaluation.

The central product contribution is an inspectable connection between a numerical result and the information that produced it. The same held-out error can look compelling or uninformative depending on whether the training history was contiguous, whether the comparison used the same targets, and whether method selection inspected those targets. The report consequently keeps five studies separate: the selected-case forecast comparison, historical one-step streaming reconstruction, external forecast combinations, disjoint-series corpus selection, and synthetic browser calibration. Their sampling units, horizons and uncertainty statements differ. Combining their row counts would not create one larger independent experiment.

The report makes four concrete contributions. It documents the implementation identities and equations needed to interpret the delivered predictions; specifies the artifact contracts and source-reconstruction checks; reports both favorable and unfavorable comparisons with bounded uncertainty; and explains how the browser exposes the evidence without converting a selected example into an operational recommendation. The title names diagnostics and evaluation because neither native model-state updating nor a general forecasting advantage has been established by these experiments.

The following sections formulate the statistical objects, describe the pipeline and contracts, then present the implemented methods and experimental results. Later sections distinguish structural validation from scientific inference and describe the remaining limitations. References identify the underlying published methods; the source and numerical archives identify the implementations actually evaluated. Reproducing an implementation and reproducing a publication's complete benchmark are different claims throughout.

2 Problem formulation and notation

An admitted univariate case supplies a finite ordered sequence, an observation-index convention, a seasonal lag and a forecast horizon. The measurement unit is denoted by U when provenance does not supply a physical unit. An index increment means one supplied observation, not necessarily one second, day or month. At origin n, the fitting information is $\mathcal{F}_n = \sigma(y_1, \ldots, y_n)$ together with declared fixed configuration. The reserved values $a_j = y_{n+j}$, $1 \leq j \leq H$, become evaluation outcomes. A valid forecasting procedure produces its path from the fitting information, not from retrospective diagnostics computed using the reserved block.

The interface's mathematical output is a finite matrix

$$\widehat{Q}_n \in \mathbb{R}^{H\times 3}, \qquad \widehat{Q}_{nj1} \le \widehat{Q}_{nj2} \le \widehat{Q}_{nj3}, \qquad Q = \{0.1, 0.5, 0.9\}. \tag{1}$$

The canonical probabilities identify requested output columns. A library point mean placed in the central column remains that library's point forecast; the label alone does not prove a conditional median. Ordered endpoints do not establish their nominal coverage. Missing forecasts, invalid dimensions and nonfinite values are therefore distinct from a valid wide interval or a large but finite error. The report records unavailable quantities instead of constructing an apparently successful forecast from incomplete data.

There are several objectives rather than one universal optimization target. A point forecast can minimize a fitting loss, a temporal selector can rank candidates by historical scaled absolute error, and an interval procedure can be evaluated by coverage together with a width-sensitive proper score. The final scientific quantity is the study-specific out-of-sample loss, not the smallest training objective. For example,

$$\widehat{R}_A = \frac{1}{N}\sum_{i=1}^{N}\frac{1}{J_i}\sum_{o\in\mathcal{O}_i} L(A; i, o) \tag{2}$$

defines equal-series aggregation after equal weighting of the admitted origins within each series. Other studies in this report explicitly use one final block or one-step trajectories. Their N, origin sets and loss definitions must not be interchanged.

The assumptions needed for an inference are stronger than those needed to compute a score. Source reconstruction requires correct ordering and identity; a seasonal scale requires enough finite observations and a positive denominator; a population interval additionally requires a sampling or exchangeability argument. Exact deduplication removes identical sequences, while related-source dependence may remain. A source-bound artifact makes these assumptions auditable but cannot make them true by construction. The problem solved by the application is thus evidence inspection and finite empirical comparison. An operational policy would require an additional loss function, deployment population, information-arrival policy and prospective acceptance study.

2.1 Notation and dimensional conventions

Table 1 gives the shared combination and evaluation notation. Table 2 adds diagnostic, adapter and browser symbols. Several conventional letters have local meanings: the backshift operator in ARIMA is unrelated to a minibatch count, and an attention query matrix is unrelated to a requested forecast probability. These meanings are stated at the defining equation. Units apply to the numerical quantity, rather than its JSON representation or plotted pixel coordinate.

Positive affine changes $y'_t = c + dy_t$, $d > 0$, provide useful structural checks. A source-unit error or interval width scales by d; squared error scales by d^2; coverage and correctly normalized MASE remain unchanged when their denominators exist. Normalized context features and delay geometry should also remain unchanged away from explicitly disclosed degeneracy thresholds. These properties do not replace a complete dimensional audit of every numerical literal. Constant and near-constant prefixes need particular care because an arbitrary unit-sized denominator can violate the expected transformation. The browser normalized map refuses a zero standard deviation; historical offline adapters retain their separately described fallback behavior.

Table 1 collects the mathematical symbols and their units. The generic unit U means the admitted series' measurement unit; the paper does not infer a physical SI unit when source provenance does not specify one. Observation counts and positions describe the supplied sequence's sampling grid.

Table 1: Notation for observed data, context features, causal selection, convex fitting and paired-series evaluation, with measurement units distinguished from dimensionless quantities.

Symbol	Meaning	Unit
y_t	Observation at sequence position t.	U
n, H, m	Prefix length, horizon length and seasonal lag.	observations
a_j, p_{okj}	Reserved target and expert point forecast at a stated lead.	U
$\mathcal{O}_n, o_{\min}$	Outer-prefix memory grid and its first permitted origin.	index set; index
o, v, q	Historical origin, inner validation origin and generic information cutoff.	index
J, K	Number of remembered origins and number of experts; $K = 5$.	counts
R, P, w	Recent window, preceding window and their shared length.	U; observations
d	Half-window length used by the context mean contrast.	observations
$s, \widehat{\sigma}$	Context-window standard deviation and expert residual standard deviation.	U
f_a	Context feature coordinate, including normalized contrasts and cycle correlation.	dimensionless
s_o, s_m	In-prefix training normalization and final seasonal error scale.	U
L_{ok}	Mean absolute expert error at a remembered origin after normalization.	dimensionless
r_j, a_j^{loc}	Recency weight and context-and-recency kernel weight.	dimensionless
D_j, b_D	Context distance and kernel bandwidth.	dimensionless
c_k, g_k, ℓ_k	Competence estimate, recency loss and local-context loss.	dimensionless
ρ, τ, ε	Shrinkage, softmax temperature and uniform-floor fraction.	dimensionless
$v_k, w_k, \widetilde{w}_k$	Unfloored allocation, fitted simplex allocation and post-fit floored allocation.	dimensionless
$\mathcal{F}_q$	Information supplied by observations through q.	information set
$\mathcal{T}_v, \mathcal{V}_n$	Completed training-origin set and eligible validation-query set.	index sets
$S_n(c)$	Mean scaled absolute validation error for candidate c.	dimensionless
E_{vhk}, A	Normalized signed expert error and its row-stacked matrix.	dimensionless
N_e	Number of error rows in a stacking fit, equal to JH.	rows
G, λ	Error Gram matrix and positive quadratic regularization coefficient.	dimensionless
u, Δ_K	Uniform allocation and the nonnegative unit-sum simplex.	dimensionless
B, M_B	Active expert set and its regularized Gram matrix.	set; dimensionless
x, z	Solutions of the two linear systems used on an active face.	dimensionless
$F(w)$	Regularized squared-error fitting objective.	dimensionless
ℓ_n, a, b	SES terminal level, SES coefficient and Theta linear slope.	U; 1; U/observation
$\Phi^{-1}(q)$	Standard-normal quantile at probability q.	dimensionless

Symbol	Meaning	Unit
s, N	External-series identity and number of admitted series in one collection.	index; count
$\bar{L}_s(A)$	Within-series mean loss for method A over its admitted outer origins.	dimensionless
$d_s(A, B)$	Paired series loss difference for methods A and B.	dimensionless
$\bar{d}, \bar{d}^*$	Collection mean difference and its paired-series resampled counterpart.	dimensionless
S_i^*, Q_p	Resampled series identity and empirical quantile at probability p.	index; same unit as argument
α	Interval miscoverage, fixed at 0.2 for the reported MSIS.	dimensionless
$[x]_+$	Positive part, $\max(x, 0)$.	same unit as x

Table 2: Additional notation for diagnostic estimates, forecast adapters and browser calibration, distinguishing source measurements from normalized variables and counts.

Symbol	Meaning	Unit
t, j, h, L, C	Observation index, lead index, input length and context cap in their local equations.	indices; observations
ℓ_t, b_t, s_t	Smoothing level, trend and additive seasonal state.	U; U/observation; U
α, β, γ	Smoothing coefficients; α separately denotes interval miscoverage where stated.	dimensionless
$B, d, D, \phi, \Phi, \theta, \Theta$	ARIMA backshift, differencing orders and polynomial coefficients.	operator; counts; dimensionless
ϵ_t	Model innovation or residual in the stated observation equation.	U
$\mathcal{L}, F_\theta$	Lag-index set and fitted recursive tree mapping.	indices; output U
μ, σ, z	Prefix mean, prefix population standard deviation and normalized values.	U; U; dimensionless
$\widehat{Q}, Q$	Forecast-column matrix and requested probability set.	U; dimensionless
Q_a, K_a, V_a, d	Attention matrices and their internal feature dimension.	latent coordinates; count
W, X, Y, λ	Browser linear weights, normalized fitting arrays and ridge coefficient.	dimensionless
$\bar{x}, s(x), \widehat{u}_h, \widehat{a}_h$	Browser window mean, scale, normalized and reconstructed forecast.	U; U; 1; U
r_{ih}, q_h, k	Absolute normalized calibration error, retained rank value and rank index.	1; 1; count
$g(x), \mathcal{C}_b, n_b$	Prefix roughness, stratum calibration set and its size.	1; set; count
P_i, I_i	Matrix Profile distance and nearest nontrivial subsequence index.	dimensionless; index
m, r, A, B	Entropy embedding length, matching tolerance and match counts.	observations; U; counts
$p_{(i)}, q_{(i)}, H_K$	Raw and adjusted test probabilities and harmonic correction factor.	dimensionless

Symbol	Meaning	Unit
$IS, MSIS, \mathrm{MASE}$	Interval score, its seasonal normalization and scaled absolute error.	U; 1; 1

3 Data provenance and temporal reconstruction

The corpus contains eleven generated cases and four real excerpts. Generated mechanisms cover seasonality, trend, intermittent arrivals, a level shift, multiple seasonalities, heteroscedasticity, long memory, a delayed nonlinear system and a known-future exogenous driver. White noise and a near-random walk remain visible as controls. The exogenous-driver comparison requires a declared arrival policy; univariate forecasts do not gain access to future covariates automatically.

Table 3 distinguishes eleven generated mechanisms from four real excerpts and fixes the history, horizon and seasonal-lag choices used by each case.

Table 3: Admitted case inventory with original contiguous history length, forecast horizon, seasonal lag and source class; generated mechanisms remain distinct from authentic observations.

Case	History	Horizon	Lag	Source
BRKV_level_shift	288	12	12	Generated
CHAO_mackey	488	12	1	Generated
CTRL_white_noise	148	12	1	Generated
EXOG_promo	226	14	7	Generated
HETV_garch	488	12	1	Generated
INTM_demand	188	12	1	Generated
LMEM_fractional	488	12	1	Generated
MSEA_daily_weekly	648	24	24	Generated
REAL_electricity	456	24	24	Real excerpt
REAL_m4_daily	386	14	7	Real excerpt
REAL_m4_hourly	456	24	24	Real excerpt
REAL_pm25	456	24	24	Real excerpt
RWLK_noise	188	12	1	Generated
SEAS_hourly	456	24	24	Generated
TRND_seasonal	168	12	12	Generated

The source manifest records provenance, redistribution permission, seed, original series descriptors, artifact paths and byte counts. Real-data CSVs are committed as compact admitted excerpts. A source's redistribution status is an explicit field; a restricted source must not expose its raw path merely because aggregate metrics are publishable. The current comparison uses only available, non-redacted inputs.

3.1 Source collections, exact slices and measured quantities

The electricity source is Artur Trindade's ElectricityLoadDiagrams20112014 collection at the UCI Machine Learning Repository [26]. It contains quarter-hour power readings for 370 clients from 2011 through 2014 on the Portuguese local clock. The redistribution licence is CC BY 4.0. The acquisition script selects client `MT_010`, starts at row 35,040 of the downloaded table and sums four consecutive recorded kilowatt values for each retained row. It uses the first timestamp in that group as the label. The retained CSV contains 480 rows from 1 January 2012 at 00:15 to

20 January at 23:15. Its numerical value is the sum of four power readings. A time integral would require multiplication by the quarter-hour duration, and an hourly average would require division by four. Neither transformation is applied. In particular the values must not be labelled as kilowatt hours.

The Beijing source is Song Chen's UCI Beijing PM2.5 collection [27], under CC BY 4.0. The full table has 43,824 hourly observations covering 2010 through 2014. The concentration column records US Embassy particulate concentration in micrograms per cubic metre. Nearby airport weather columns are not used by the univariate comparison. The loader skips 8,760 rows and takes 480 observations dated 1 January 2011 at 00:00 through 20 January at 23:00. It carries forward the last available concentration for a missing token, with an initial value of zero when no earlier retained measurement is available. The excerpt contains 24 zero values. The CSV does not distinguish an imputed initial zero from a recorded zero, so the original missingness cannot be reconstructed from those values alone. This matters for lag estimates near the start and for any score normalized by observed target mass.

The two M4 excerpts are attributed to the M4 and Monash archives [30, 28]. The retained hourly series has 480 values from 7 January 2015 at 12:00 through 27 January at 11:00. The daily series has 400 values from 1 March 1994 at 12:00 through 4 April 1995 at 12:00. Neither retained identity supplies a physical measurement unit. The loader used the first available local Arrow cache of the corresponding source, choosing the first full finite window with standard deviation at least 10^{-6} and lag correlation at least 0.2 at lag 24 or 7. That retrospective eligibility rule can inspect observations that later become held-out targets. It defines the selected case corpus and precludes interpreting those excerpts as an unbiased draw. The archived acquisition cache revision is unavailable. The actual consumed inputs are the committed CSVs with four-decimal values, bound by content hashes. The Monash record identifies the related collection and licence, while the CSV hash identifies the precise numerical observations in this report.

3.2 Why a replay history is not a fitting series

The released trace stores at most three hundred historical display samples and their original indices. Nine case histories exceed that display limit. Treating those samples as consecutive observations changes the time interval and can destroy the meaning of a seasonal lag. It also changes a historical scale used by MASE. The initial comparison made this error and is invalidated; its earlier bytes remain identifiable through Git, not as admitted scientific evidence.

The corrected protocol reconstructs the original series from each committed CSV or deterministic case definition and its manifest seed. At every displayed index the reconstructed observation must agree with the released rounded value within 0.000051. The final targets must also agree at that tolerance, and the original prefix length must match the trace's declared history length. All fifteen reconstructions satisfy these checks. The complete contiguous prefix is used for light fitting and historical scaling. Original per-engine context caps remain explicit; a common available prefix does not mean every algorithm consumes the same number of observations.

The reconstructed history retains source precision. Frozen targets and model paths retain their released four-decimal precision. This small rounding distinction is disclosed and is not the much larger temporal-decimation error. Source and implementation hashes bind the corrected artifact to the exact inputs. A replay-only array whose declared length exceeds its display length is refused as a consecutive fit input.

4 Data contracts and observable quantities

The contract catalog separates fourteen boundaries because similarly shaped arrays can represent different experiments. Its semantic version belongs to the interface; a payload's schema tag identifies its serialized family. These identifiers need not have the same numeric suffix. Normative JSON Schemas validate committed JSON and generate TypeScript types. A generated type is not a runtime validator for an arbitrary network response. Numerical reconciliation, subject ownership, temporal eligibility, graph identity and licensing are separate checks.

The tables below are generated from the catalog and schema files by `scripts/build_report_contract_tables.py`. They list every root field, its declared type, root constraint, required status and quantity category. Nested arrays and objects follow the corresponding normative schema; the prose states the scientific meaning that a structural type cannot supply. U denotes a source series' measurement unit, which remains unspecified when the admitted provenance does not identify a physical unit. Counts and observation positions are distinguished from measured values. Statistical ratios and normalized residuals are dimensionless.

4.1 Trace: displayed history and retained predictions

The trace boundary records one selected case's display history, reserved targets, method forecasts and summaries. Table 4 describes interface revision `2.0.0`. Its history length is the length of the original prefix, while the exported history may contain fewer display samples. The parallel history-index array supplies original observation positions. Those positions are essential: a three-hundred-point display is not automatically a three-hundred-step contiguous fitting sequence. Forecast paths and targets retain source units; seasonal lag and horizon are observation counts, and quantile levels are probabilities. Method records combine prediction arrays with scaled and unscaled scores, so the container has mixed units rather than one inferred physical unit. Redaction is explicit and does not license a consumer to reconstruct or expose restricted source values. The pipeline's trace serialization is checked against the schema and cross-field tests before browser copying. The atlas uses the admitted fresh comparison for contiguous fitting history and verifies its target relationship to the trace. A missing case does not acquire another trace's identity, and a retrospective diagnostic result is not silently substituted for a forecast path.

Table 4: Root fields of the trace contract, interface revision `2.0.0`. Types, units and structural constraints distinguish required members from optional metadata; nested records are listed separately.

Field	JSON type	Unit	Root constraint	Required
`schema`	fixed value	metadata	const="chronoscope.trace/v2"	yes
`case_id`	string	metadata	No explicit range	yes
`seasonality`	integer	observations	minimum=1	yes
`horizon`	integer	observations	minimum=1	yes
`quantile_levels`	array	probabilities		yes
`redacted`	boolean	metadata	No explicit range	yes
`history_len`	integer	observations	minimum=0	yes
`history_index`	array	indices		yes
`history`	array	source U		yes
`actual`	array	source U		yes
`methods`	array	mixed; see text		yes
`summary`	object reference	mixed; see text	No explicit range	yes

4.2 Manifest: subject identity, provenance and delivery references

The manifest binds a case identifier to source classification, provenance, engine identity, series descriptors, artifact references and an execution-lane decision. Table 5 describes interface revision

2.0.0. Its best-method and metric fields summarize an existing experiment; they are not permission to train a selector using the reserved block. Series descriptors have mixed units: counts describe lengths, distribution summaries inherit source units, and correlations are dimensionless. The seed is a reproducibility identifier rather than a physical measurement. Artifact references include paths and byte counts, with analysis and streaming references allowed to represent absence under their declared forms. A public-artifact permission belongs to provenance and must be checked separately from whether a JSON object parses. The browser loads the selected manifest before its dependent trace and checks that the returned subject matches the requested admitted case. Index membership prevents an arbitrary case path from becoming a selected subject. Missing optional analysis differs from a missing core trace. Producer tests protect the manifest and reference structure, while consumer tests protect case consistency; neither a human-readable category nor an engine label independently establishes that two payloads share the same numerical inputs.

Table 5: Root fields of the manifest contract, interface revision 2.0.0. Types, units and structural constraints distinguish required members from optional metadata; nested records are listed separately.

Field	JSON type	Unit	Root constraint	Required
schema	fixed value	metadata	const="chronoscope.manifest/v1"	yes
case_id	string	metadata	No explicit range	yes
category	string	metadata	No explicit range	yes
real_or_synthetic	string	metadata	No explicit range	yes
expected_band	string	metadata	No explicit range	yes
engine	object	metadata		yes
provenance	object	metadata		yes
analysis_artifact	declared union	metadata	No explicit range	yes
streaming_artifact	declared union	metadata	No explicit range	yes
series	object reference	mixed; see text	No explicit range	yes
seed	integer	metadata	minimum=0	yes
artifact	object reference	metadata	No explicit range	yes
lane	fixed value	metadata	enum=["live","precompute"]	yes
gate	object reference	metadata	No explicit range	yes
flags	array	metadata		yes
best_method	string/null	metadata	No explicit range	yes
metrics	object	mixed; see text		yes

4.3 Case index: inventory rather than an accuracy claim

The case index is the compact entry point for browsing the selected corpus. Table 6 describes interface revision 1.0.0. It contains a schema tag, engine-version metadata, a declared case count and the case entries. The count is a number of admitted subjects, not a number of independent forecasting trials. Entry descriptors support grouping and navigation; scientific comparison still requires the associated trace and experiment records. The index is produced from the committed case inventory and consumed by the source selector. A loading failure leaves the inventory unavailable rather than creating a plausible set of cases. An unknown requested identifier is refused before a manifest request. This matters for shared links: displaying the first available case under a stale requested identifier would give the wrong subject to otherwise valid forecasts. Structural validation can check field types and required members, but reconciliation must also compare the declared count with actual entries and referenced manifests. The original fifteen cases remain separate from the external combination inventory and the complete daily corpus. A larger count in either of those experiments does not change what this selected-case index represents.

Table 6: Root fields of the index contract, interface revision `1.0.0`. Types, units and structural constraints distinguish required members from optional metadata; nested records are listed separately.

Field	JSON type	Unit	Root constraint	Required
`schema`	fixed value	metadata	const="chronoscope.index/v1"	yes
`engine_version`	string	metadata	No explicit range	yes
`n_cases`	integer	counts	minimum=0	yes
`cases`	array	metadata		yes

4.4 Analysis: heterogeneous diagnostics and explicit availability

The analysis boundary holds retrospective diagnostics for one supplied sequence. Table 7 describes interface revision `2.1.0`. Its required groups cover stationarity, autocorrelation, seasonality, filtering, change points, volatility, distribution, fractal measures, nonlinear analysis and causality. An optional sequence group carries the later original-axis pattern calculations. These objects cannot share a single numerical unit: a spectral frequency, correlation coefficient, residual variance, entropy estimate and raw time-reversal statistic have different dimensions. The sequence length and declared seasonal period are observation counts. Method descriptions define the remaining quantities, availability rules and assumptions. Some groups are deliberately flexible objects, so a structurally valid analysis file is not a certificate that every numerical diagnostic is meaningful on every input. Missing dependencies and insufficient samples must remain visible as unavailable calculations. The analysis producer runs outside the browser for heavier methods; lightweight browser diagnostics are independent computations with their own scope. Several stored analyses describe the complete sequence, including later forecast targets. They are therefore not admissible as origin-time predictors without a separate prefix-only computation. The consumer never turns such retrospective information into evidence that a forecast issued earlier knew the future.

Table 7: Root fields of the analysis contract, interface revision `2.1.0`. Types, units and structural constraints distinguish required members from optional metadata; nested records are listed separately.

Field	JSON type	Unit	Root constraint	Required
`n`	integer	observations	minimum=1	yes
`seasonality_period`	integer	observations	minimum=1	yes
`stationarity`	object	mixed; see text	Open object	yes
`autocorrelation`	object	mixed; see text	Open object	yes
`seasonality`	object	mixed; see text	Open object	yes
`filters`	object	mixed; see text	Open object	yes
`changepoints`	object	mixed; see text	Open object	yes
`volatility`	object	mixed; see text	Open object	yes
`distribution`	object	mixed; see text	Open object	yes
`fractal`	object	mixed; see text	Open object	yes
`nonlinear`	object	mixed; see text	Open object	yes
`causality`	object	mixed; see text	Open object	yes
`sequence`	declared union	mixed; see text	No explicit range	no

4.5 Streaming: aggregate trajectories from a specific replay

The streaming boundary records nominal coverage, horizon, warmup, method trajectories, optional covariate information and reference metadata. Table 8 describes interface revision `2.0.0`. Coverage is a fraction; the horizon and warmup count observations. Method objects retain metric trajectories and timing-related quantities whose units belong to their named measures. The optional covariate distinguishes absence from an aligned driver and must not imply that future values were known unless its arrival policy says so. This serialized family represents the historical replay configuration, including the smoothing-plus-half-drift base and the evaluated adaptive controllers. It is not interchangeable with the corrected finite-history Theta forecaster in the common final-block

experiment. Nor does a stored latency establish a resource-normalized comparison without its execution environment. The producer's schema and numerical tests protect the historical artifact; the later per-step reconstruction supplies a separate detailed audit. If streaming output is missing, the web view reports unavailability rather than simulating replacement trajectories. The nominal target is not a mathematical guarantee for each finite window, and temporal averages must retain their distinction from averages of local absolute coverage errors.

Table 8: Root fields of the streaming contract, interface revision `2.0.0`. Types, units and structural constraints distinguish required members from optional metadata; nested records are listed separately.

Field	JSON type	Unit	Root constraint	Required
`nominal_coverage`	number	probabilities	minimum=0; maximum=1	yes
`horizon`	integer	observations	minimum=1	yes
`warmup`	integer	metadata	minimum=0	yes
`methods`	object	mixed; see text		yes
`covariate`	object/null	mixed; see text	Open object	yes
`references`	object	metadata		yes

4.6 Browser point-model metadata and binary execution

The browser point-model boundary pairs legacy metadata with an ONNX binary. Table 9 describes interface revision `2.0.0`. The metadata retains its historical name while the interface identifies the computation as a normalized linear map. It declares lookback, horizon, normalization, normalized residual scale and training-window count. Counts and the residual scale are dimensionless, with lookback and horizon measured in observations. The historical metadata schema permits positive dimensions generally, but the admitted graph and current calibration have the specific forty-eight-input, twelve-output identity. The graph is a binary asset; JSON validation of its companion metadata is not ONNX execution or numerical parity. The browser verifies the graph digest through the calibration binding and checks actual output dimensions, type and finite values. The original point matrix remains unchanged by the new interval study. A missing model is omitted from the available live paths; it does not replace another method or acquire a favorable score. Constant lookbacks and unsupported horizons are refused because the required normalization or output contract is unavailable. Legacy Gaussian dispersion is retained as historical metadata, not reused as a silent fallback for the confirmed interval procedure.

Table 9: Root fields of the nlinear-model contract, interface revision `2.0.0`. Types, units and structural constraints distinguish required members from optional metadata; nested records are listed separately.

Field	JSON type	Unit	Root constraint	Required
`name`	fixed value	metadata	const="NLinear"	yes
`lookback`	integer	observations	minimum=1	yes
`horizon`	integer	observations	minimum=1	yes
`normalization`	fixed value	metadata	const="subtract_last_then_divide_std"	yes
`sigma_norm`	number	dimensionless	minimum=0	yes
`trained_windows`	integer	counts	minimum=1	yes

4.7 Confirmed browser calibration: dimensions and statistical population

The confirmed-calibration boundary binds an interval procedure to one exact point graph. Table 10 describes interface revision `1.0.0`. Required fields fix the graph digest, observed-window normalization, lookback, horizon, miscoverage, roughness thresholds, calibration counts and absolute-residual radii. Radii are normalized by the observed-prefix standard deviation; multiplying by that scale restores source units. Roughness is dimensionless because both its numerator and denominator have squared source units. Three groups each carry twelve lead-specific radii, while global radii

retain a separate comparator. The producer uses disjoint calibration and test identities, and independent evidence checks reconstruct all radii and held-out paths. The consumer refuses a mismatched graph or malformed calibration instead of returning to Gaussian bands. This structural binding prevents accidental model-calibration substitution, but it does not prove that an authentic deployment series is exchangeable with the generated calibration population. Overall coverage, family outcomes and lead outcomes remain statistical evidence with that population boundary. A fixed probability level in the schema also does not authorize interpolation to another level or a simultaneous whole-path coverage claim.

Table 10: Root fields of the browser-calibration contract, interface revision `1.0.0`. Types, units and structural constraints distinguish required members from optional metadata; nested records are listed separately.

Field	JSON type	Unit	Root constraint	Required
`schema`	fixed value	metadata	const="chronoscope.browser-calibration/v1"	yes
`point_model_sha256`	fixed value	metadata	const="621265165 211f756cff537ade e364b9790c34ced5 67bc2a5087513360e1f69f6"	yes
`lookback`	fixed value	observations	const=48	yes
`horizon`	fixed value	observations	const=12	yes
`alpha`	fixed value	probabilities	const=0.2	yes
`normalization`	fixed value	metadata	const="subtract_last_then_divide_population_std"	yes
`roughness_boundaries`	fixed value	dimensionless	const=[0.5,1.25]	yes
`calibration_bin_counts`	array	counts	minItems=3; maxItems=3	yes
`absolute_residual_radii`	array	dimensionless	minItems=3; maxItems=3	yes
`global_radii`	array	dimensionless	minItems=12; maxItems=12	yes

4.8 Ridge reference study: a retained unsuccessful point candidate

The ridge-study boundary preserves the independent point-model selection experiment even though its selected point map did not improve the declared criterion. Table 11 describes interface revision `1.1.0`. Its JSON identifies the candidate, fixed input and output dimensions, miscoverage, lead-specific absolute-residual radii and seven selection records. The optional explicit normalization descriptor documents the producer's actual convention. A companion ONNX binary makes the selected float32 map executable without implying that it is the served point model. Selection losses and residual radii are normalized quantities; horizon and lookback count observations. Training, validation, calibration and test roles remain distinct in the numerical evidence. The publisher and independent checker bind source hashes, split identities, graph structure and scored forecasts. Removing an unfavorable candidate or replacing its stored outcomes with a later successful interval experiment would erase the question this contract exists to preserve. The browser's production calibration is a different boundary with a different point-graph identity. Missing or changed ridge evidence is therefore an admission failure for the study, not permission to substitute the live graph and continue reporting the original candidate's results.

Table 11: Root fields of the browser-ridge-study contract, interface revision `1.1.0`. Types, units and structural constraints distinguish required members from optional metadata; nested records are listed separately.

Field	JSON type	Unit	Root constraint	Required
`name`	fixed value	metadata	const="Held-out normalized ridge"	yes
`lookback`	fixed value	observations	const=48	yes
`horizon`	fixed value	observations	const=12	yes
`alpha`	fixed value	probabilities	const=0.2	yes
`absolute_residual_radii`	array	dimensionless	minItems=12; maxItems=12	yes
`selection`	array	metadata	minItems=7; maxItems=7	yes
`normalization`	fixed value	metadata	const="subtract_last_then_divide_population_std"	no

4.9 Archived comparison: common targets with mixed execution provenance

The archived comparison boundary represents a common final-block rescoring experiment. Table 12 describes interface revision `2.1.0` and payload family v2. It records the protocol, source engine version, trace and implementation digests, full case records, summaries and input identities. Reconstructed contiguous histories and common targets distinguish it from a comparison of incompatible rolling-backtest aggregates. Targets and prediction paths have source units; MAE and RMSE retain those units, while MASE, normalized quantile loss and scaled interval score are dimensionless. Empirical coverage is a fraction. Heavy paths in this experiment remain frozen, while specified lightweight paths are recomputed. Their coexistence is explicit provenance, not a claim that every model was freshly executed. Independent numerical checks reconcile stored paths, historical scales, targets and summaries. The web consumer labels the archived experiment separately from the fresh one and does not silently use it when fresh results are unavailable. Optional later audit fields extend the schema without changing the meaning of the original calculation. A source hash binds bytes, but does not by itself establish representative sampling, absence of pretraining overlap or a justified population uncertainty interval.

Table 12: Root fields of the reevaluation contract, interface revision `2.1.0`. Types, units and structural constraints distinguish required members from optional metadata; nested records are listed separately.

Field	JSON type	Unit	Root constraint	Required
`schema`	fixed value	metadata	const="chronoscope.reevaluation/v2"	yes
`protocol`	object	metadata		yes
`source_engine_version`	string	metadata	No explicit range	yes
`trace_sha256`	object	metadata		yes
`code_sha256`	object	metadata		yes
`cases`	array	metadata	minItems=1	yes
`summary`	array	mixed; see text	minItems=1	yes
`input_sha256`	object	metadata		yes
`source_comparison_sha256`	string	metadata	pattern="^[0-9a-f]{64}$"	no
`runtime`	object	metadata		no

4.10 Fresh comparison: execution identity as an additional invariant

The fresh-comparison catalog row has interface revision `1.0.0` and intentionally uses the comparison schema rather than inventing another incompatible case format. Table 13 shows that shared

structure. Its scientific distinction is the producer and admission contract: every configured variant is fitted or inferred on the stated original history, with package, checkpoint, source and runtime records retained. Per-method fitting and inference duration is reported in seconds for the specified execution environment; it is not an environment-independent computational complexity measurement. Exact identities and the complete inventory protect the difference between a new execution and an archived path with renamed provenance. The source provider additionally verifies case identity, horizon, seasonal lag, history length and target agreement before exposing the comparison under a selected subject. The fresh target values retain their own precision for scoring and are checked against the released trace's display precision. Missing, partial or stale fresh evidence fails admission; an archived result remains an explicitly selected separate experiment. Matching aggregate values can confirm numerical recomputation while still representing the same source series and reserved outcomes, so they are not independent statistical replications.

Table 13: Root fields of the fresh-reevaluation contract, interface revision `1.0.0`. Types, units and structural constraints distinguish required members from optional metadata; nested records are listed separately.

Field	JSON type	Unit	Root constraint	Required
`schema`	fixed value	metadata	const="chronosco pe.reevaluation/v2"	yes
`protocol`	object	metadata		yes
`source_engine_ version`	string	metadata	No explicit range	yes
`trace_sha256`	object	metadata		yes
`code_sha256`	object	metadata		yes
`cases`	array	metadata	minItems=1	yes
`summary`	array	mixed; see text	minItems=1	yes
`input_sha256`	object	metadata		yes
`source_comparison_ sha256`	string	metadata	pattern="^[0-9a-f]{64}$"	no
`runtime`	object	metadata		no

4.11 Historical selector: training metadata rather than an oracle

The historical selector boundary records a method-ranking artifact produced by an offline training calculation. Table 14 describes interface revision `1.0.0`. Its kind, metric and scope fields are fixed identifiers; ranking pairs contain method names and dimensionless scaled errors. The default method is a selected identifier, and the training-case count is a count rather than a performance confidence measure. This legacy artifact is not used to assign the winner of the common reserved block. In particular, a future-informed minimum over methods is an oracle summary and must not be presented as a deployable selector merely because both outputs contain method names. The later daily-corpus selector has its own discovery and confirmation protocol in a different artifact. Keeping the boundaries separate prevents an old default ranking from acquiring that newer experiment's validation claim. A missing selector does not change the admitted case comparison. Rebuilding it is an explicit offline operation, not something startup or deployment silently performs. The producer's contract checks fixed metadata and ranking structure; meaningful generalization still depends on which cases were available for training and which outcomes remained reserved for evaluating the selection rule.

Table 14: Root fields of the selector contract, interface revision `1.0.0`. Types, units and structural constraints distinguish required members from optional metadata; nested records are listed separately.

Field	JSON type	Unit	Root constraint	Required
`kind`	fixed value	metadata	const="method-selector"	yes
`metric`	fixed value	metadata	const="mean_mase_over_ training_cases"	yes
`scope`	fixed value	metadata	const="classical"	yes

Field	JSON type	Unit	Root constraint	Required
ranking	array	mixed; see text		yes
default_method	string	metadata	No explicit range	yes
n_train_cases	integer	counts	minimum=1	yes

4.12 Daily corpus: complete accounting and partition identity

The daily-corpus boundary holds the complete source accounting, protocol, implementation and package identities, excluded records, model roster, summaries, per-series rows and audit metadata. Table 15 describes interface revision 1.0.0. Its source count is fixed at 4,227 identities; completeness is a required state rather than an inference from a large displayed table. Eligibility, duplicate handling and undefined scales can change the included denominator, so excluded identities remain part of the result. Forty prefix features have heterogeneous definitions and availability conditions. Association targets and MASE summaries are dimensionless, while feature-specific units are stated by their mathematical construction. Discovery and confirmation assignments bind exact sequence identities. Multiple-testing adjustment is performed separately for the declared feature family in each partition. Those corrected associations do not automatically imply useful method selection on new series. The producer and independent checker reconstruct the sufficient statistics and retain the unfavorable confirmation-selector result. The browser consumes the complete corpus artifact rather than substituting the fifteen selected examples. A missing or incomplete file is unavailable evidence, not an opportunity to infer the missing population from a convenient sample.

Table 15: Root fields of the forecastability contract, interface revision 1.0.0. Types, units and structural constraints distinguish required members from optional metadata; nested records are listed separately.

Field	JSON type	Unit	Root constraint	Required
schema	fixed value	metadata	const="chronosco pe.forecastability/v1"	yes
complete	fixed value	metadata	const=true	yes
protocol	object	metadata		yes
packages	object	metadata		yes
code_sha256	object	metadata		yes
source_sha256	string	metadata	pattern="^[a-f0-9]{64}$"	yes
source	object	metadata		yes
source_n	fixed value	series	const=4227	yes
included_n	integer	counts	minimum=3000	yes
excluded	array	metadata		yes
model_names	array	metadata		yes
summary	object	mixed; see text		yes
series	array	mixed; see text	minItems=3000	yes
audit	object	metadata		yes

4.13 Combination index: inventory, exclusions and compact summaries

The combination-index boundary is the compact navigation and summary layer for three cohorts: weekly, monthly and selected cases. Table 16 describes interface revision 1.0.0. Its completeness flag, experiment identity, protocol, implementation digests, package versions, source records and independent audit prevent a partial export from appearing to be the complete comparison. The case array has the expected 1,802 inventory entries, including identities excluded from scored inference. This differs from the 1,716 admitted external series and from the number of outer origins or prediction values. Browser forecast precision is fixed at eight decimal places, while the compressed numerical audit retains the underlying floating-point paths. The selected-case source binding prevents development examples from being joined to an unrelated reconstruction. Summaries average origins within series and then series within each external collection. They

therefore require their stated denominators and cannot be reconstructed by giving every origin equal weight across an entire mixed collection. The browser first reads this index and fetches individual case documents on demand. A missing study is reported as unavailable; another experiment is not substituted under the same cohort labels or method names.

Table 16: Root fields of the combination-index contract, interface revision `1.0.0`. Types, units and structural constraints distinguish required members from optional metadata; nested records are listed separately.

Field	JSON type	Unit	Root constraint	Required
`schema`	fixed value	metadata	const="chronosco pe.combination-index/v1"	yes
`complete`	fixed value	metadata	const=true	yes
`identity`	string	metadata	pattern="^[a-f0-9]{64}$"	yes
`protocol`	object	metadata		yes
`code_sha256`	object	metadata		yes
`export_code_sha256`	string	metadata	pattern="^[a-f0-9]{64}$"	yes
`packages`	object	metadata		yes
`selected_case_ source_sha256`	string	metadata	pattern="^[a-f0-9]{64}$"	yes
`cohorts`	array	metadata	minItems=3; maxItems=3	yes
`cases`	array	metadata	minItems=1802; maxItems=1802	yes
`sources`	object	metadata		yes
`browser_forecast_ decimals`	fixed value	counts	const=8	yes
`audit`	object	metadata		yes

4.14 Combination case: a source-bound origin and its allocation

The per-case combination boundary carries a source identity, dataset label, sequence length and digest, origin records and any explicit exclusion or representative relationship. Table 17 describes interface revision `1.0.0`. An admitted origin retains its supplied prefix start and length, exact prefix digest, positive historical scale, reserved targets and method records. Each forecast row contains three quantile columns, while weights, selected configuration and validation metadata expose how a combination was obtained. Forecasts, targets and scale have source units; normalized scores and allocation weights are dimensionless. Origin indices and memory positions count observations. The loader restricts case paths to the declared cohort families and verifies the returned dataset and identifier. The walkthrough additionally requires the complete little-endian float64 prefix digest, horizon, seasonal lag and target agreement before showing weights beside an active comparison. Length and subject-name agreement alone are insufficient because two different histories can have those properties in common. An excluded case can legitimately have no scored origins; its absence from a leaderboard must remain traceable to its exclusion record. A missing requested document produces an explicit loading failure rather than a generated replacement sequence or weights borrowed from another case.

Table 17: Root fields of the combination-case contract, interface revision `1.0.0`. Types, units and structural constraints distinguish required members from optional metadata; nested records are listed separately.

Field	JSON type	Unit	Root constraint	Required
`schema`	fixed value	metadata	const="chronosco pe.combination-case/v1"	yes
`id`	string	metadata	No explicit range	yes
`dataset`	fixed value	metadata	enum=["m4_weekly","m3_ monthly","selected_cases"]	yes
`source_n`	integer	observations	minimum=0	yes

Field	JSON type	Unit	Root constraint	Required
sequence_sha256	string	metadata	pattern="^[a-f0-9]{64}$"	yes
origins	array	metadata		yes
excluded	string	metadata	No explicit range	no
representative	string	metadata	No explicit range	no
seasonality	integer	observations	minimum=0	no
horizon	integer	observations	minimum=0	no

4.15 Release readiness and the judged tree

Table 18 describes the offline readiness record. The catalog digest binds the newline-normalized UTF-8 catalog text. The product version records the judged tree's VERSION value. Contract entries follow the catalog order. Each entry identifies the contract revision and its declared status. A missing list contains the required document patterns for which the judged tree contains no file. Availability records the presence of required files; schema and scientific checks independently establish whether their contents are valid. The local stage is checked with the provider from the product checkout. Missing files, stale catalog or product bindings, and changed frozen research inputs stop admission.

Each positive glob must match at least one file. Negative globs limit the document sets considered by the schema checker and add no file requirement. A positive glob that expects an entire cohort can still match a partial cohort, so the separate scientific validator checks the complete identities, bindings and counts. Readiness alone does not attest the number of forecasts or cases.

Table 18: Root fields of the readiness contract, interface revision 1.0.0. Types, units and structural constraints distinguish required members from optional metadata; nested records are listed separately.

Field	JSON type	Unit	Root constraint	Required
schema	fixed value	metadata	const="chronoscope.readiness/v1"	yes
product_version	string	metadata	pattern="^[0-9]+\\.[0-9]+\\.[0-9]+$"	yes
catalog_sha256	string	metadata	pattern="^[0-9a-f]{64}$"	yes
contracts	array	metadata	minItems=1	yes

5 System architecture and artifact admission

ChronoScope separates expensive calculation from evidence delivery. The offline producer reconstructs an admitted series, fits a declared implementation on its observed prefix, scores a separately retained target block, and writes versioned numerical artifacts. Static hosting then serves those artifacts and the browser bundle. There is no active request-time forecasting service behind the published application. The dormant API directory and the OpenAPI components used for type generation do not establish that such a service has been operated or validated. A reproducible offline experiment and a responsive browser view therefore have different execution and acceptance boundaries.

Figure 1 shows the responsibilities across these boundaries. Arrows denote artifact dependencies rather than a synchronous prediction API. The browser source is the consumer of admitted artifacts and the owner of current view state.

The first stage resolves a case identity to its manifest, source description and preprocessing policy. For committed real excerpts, the input is the exact admitted CSV. For a generated mechanism, the case definition and seed determine the observations. Reconstruction checks the original length,

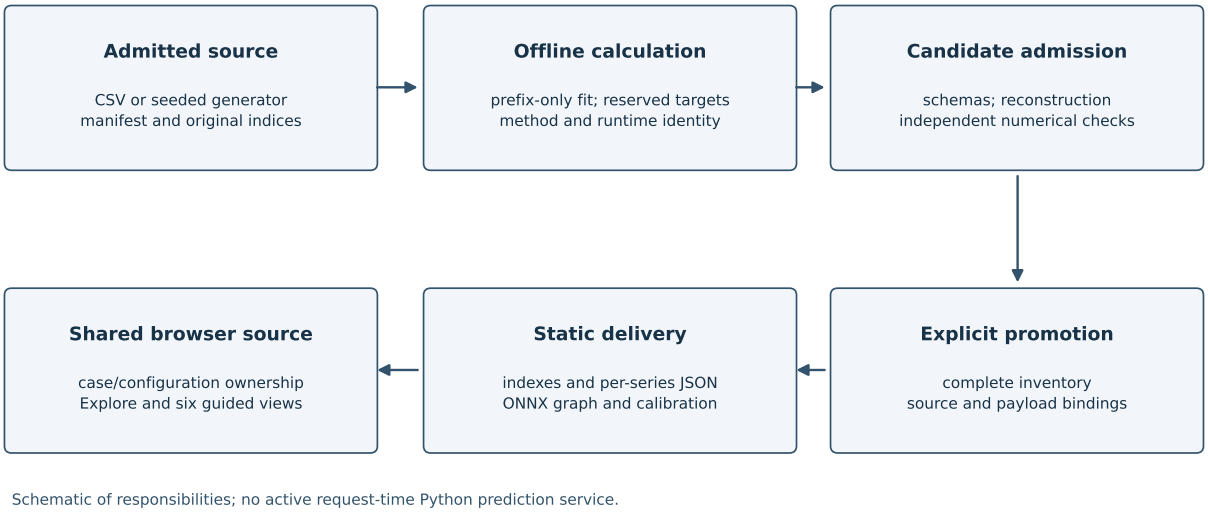


Figure 1: Offline calculation, candidate admission, explicit promotion and static browser delivery form separate evidence boundaries; the diagram describes the implemented data flow rather than a request-time forecasting service.

the retained display samples and the reserved targets before permitting a comparison. A display trace can be sufficient to draw a picture while being insufficient to reconstruct the fitting series. The second stage supplies the contiguous prefix to the chosen adapter, applying any recorded context cap. Its output contains point and interval columns with the requested horizon. Fitting failures, unavailable dependencies and malformed outputs remain failures or explicit unavailable records; they are not filled by a different algorithm under the requested method name.

The third stage joins forecasts with targets and a historical seasonal scale. It computes scores at the method–case or method–origin level before aggregating at the study's declared unit. This distinction matters: averaging overlapping origins as if they were independent observations would change both the estimand and its uncertainty. The fourth stage constructs delivery documents and compact indexes. Large external-combination records are split by source series, while the complete compressed numerical archive retains the detailed historical selection calculation. The browser index is a navigation aid rather than a replacement for that archive. Independent admission rederives targets, scales, scores and combined paths before a candidate can replace the served documents.

The fifth stage is browser ownership. A single source provider owns the URL-selected case or generator configuration. It fetches the corresponding manifest and required artifacts, rejects subject mismatches, and discards obsolete asynchronous completions. Explore and the six guided views consume this same source. A remembered combination origin may be shown alongside a forecast only when its complete contiguous prefix, length, horizon and targets match; the prefix binding hashes little-endian float64 values. A coincident case name or equal sample count does not satisfy this condition. Generated histories use local computation, and the embedded ONNX path additionally verifies its graph and calibration metadata. These are distinct execution modes, stated in the interface.

Algorithm 1 describes admission as a refusal-oriented sequence. It does not retrain an archived model or certify its scientific accuracy. The existing process also binds stage inputs and payload fingerprints so that a partial case inventory cannot acquire the identity of a complete release. A candidate directory is built and checked before promotion; creating it does not silently overwrite the published data root. This ordering permits independent inspection of a concrete candidate and preserves evidence of a failed attempt.

Algorithm 1 Admit a complete forecast artifact candidate against its declared source and execution identities before explicit promotion.

Require: Declared case inventory, exact source inputs, implementation bindings, candidate artifacts

Refuse missing, extra or duplicated case identities
for each declared case **do**
    Validate manifest, trace and analysis against their normative schemas
    Reconstruct the complete contiguous source and verify retained samples and targets
    Verify the recorded producer and input bindings without rewriting historical identities
    for each required forecast path **do**
        Check horizon, finite values, ordered interval columns and method identity
        Recompute scores using the declared targets and historical scale
        Refuse disagreement beyond the recorded numerical tolerance
    end for
    Verify optional study links against the complete prefix and target binding
end for
Check index references, expected inventory and candidate payload fingerprints
Report an admitted candidate; retain explicit promotion as a separate operation

This architecture supports independent numerical checks without making every browser request repeat model training. It also creates a responsibility to distinguish a stored result from a new execution. The archived-path comparison reuses some historical predictions, the fresh comparison executes the full roster again, and the external combination study has its own inputs and aggregation. Their schemas, provenance and consumer labels retain these distinctions. A successful schema check cannot substitute for a source reconstruction, and a successful reconstruction cannot establish representative sampling or pretraining independence. The report's evidence claims remain attached to the relevant stage rather than inherited from successful delivery.

6 Implementation identities

The method roster distinguishes algorithms from their implementations. A reference library, a direct adaptation and a small browser-exported model can share an architectural idea while differing in objectives, preprocessing and training budget. Their results remain separate.

Table 19: Forecasting implementations and their roles in the archived common-holdout comparison; a reference package, direct adaptation and browser demonstration retain distinct execution identities.

Methods	Implementation and settings	Evidence in the archived comparison
SeasonalNaive, SES, Holt, HoltWinters, Theta	NumPy core; browser TypeScript parity; finite smoothing grids	Recomputed on original contiguous prefixes. Gaussian residual intervals are approximations.
AutoETS, AutoTheta, AutoARIMA	statsforecast; ARIMA stepwise and approximate, context cap 360	Released paths rescored. Separate worked lessons freshly execute the installed adapters.
LightGBM	MLForecast lag table; 100 trees, 31 leaves, learning rate 0.05, context cap 500	Released point-regression path rescored; Gaussian residual intervals. A lesson freshly executes the adapter.

Methods	Implementation and settings	Evidence in the archived comparison
NLinear, DLinear, NHITS	Direct-torch quantile-head adaptations	Released paths rescored; not asserted to be exact replications of the papers.
NLinear (nf), DLinear (nf), NHITS (nf)	neuralforecast reference implementations; fixed 300-step training budget	Released paths rescored; the budget is part of the experiment.
Chronos-Bolt, Chronos-2, TimesFM 2.5, TiRex-2	Pretrained checkpoint adapters; TiRex bridge uses WSL/CUDA	Released paths rescored, without current-checkpoint or pretraining-independence claims.
ContextMemory, RecencyMean, EqualMean	Same five light experts; fixed causal memory and two ablations	Newly computed with matching original prefixes and final targets.

Table 19 identifies the archived comparison's implementations. The later complete fresh execution is reported separately, so a reader can distinguish rescoring an old path from executing its producer again.

The additional browser model is a separately identified normalized linear map, exported to ONNX after one global least-squares fit on generated windows. It predicts twelve observations from a forty-eight-observation prefix and is not the per-prefix NeuralForecast NLinear configuration in the twenty-two-variant experiment. Its frozen point matrix is preserved. A separate ridge-selection experiment did not improve point error; that candidate remains an experimental reference, not a replacement for the served graph.

The working browser implementation uses the independently evaluated prefix-stratified calibration described later in this report, rather than the old Gaussian scale in the legacy metadata. The calibration binds the exact graph digest, normalization, supported dimensions, roughness boundaries and finite-sample residual ranks. The browser refuses unsupported horizons, nonfinite or constant lookbacks, a changed model graph and malformed outputs. It does not silently truncate a requested longer horizon. Marginal calibration evidence comes from generated series; successful model execution on an authentic history does not establish coverage on that history's population.

6.1 Corrected lightweight methods

SeasonalNaive repeats the latest observed cycle. SES updates a level, Holt adds a trend, and additive HoltWinters additionally updates a seasonal component. At a prefix length n, the seasonal slot for zero-based future lead k must continue as $(n + k) \bmod m$. Restarting at $k \bmod m$ is incorrect when n is not a multiple of m. An adversarial partial-season example tests this condition independently in Python; the browser parity fixture includes a non-multiple history and a horizon longer than one cycle.

For the nonseasonal Theta implementation, the corrected point equation is

$$\widehat{y}_{n+h} = \ell_n + \frac{b}{2}\left[h - 1 + \frac{1-(1-\alpha)^n}{\alpha}\right], \qquad h \geq 1, \tag{3}$$

where ℓ_n is the selected SES level and b is the fitted linear slope. An independent two-Theta-line construction verifies the finite-history term following Hyndman and Billah [1]. This local method does not inherit AutoTheta's automatic seasonal adjustment merely because the names are related.

The Gaussian widening rule maps a residual standard deviation to three ordered columns at probabilities 0.1, 0.5 and 0.9. Residual estimation, Gaussian shape and horizon widening are

assumptions. Neither an ordered matrix nor a quantile-loss fit establishes marginal or conditional calibration. These distinctions also apply to the direct neural adaptations and the lag-tree adapter.

7 Forecasting equations and execution policies

The following descriptions identify the fitted computation, its observation units and its actual execution policy. A library's ability to update a latent state does not mean the current adapter exercises that ability. All outer targets remain reserved; prefix caps, parameter search and numerical postprocessing belong to the compared configuration. Reference package versions and checkpoint revisions are retained with the corresponding experiment rather than inferred from a method's display name.

7.1 Seasonal continuation and local smoothing experts

The five local experts share a finite observed prefix and a requested observation-count horizon. They supply the matching inputs to the fixed and temporally selected combinations, and the browser implements the same point equations for generated examples. This parity does not imply that every browser score equals an offline score when the supplied history or scale differs. The pipeline first requires the experiment's admitted sequence. Interpreting decimated samples as consecutive observations would change the model, even if its implementation passed an arithmetic fixture.

SeasonalNaive continues the last observed seasonal cycle. With one-based lead j and a zero-based sequence index, the future seasonal phase is $(n + j - 1) \bmod m$. It therefore remains correct when the prefix ends partway through a cycle. The computational work for the point path is linear in the requested horizon after the relevant tail has been identified. Residual dispersion comes from observed lag-m differences. A Gaussian interval constructed from those differences is an uncertainty approximation, not a distribution-free property of seasonal copying.

Simple exponential smoothing keeps a level and minimizes the implemented in-prefix one-step squared error over its fixed coefficient grid. With level ℓ_t and observation y_t, its update and point forecast are

$$\ell_t = \alpha y_t + (1 - \alpha)\ell_{t-1}, \qquad \widehat{y}_{n+j} = \ell_n. \tag{4}$$

The initial level is the first observation. The coefficient grid runs from .05 to .95 in increments of .05, with deterministic first-minimum selection. This is a finite search, not a claim of global continuous-parameter optimality. For a fixed grid its fitting cost grows linearly with prefix length. The level has source unit U and the coefficient is dimensionless. Residual variance and final interval width still depend on how well a level-only description represents the supplied sequence.

Holt introduces an additive trend b_t with units U per observation. Its equations are

$$\ell_t = \alpha y_t + (1 - \alpha)(\ell_{t-1} + b_{t-1}), \tag{5}$$

$$b_t = \beta(\ell_t - \ell_{t-1}) + (1 - \beta)b_{t-1}, \qquad \widehat{y}_{n+j} = \ell_n + jb_n. \tag{6}$$

The initial trend is the first observed difference. The implementation searches nine level coefficients from .1 to .9 and six trend coefficients from .05 to .5. It does not use a damped trend or select an arbitrary damping parameter. Extrapolation can therefore continue a recent trend too far after a break. Fitting every coefficient pair on the same prefix makes the finite objective reproducible but cannot ensure that its in-prefix preference remains appropriate for the reserved block.

Additive HoltWinters adds seasonal offsets s_t in source units [9]. The implemented component

updates are

$$\ell_t = \alpha(y_t - s_{t-m}) + (1-\alpha)(\ell_{t-1} + b_{t-1}), \tag{7}$$
$$b_t = \beta(\ell_t - \ell_{t-1}) + (1-\beta)b_{t-1}, \tag{8}$$
$$s_t = \gamma(y_t - \ell_t) + (1-\gamma)s_{t-m}. \tag{9}$$

The first cycle initializes level and offsets; the first two cycle means initialize trend. Four level coefficients, two trend coefficients and three seasonal coefficients form the fixed search. The residual objective begins after the first cycle, and a short or nonseasonal history falls back to Holt. A particular seasonal lag is supplied, not inferred by this expert. A wrong lag can therefore harm both the forecast and its residual-based interval.

The finite-history Theta equation given earlier uses the selected SES level and a fitted linear slope [1]. Its correction term matters on short prefixes; replacing it by an arbitrary half-drift continuation changes the method. The five experts retain Gaussian residual endpoints on the specified probability grid. Ordered endpoints follow the implemented construction, while calibration remains an empirical question. Degenerate residuals, finite parameter grids and positive affine unit changes deserve explicit checks. These local experts provide transparent baselines and combination components; they do not establish that a short observed series admits an identifiable trend, stable seasonality or a calibrated future distribution.

For a fixed seasonal lag, evaluating a single SES or Holt coefficient tuple takes one pass through the prefix and a linear pass through the requested horizon. HoltWinters additionally retains one cycle of seasonal state. The finite coefficient grids multiply this cost by their recorded candidate counts; they are not continuous optimizers. Forecasting with an incorrect seasonal lag, a near-zero residual scale or an abrupt trend reversal can still fail despite an exact implementation. In particular, a zero residual history is not evidence that all future observations are known. The interval approximation and the point recurrence must therefore be evaluated separately, and a numerical parity fixture cannot establish distributional correctness.

Algorithm 2 gives the common evaluation order for the local experts. Their candidate grids and recurrences are the ones specified above.

Algorithm 2 Fit and forecast the local seasonal continuation and smoothing experts on one observed prefix.

Require: Observed prefix, seasonal lag m, horizon H and probabilities Q

Continue the last cycle for SeasonalNaive and estimate dispersion from lag differences
for each of SES, Holt and admissible HoltWinters **do**
    Initialize the level, trend and offsets required by that method
    Run its recurrence for each grid candidate and retain the first minimum of residual SSE
    Extrapolate its terminal components for H leads and retain residual dispersion
end for
Use Holt for the HoltWinters result when its seasonal-history requirement fails
Build Theta from the selected SES fit, the prefix linear slope and the finite-history correction
Add Gaussian residual columns using the square-root-of-lead scale and return ordered paths

7.2 Automatic ARIMA, exponential smoothing and Theta selection

The automatic statistical adapter calls StatsForecast's AutoARIMA, AutoETS and AutoTheta implementations [10]. These are separate methods from the local finite-grid experts. The package owns its candidate models, optimization and interval construction. Passing the same seasonal count does not make the local Theta equation an equivalent implementation of AutoTheta, nor does an automatic label establish that every plausible forecasting model was searched. The

published forecasting literature supplies the model families and selection motivation; the installed package and experiment records define the executed configuration [35, 36].

A seasonal ARIMA family can be expressed using a backshift operator B as

$$\phi(B)\Phi(B^m)(1-B)^d(1-B^m)^D y_t = c + \theta(B)\Theta(B^m)\epsilon_t. \tag{10}$$

Here d, D are differencing orders, lower- and upper-case polynomials describe ordinary and seasonal autoregressive or moving-average terms, and innovations ϵ_t have source units. Model identification, stationarity constraints, root locations and innovation assumptions matter to forecasts and intervals. Differencing removes specified low-frequency components; it is not proof that the transformed future process will remain stationary. An estimated linear stochastic model also does not inherit coverage guarantees when its innovation distribution or stability assumptions fail.

The current AutoARIMA adapter requests stepwise search and approximate selection calculations. It caps the supplied history at the final 360 observations and uses a smaller historical backtest-window budget than the cheaper automatic methods. Those settings bound its cost; they can also alter its fitted result. A stepwise search is not exhaustive minimization over all order tuples. The common final-block experiment aligns reserved outcomes, but does not erase the context-cap difference or convert historical rolling aggregates with different origin counts into a common experimental budget.

Exponential smoothing can be written as an innovations state-space family. An additive level-and-trend member illustrates its form:

$$y_t = \ell_{t-1} + b_{t-1} + \epsilon_t, \tag{11}$$

$$\ell_t = \ell_{t-1} + b_{t-1} + \alpha\epsilon_t, \qquad b_t = b_{t-1} + \beta\epsilon_t. \tag{12}$$

AutoETS selects among the package's admissible error, trend and seasonal structures; the equations above do not assert that every case selected this particular additive member. Multiplicative structures have additional positivity and distributional implications. The package's available model family and numerical fit determine whether a candidate is admissible. Automated selection can be useful when manual specification across many series is impractical, while remaining sensitive to sample size, initialization, objective choice and a structural break near the forecast origin.

For all three methods, the adapter fits a new model on each supplied prefix. It obtains the library's central point output from its mean field. For the canonical .1 and .9 columns it requests the corresponding central eighty-percent band. Calling the central output a median quantile would require assumptions beyond the field name; the report treats it as the package point forecast placed in the central column. The final cumulative maximum enforces column order. That repair is numerical postprocessing, not evidence that the columns equal the true conditional quantiles of an unknown data-generating process.

The library may support efficient state updates in other applications, but the evaluated adapter's interface uses a new fit and predict call. Its timings therefore cannot establish the performance of a native online state-update service. Computational work includes candidate search and parameter optimization as well as the recurrence associated with the selected model. JIT initialization, cached code, hardware and process policy can affect recorded latency. The shared targets and independently rescored paths support the finite accuracy comparison; they do not supply resource-normalized superiority, exhaustive search optimality or a universal forecast ranking.

Reproduction should record the supplied observations and the package version rather than attempting to infer an automatic configuration from its displayed score. A finite point path can survive an optimization warning; the warning is retained so the reader can distinguish a returned forecast from evidence of a well-converged fit. Conversely, a failed fit cannot become a successful experiment by copying another method's output. Short prefixes, multiplicative positivity restrictions and a seasonal period large relative to history are specific stress cases. The current

selected-case result does not quantify how frequently these conditions occur in an operational population. Those frequencies and the cost of refitting at each arrival would have to be measured in a separate deployment study.

Algorithm 3 separates package model selection from application quantile-column assembly.

Algorithm 3 Refit each automatic StatsForecast adapter and assemble a shared forecast array at one evaluation origin.

Require: Observed prefix, seasonal lag m, horizon H and method identity

1: For AutoARIMA cap the prefix at 360 observations and enable stepwise approximate search
2: Instantiate the requested AutoARIMA, AutoETS or AutoTheta family with its seasonal count
3: Fit that family on the supplied observations and request H forecasts with an 80 percent interval
4: Assemble the lower endpoint, mean path and upper endpoint as three aligned columns
5: Apply cumulative maxima across columns and retain the fit's recorded warnings

7.3 Recursive lag-tree regression and approximate intervals

The lag-tree adapter uses LightGBM through MLForecast to fit a point regressor to one observed series [37, 38]. The transformation is explicit: earlier values become input columns and the current observed value becomes the training target. It does not use the final reserved outcomes during fitting. The adapter constructs integer observation indices rather than genuine calendar features, and declares no static regressors. Consequently, its label should not imply that holidays, promotions, weather or future exogenous variables were supplied merely because the surrounding application can discuss those factors.

Let $\mathcal{L}$ be the selected lag set. The fitted one-step mapping and recursive multi-step use can be represented as

$$\widehat{y}_t = F_\theta((y_{t-\ell})_{\ell\in\mathcal{L}}), \qquad \widehat{y}_{n+j} = F_\theta((\widetilde{y}_{n+j-\ell})_{\ell\in\mathcal{L}}), \tag{13}$$

where $\widetilde{y}_t$ is an observed prefix value when $t \le n$ and a previously generated prediction otherwise. This definition makes the origin's information boundary clear. It also exposes error propagation: after the first lead, some inputs are model outputs, not newly observed truth. An unusually accurate one-step fit does not establish stability of that recursion over a longer horizon or under a regime not represented in the supplied prefix.

The candidate lags begin with one, two and three. When the seasonal period is at least two, the candidates also include m, $m+1$ and $2m$. Duplicates are removed, and lags too close to the available prefix length are excluded using the implemented training-row allowance. The context cap is five hundred observations. Thus the number of retained lag columns and usable rows can differ across cases. A seasonal lag is a count of observations, while the lag values and regression target have the same source unit U. This adapter does not estimate a new seasonal frequency from the reserved targets.

The configured regressor has one hundred trees, at most thirty-one leaves, a learning rate of .05, a minimum child sample count of five and one worker thread. A generic squared-error boosting description is

$$F_T(x) = F_0(x) + \eta \sum_{r=1}^{T} f_r(x), \qquad \eta = .05, \quad T = 100. \tag{14}$$

Each fitted tree approximates an improvement to the current loss under the package's split and regularization policy. The LightGBM paper describes efficient implementation techniques, but this application does not separately reproduce the paper's speed comparisons or assert that every optional optimization is enabled. Fixing a tree count is a computational budget, not proof that

the fitted function reached a global minimum or that a more expensive fit would improve the held-out criterion.

MLForecast is asked to retain fitted values. The adapter computes the population standard deviation of its in-sample one-step residuals, then applies the shared Gaussian horizon-widening rule around the recursive point path. Those residuals have already participated in fitting. Their dispersion can therefore be optimistic as an estimate of future uncertainty, and the recursive multi-step error distribution need not have the same shape. No native quantile-objective models are fitted by this adapter. The separate browser calibration experiment does not calibrate these tree intervals or alter their source identity.

A prediction call traverses the fitted trees at each future lead; a useful descriptive bound is proportional to horizon, tree count and visited tree depth. Training additionally depends on available rows, bins, feature count and the split search. The result is not constant-time forecasting in the observed history. The complete comparison retains both point and interval scores so that an apparent point improvement cannot conceal poor uncertainty quality. A faithful future change could evaluate direct quantile objectives or a separately calibrated residual procedure, but it would require its own method identity and held-out evaluation rather than rewriting the outcomes of this configured regression model.

The regressor's interval is consequently a separate approximation layered on a deterministic recursive mapping. Its training residuals are fitted residuals rather than errors from a held-out calibration sample, and their empirical scale may be optimistic. Heteroscedasticity, a changing level or accumulating recursive error can invalidate a common Gaussian shape even when its endpoints remain ordered. Increasing the number of trees would change the point model but would not by itself repair this uncertainty construction. A fair extension would compare a declared residual-calibration or quantile procedure on an untouched future block, retaining its information cutoff, resource budget and source units. That extension has not been substituted for the admitted tree result.

Algorithm 4 makes explicit which values can enter the recursive prediction.

Algorithm 4 Fit the observed-prefix lag tree and extend it recursively over the common reserved block.

Require: Observed prefix, seasonal lag m, horizon H and probabilities Q

1: Retain the last 500 observations and form the eligible lag columns
2: Train the declared LightGBM configuration through MLForecast with fitted values enabled
3: Estimate residual dispersion from the fitted one-step values
4: **for** $j = 1$ to H **do**
5: Evaluate the tree on lagged observations and earlier forecasts available at this lead
6: Append the resulting point to the working history
7: **end for**
8: Form ordered Gaussian residual columns around the point path

7.4 Quantile-head neural adaptations and reference implementations

The neural roster distinguishes three direct PyTorch adaptations from three NeuralForecast implementations. The source papers motivate NLinear, DLinear and N-HiTS [39, 40]; the package supplies its maintained model classes [41]. Matching a name does not prove matching architecture, optimization, preprocessing or training budget. The report therefore retains the direct and library paths as separate experiment identities. Neither a favorable direct adaptation nor an unfavorable short-budget library fit is a replication of every result in the original paper.

The direct adapter standardizes its supplied observed prefix using the prefix mean μ and population

standard deviation s. For $s > 0$, the normalized sequence and inverse transform are

$$z_t = (y_t - \mu)/s, \qquad \widehat{y} = \mu + s\widehat{z}. \tag{15}$$

Only prefix observations enter those quantities. The implementation uses a unit denominator when the standard deviation is exactly zero; that is a numerical fallback, not an empirically calibrated uncertainty model for constant series. It must not be confused with the browser linear consumer, which refuses a constant required lookback. Sliding training inputs and their future blocks are drawn entirely inside the supplied prefix. The final reserved outer block is not a training window.

A direct NLinear head subtracts the last value of each input window, applies one affine map and adds that value back to every lead and quantile:

$$\widehat{Z} = \mathrm{reshape}_{H\times|Q|}\{W(x - x_L\mathbf{1}) + b\} + x_L. \tag{16}$$

The output dimension is horizon times probability count, rather than the original single point path. DLinear instead forms a moving-average trend and a remainder, applies one affine map to each and sums their outputs. Its moving-average kernel is at most twenty-five and is adjusted to an available odd length; replicated edge padding remains inside the input window. The branch called seasonal in the implementation is the remainder after this moving average. It is not an estimated physical seasonal component with guaranteed period m.

The direct method labelled NHITS is a compact multi-rate MLP adaptation. It pools the input at rates four, two and one, sends each pooled vector through two hidden layers of width 128 with ReLU activations, and sums their horizon-by-quantile outputs. Its forward computation does not contain the full paper's hierarchical interpolation or an explicit residual-backcast chain. Accordingly, the description treats it as a separately documented adaptation, not a faithful reproduction of that architecture. This distinction also explains why the maintained library variant remains an independent comparator.

All three direct adaptations minimize the implemented multi-quantile pinball objective,

$$\mathcal{L}(\theta) = \frac{1}{BH|Q|}\sum_{i=1}^{B}\sum_{j=1}^{H}\sum_{q\in Q}\max\{qe_{ijq}, (q-1)e_{ijq}\}, \qquad e_{ijq} = z_{ij} - \widehat{z}_{ijq}. \tag{17}$$

The supplied probabilities are .1, .5 and .9. The direct optimizer is Adam with learning rate .001 and a three-hundred-step budget. Each minibatch draws at most sixty-four training windows with replacement using a local seeded generator. Initial model weights are seeded separately through PyTorch. Those settings define the fit; they do not guarantee a global optimum, training convergence or bitwise equality across devices and library versions. The lookback prefers the larger of twice the horizon and three seasonal periods, capped to leave the required in-prefix training allowance. An insufficient lookback is an error rather than an invented fit.

The reference adapter instantiates NeuralForecast's corresponding classes with its multi-quantile loss, the declared input length, seed and three-hundred-step budget. It uses artificial hourly timestamps to supply regular index spacing, not real calendar semantics. GPU selection depends on the recorded runtime; CPU fallback is a distinct execution environment. The adapter resolves the package's output-column naming conventions for each requested probability and fails if a column cannot be identified. It then applies the same nondecreasing column repair used by other adapters:

$$\widetilde{q}_j^{(r)} = \max_{a\le r}\widehat{q}_j^{(a)}. \tag{18}$$

This cumulative maximum can change a central or upper column when raw heads cross. It preserves order but is not an isotonic least-squares projection and does not prove probabilistic calibration. The separate browser ONNX map has a global generated-window least-squares training identity

and an independently studied residual calibration; it is neither one of these per-prefix neural fits nor a replacement for their empirical interval results.

Computational cost depends on the implementation as well as the architecture name. The direct affine models have parameter counts proportional to input length times horizon times the number of probability heads; the pooled MLP adaptation adds hidden-layer products at each pooling rate. Training multiplies per-minibatch work by the fixed three-hundred-step budget. The maintained library can use different internal batching and device policies. None of these finite budgets constitutes a convergence certificate. Reporting them makes a repeated experiment interpretable and avoids attributing a budget-specific failure to an entire model family.

Algorithm 5 distinguishes the direct adaptations from the maintained reference package fit.

Algorithm 5 Fit direct or maintained neural quantile heads using training windows contained in the observed prefix.

Require: Observed prefix, lookback L, horizon H, probabilities Q and seed
1: **if** the method uses a direct adaptation **then**
2: Standardize the prefix, extract fitting windows and initialize the selected head
3: Apply 300 Adam steps to seeded minibatches with the pinball objective
4: Predict from the final normalized window and undo the prefix normalization
5: **else**
6: Construct the regular index and instantiate the named NeuralForecast method
7: Fit with the declared loss and step budget, predict, and resolve the requested columns
8: **end if**
9: Refuse missing output columns and enforce nondecreasing columns on the aligned array

7.5 Pretrained checkpoint adapters and their information limits

The pretrained roster contains Chronos-Bolt, Chronos-2, TimesFM 2.5 and TiRex-2. The local adapters load existing checkpoints and request forecasts; they do not fine-tune those weights on each selected case. Primary model descriptions explain the originating model families, while repository APIs and exact checkpoint revisions identify the executed variants [42, 12, 43, 13, 14]. A paper about an earlier family does not by itself establish the architecture, licence or numerical identity of a later checkpoint. The admitted runtime records therefore retain checkpoint provenance separately from the human-readable method name.

At an outer origin, the adapter supplies a capped contiguous suffix of the observed prefix. A schematic description is

$$\widehat{Q} = \mathcal{M}_{\theta^*}(y_{n-C+1:n}; H, Q), \qquad C \le n, \tag{19}$$

where θ^* is a retained pretrained parameter set, H is the requested horizon and Q is the canonical probability grid. The prefix cap is part of the configuration. The expression is an interface definition, not an assertion that every checkpoint uses the same normalization, distribution head or internal computation. A common reserved block can align evaluation outcomes while the models consume different context lengths and possess very different external training histories.

Transformer-family attention is commonly represented as

$$\mathrm{Attention}(Q_a, K_a, V_a) = \mathrm{softmax}(Q_a K_a^\top / \sqrt{d}) V_a. \tag{20}$$

Here the subscript distinguishes attention matrices from the forecast probability set, and d is an internal feature dimension. This equation describes a model component, not every operation in the Chronos or TimesFM checkpoint. Original Chronos uses scaling, quantization and language-model training; later family APIs and output heads must be identified separately [42, 12]. TimesFM's

originating description uses a patched decoder, while the evaluated 2.5 release has its own package interface and continuous quantile head [43, 13]. The report does not transplant benchmark scores from those papers into the selected ChronoScope cases.

TiRex-2 instead uses a recurrent xLSTM-based design [14]. Its originating paper reports streaming and multivariate capabilities, including covariate handling. Those capabilities must not be denied merely because the present adapter evaluates a univariate prefix interface. Conversely, loading that checkpoint for a full-prefix forecast does not mean this experiment exercised or timed its native incremental state update. The separate historical ACI/PID-style replay is also not a test of TiRex-2's streaming architecture. The mathematical distinction between a recurrent state and an external repeated-fit wrapper remains essential to interpreting the app's method names and timing plots.

The Chronos adapter maps requested quantile outputs from the package's supported tensor forms into a horizon-by-probability matrix. The TimesFM adapter maps the evaluated deciles to the corresponding output columns; the canonical .1, .5 and .9 requests are exact deciles. General non-decile behavior is not established by this experiment. The TiRex adapter explicitly rejects unsupported probability levels. A common final cumulative maximum enforces nondecreasing columns where applicable. None of these shape or ordering operations establishes that reported marginal probabilities are calibrated on the selected sources.

Checkpoint loading is cached within the relevant process so repeated windows can reuse a model. That cache changes loading cost, not the observations available at an origin. A timing comparison would need to distinguish cold loading, warm inference, process bridges, host memory and device transfers. The WSL/CUDA path for TiRex is an execution policy and cannot be treated as equivalent hardware to a native CPU call without measurement. The complete fresh comparison records package, checkpoint and source identities, but it is still a finite collection with one reserved block per selected series. Undocumented pretraining overlap, domain shift, training-corpus differences and configuration-specific context limits remain unresolved. The observed ranking therefore describes the admitted paths and targets; it does not certify universal zero-shot superiority or operational suitability.

For authentic sources, a further concern is information that entered the checkpoint before this experiment began. Hashing the input prefix proves the local inference input and cannot prove that the source was absent from pretraining. Likewise, a permissive software licence does not determine the terms of every weight release. The report retains the admitted checkpoint identity and its evaluated interface instead of silently upgrading to a newer family member. Any replacement would require new licence and configuration review, fresh matched-target execution and new evidence bindings before its score could replace the current result.

Algorithm 6 describes the admission and inference steps shared by the retained checkpoint comparisons.

Algorithm 6 Run a declared pretrained checkpoint on a contiguous prefix and retain aligned forecast columns.

Require: Checkpoint identity and revision, runtime, observed prefix, horizon H and probabilities Q

1: Resolve the declared runtime and the checkpoint cache entry for that revision
2: Supply the permitted contiguous suffix, excluding all reserved outcomes
3: Request quantile forecasts through the checkpoint's declared univariate interface
4: Map returned columns to their declared probabilities and refuse unsupported shapes or levels
5: Preserve the resulting paths and provenance for independent scoring

8 A fixed context-memory proposal

ContextMemory tests whether similar completed historical contexts help estimate expert competence beyond recency alone. Feature-dependent combinations and dynamic ensembles are established ideas [4, 5]. The experiment tests a specific within-series recurrence rule and makes no priority claim for contextual weighting.

The expert set is fixed to SeasonalNaive, SES, Holt, HoltWinters and Theta. Up to twelve completed historical origins are selected within the prefix. For every origin o, eligibility requires $o + H \leq n$. A normalized half-window mean contrast, level shift, volatility-change statistic and seasonal correlation describe the context. Expert multi-step absolute losses are normalized using information available at that origin. A Gaussian context-distance kernel combines with recency weights of half-life six origins. Local estimated losses are shrunk 25% toward the recency-only loss estimate.

An exponential loss transformation produces a simplex allocation, followed by a 5% uniform floor. With $K = 5$ experts, each final weight is at least $0.05/K = 0.01$ and the weights sum to one. Corresponding point and quantile columns are averaged with these same weights. Nonnegative convex weights preserve the order of the input quantile columns. They do not make the result a calibrated predictive mixture.

RecencyMean removes context similarity while retaining the experts, historical origins, recency weighting and competence transformation. EqualMean assigns a uniform allocation. These matching ablations distinguish the contextual component from the more general benefit of combining methods. A positive scale and location transformation supplies a structural check away from numerical degeneracy thresholds. Completed-target eligibility and quantile ordering have independent adversarial tests.

The fixed-allocation means in Figure 2 retain the original contextual rule and both matching controls; their ordering does not establish a benefit from the added kernel.

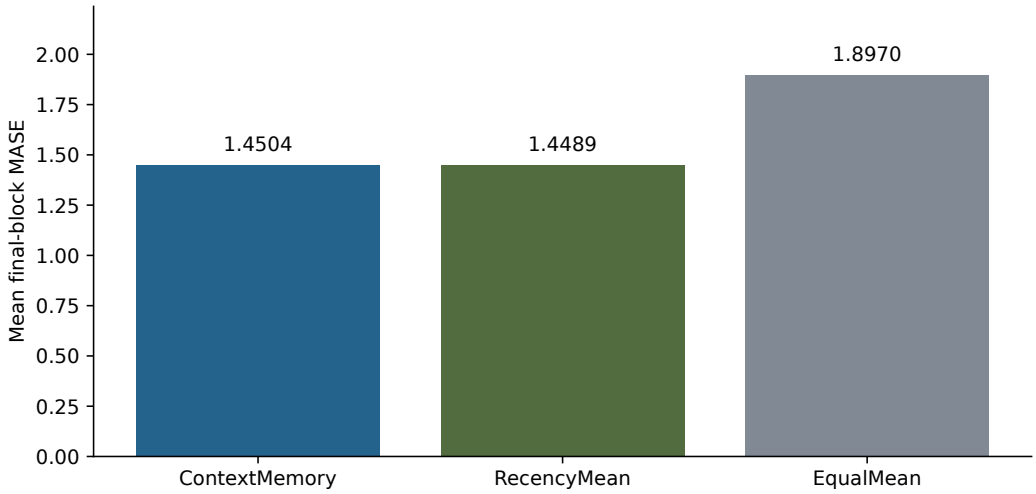


Figure 2: Fixed matching-expert combinations on the corrected fifteen-case protocol. Lower mean MASE is preferable. The contextual component does not improve on the recency-only ablation. Bars summarize selected final blocks, not independent population replications.

This section gives a complete mathematical interpretation of the implemented selection procedure. The proposal is the finite configuration tested in this paper, not a new general principle of stacking. Its input is one contiguous observed prefix, a positive seasonal lag and a positive forecast horizon. Its output includes three quantile columns, an expert allocation, the selected

configuration, historical origins and validation records. There is no argument through which the caller can supply the outer reserved targets. Rejecting invalid or nonfinite input is part of the computation, rather than a reason to omit a difficult case after seeing its score.

8.1 Three distinct information boundaries

Write $\mathcal{F}_q = \sigma(y_1, \ldots, y_q)$ for the information supplied by the series through query q. The memory grid $\mathcal{O}_n$ is fixed from the length of the outer prefix and the declared lag and horizon. A validation query $v \in \mathcal{O}_n$ may use only the completed subset

$$\mathcal{T}_v = \{o \in \mathcal{O}_n : o + H \leq v\}. \tag{21}$$

The rule requires at least three members before scoring that query. The validation origin's own context is computed from its prefix; its expert forecasts are computed from that same prefix. Historical loss rows and normalized signed errors used to fit its candidate weights come only from $\mathcal{T}_v$. The candidate's validation outcome is then formed from the block after v. That block is a target at this inner stage, although it is already observed at the final outer origin n.

Consequently, the whole inner validation calculation is available when the outer fit is issued. Provided the expert implementations are deterministic functions of their supplied prefixes and fixed configuration, the final weights are $\mathcal{F}_n$-measurable. This is an information-order statement. It does not assert that historical targets are statistically independent of their prefixes or of each other. Nor does it assert that a chronological split makes a changing process stationary. It simply specifies which observations can participate in each role and rules out use of the outer reserved block in fitting or selection.

There is a subtle limitation to the grid that must remain explicit. The origin grid is constructed from the supplied outer-prefix length. We then filter that grid at each historical query; we do not claim to recreate a historical service that would have selected its own independent grid knowing only its then-current length. The selected errors are chronologically available, but their sampling schedule is an offline evaluation design. A deployment requiring exact historical service reconstruction should issue and retain each prediction prospectively instead of interpreting this capped grid as such a log.

8.2 Candidate scores and deterministic selection

Let $\mathcal{V}_n$ be the validation queries with at least three completed training blocks. For a candidate c, let $w^{(c,v)}$ be its allocation fitted using those blocks, and let E_{vhk} be expert k's signed error at query v, lead h, divided by the query-prefix training scale. The recorded criterion is

$$S_n(c) = \frac{1}{|\mathcal{V}_n|} \sum_{v \in \mathcal{V}_n} \frac{1}{H} \sum_{h=1}^{H} \left| \sum_{k=1}^{K} E_{vhk} w_k^{(c,v)} \right|. \tag{22}$$

Every candidate is scored on the same eligible queries. The selected configuration is the first configured minimizer of this criterion. Selection is then followed by a new weight fit using every completed memory block in the outer prefix. Historical validation estimates the performance of the allocation rule; its particular inner coefficient vector is not copied as if it were necessarily the correct final vector.

If no validation query qualifies, there is no historical selection estimate to minimize. The implementation records unavailable candidate means and explicitly selects the original recency-only candidate. That fallback can still allocate weights from any completed outer memory that exists. If the memory itself is empty, the allocation is uniform. Distinguishing these two cases matters: absence of an eligible validation split is not identical to absence of historical expert losses.

Neither case justifies inventing a favorable validation score or replacing the fallback with the best outer-test method.

For a contextual candidate, let g_k and ℓ_k be recency-weighted and context-and-recency-weighted normalized losses. Its shrinkage ρ and temperature τ determine

$$c_k = (1-\rho)\ell_k + \rho g_k, \qquad v_k = \frac{\exp\{\mathrm{clip}[-(c_k - \min_j c_j)/\tau, -700, 0]\}}{\sum_l \exp\{\mathrm{clip}[-(c_l - \min_j c_j)/\tau, -700, 0]\}}. \tag{23}$$

Subtracting the minimum does not change exact softmax ratios and ensures that at least one unnormalized term is one. Clipping bounds numerical underflow in the remaining terms. Temperature changes concentration; shrinkage changes which loss estimate is concentrated. At $\rho = 1$, the context distance has no effect. Thus selecting a configuration whose name belongs to the contextual family is not evidence that its kernel contributed to prediction quality.

8.3 Regularized stacking on a simplex

For stacking, arrange all completed training errors into a matrix A with $N_e = JH$ rows and $K = 5$ columns. Let $u = \mathbf{1}/K$ and $G = A^\top A/N_e$. The unfloored objective can be written

$$F(w) = w^\top G w + \lambda(w-u)^\top(w-u), \qquad w \in \Delta_K = \{w \geq 0 : \mathbf{1}^\top w = 1\}. \tag{24}$$

The Gram term preserves correlations and relative signs of expert errors. A candidate can therefore benefit from cancellation even when no single expert has uniformly small absolute loss. The penalty trades that fitted cancellation against proximity to equal allocation. Its fixed positive grid is part of the evaluated algorithm. Changing the penalty grid, expert set, memory schedule or error normalization produces another configuration, even if the public method label were left unchanged.

Because G is positive semidefinite and $\lambda > 0$, the Hessian is

$$\nabla^2 F = 2(G + \lambda I) \succeq 2\lambda I. \tag{25}$$

The objective is strictly convex, and the nonempty compact simplex supplies a unique exact minimizer. This follows from standard convex optimization [44]; it is not a new statistical generalization theorem. Strict convexity does not imply good future forecasts, well-estimated covariances or correct interval coverage. It gives a well-defined fitting problem whose numerical answer can be checked independently of those empirical questions.

Five columns make exhaustive face enumeration practical. For a nonempty active set B, define $M_B = G_{BB} + \lambda I$, $x = M_B^{-1}\lambda u_B$ and $z = M_B^{-1}\mathbf{1}$. The equality-constrained stationary point on that face is

$$w_B = x + \frac{1 - \mathbf{1}^\top x}{\mathbf{1}^\top z} z, \qquad w_{B^c} = 0. \tag{26}$$

Positive definiteness makes the denominator positive in exact arithmetic. The implementation solves the two right-hand sides together, rather than explicitly forming an inverse. A face with a coefficient below -10^{-10} or a nonfinite answer is rejected. Accepted coefficients are clipped only within the stated numerical feasibility tolerance and normalized before evaluating the original objective. All 31 nonempty faces are considered, including single-expert vertices, and a feasible equal allocation initializes the objective comparison.

Any simplex minimizer has a nonempty support. Its restriction to the relative interior of that support satisfies the corresponding equality-constrained first-order condition, while a boundary optimum appears on a smaller enumerated face. Therefore evaluating all feasible face solutions recovers the exact constrained optimum under exact arithmetic. In floating-point execution, tolerances and linear-system failures remain numerical concerns. Retaining feasibility, objective

and independent reference checks is more informative than treating the absence of an iterative optimizer as an automatic certificate of numerical correctness.

The common uniform floor is applied after fitting:

$$\widetilde{w} = .95w + .05u. \tag{27}$$

The final allocation lies inside the simplex and each expert receives at least .01. It generally does not minimize F; the objective was solved for w, not for its floored transform. Likewise, the candidate is selected by scaled absolute validation error, not by F. These are two separate layers: a squared-error regularized fitting rule within a candidate and an absolute-error historical selection rule across candidates. Reporting both prevents the word optimal from obscuring which optimization was actually performed.

8.4 What convex allocation guarantees

At a fixed observation, convexity of absolute value yields

$$\left| y - \sum_k w_k p_k \right| \leq \sum_k w_k |y - p_k|. \tag{28}$$

The right side is a weighted average of component losses. It is not their minimum, and the inequality does not establish that the combination beats the best expert. The error may still be larger than that of an individual method or an equal average. Averaging over a horizon preserves the inequality for fixed weights, but does not turn historical weight estimation into an oracle for future errors. This limited consequence is useful for interpreting the algorithm without promising an empirical ranking.

For the same reason, positive expert weights preserve the order of already ordered quantile columns, but do not make their weighted columns a calibrated predictive distribution. Our experts' Gaussian residual approximations can be too wide, asymmetric relative to actual errors, or poorly adapted to horizon-specific uncertainty. A weight rule selected for absolute point error has no direct objective encouraging nominal marginal coverage. The observed high coverage and poor MSIS of several combinations are therefore substantive evidence about this construction, rather than a contradiction of convexity.

Finally, normalized features and errors explain why the allocation aims to be stable under a positive affine unit conversion. Away from degeneracy thresholds, translating the series cancels from mean contrasts and errors, while multiplication by a positive scale cancels from the corresponding standard deviations and seasonal error denominators. The expert equations must also transform consistently. The implemented absolute tolerances and finite precision require explicit exceptions near zero scales; we do not infer universal unit invariance from symbolic cancellation alone. Final evaluation retains unavailable MASE for a seasonal scale at most 10^{-12} in source units instead of silently using the training fallback.

The mathematical properties above support deterministic, inspectable fitting. They do not supply the missing independent comparator experiments, prospective confirmation on a deployment population, or an isolated attribution of gains to context. Those remain empirical questions. Complete validation records and retained adverse comparisons make the next experiment more precise: one can change a single allocation ingredient, preserve its information contract, and evaluate whether the previously observed point gains survive without reusing the new target block for selection.

8.5 Computational and memory requirements

Let K be the expert count, J the number of remembered origins, $V \leq J$ the eligible inner-query count, $d = 4$ the context dimension, and $Q = 3$ the retained quantile count. Write $F_k(n_o, H)$ for the actual fitting and forecasting work of expert k at prefix length n_o. Building the memory requires $\sum_{o \in \mathcal{O}_n} \sum_{k=1}^{K} F_k(n_o, H)$, in addition to context extraction and array construction. For the finite smoothing grids used here, the recursive fits are linear in prefix length with a fixed grid-size multiplier. They are fresh finite-prefix fits, not constant-time native state updates. The final expert paths require another fit on the complete supplied prefix.

Memory stores context rows, expert losses, normalized error rows and all historical quantile paths. Its numerical-array requirement is $O(Jd+JK+JHK+JKHQ)$, dominated by the historical paths when H grows. The local LRU cache retains at most sixteen memory entries; their history-tuple keys also retain observed values. Cache reuse avoids repeating identical calculations but does not turn those values into newly observed information. Each worker has its own process-local cache, so increasing worker count can increase peak memory.

For one contextual candidate and inner query, distance and competence allocation cost $O(Jd+JK)$ after memory construction. Stacking first forms a K-by-K Gram matrix from JH error rows, costing $O(JHK^2)$. Enumerating every nonempty active expert set then has the upper bound $O(2^K K^3)$ for dense face solves. With C_c non-stacking and C_s stacking candidates, an upper bound for the inner allocation work is

$$O\Big[C_c V(Jd + JK) + C_s V(JHK^2 + 2^K K^3)\Big], \tag{29}$$

plus scoring the H-lead paths and the final selected allocation. In this experiment $K = 5$, $J \leq 12$, $C_c = 11$ and $C_s = 4$; the exhaustive solver considers 31 faces. These fixed small values explain why exact face enumeration is feasible here. The exponential dependence on K precludes inferring scalability to a large expert library from this experiment. We report no recovered hardware-normalized latency comparison: retained per-series seconds do not identify the historical machine or the elapsed makespan of parallel execution.

Algorithm 7 specifies the fixed context rule and its controls. Algorithm 8 specifies the completed-target selection used by the separately validated variants. The outer reserved targets never enter candidate fitting.

Algorithm 7 Combine five current expert forecasts using fixed contextual, recency-only or uniform weights estimated from completed origins.

Require: Observed prefix, horizon H, seasonal lag m, probabilities and weighting rule

Form up to 12 unique integer origins evenly spaced from $\max(2m, 2H + 8, 24)$ to $n - H$
Use the empty roster if the last eligible origin precedes the first
Compute the roster's prefix features and seasonal-scaled expert losses
Initialize equal weights
if the roster is nonempty and the rule is ContextMemory or RecencyMean **then**
    Compute mean loss for each expert with a six-origin recency half-life
    if the rule is ContextMemory **then**
        Compute RMS query distances and set bandwidth to the larger of their median and .25
        Compute the local kernel-weighted loss with .25 shrinkage toward the recency mean
    end if
    Convert scores to softmax weights at temperature one and mix with .05 uniform weight
end if
Fit the five expert quantile paths on the current prefix
Return their weighted columns, the weights and the eligible origin roster

Algorithm 8 Temporally completed candidate selection. Every expert forecast and validation target used to choose a combination is available before the current reserved target block begins.

Require: Observed prefix $y_{1:n}$, horizon H, fixed candidates $\mathcal{C}$

Construct the fixed historical origin schedule and cache expert paths
for each historical query origin q **do**
    Retain earlier origins o satisfying $o + H \leq q$
    Use the query only when at least three completed origins are available
    for each candidate $c \in \mathcal{C}$ **do**
        Fit its weights from the completed historical memories only
        Score its combined path on the query block using the query's prefix scale
    end for
end for
Select minimum mean validation loss; break ties by the fixed grid order
With no eligible query, use the explicitly defined recency-only fallback
Fit the selected rule's current weights from all completed prefix memories
Return the weighted current expert path without observing $y_{n+1:n+H}$

9 Experimental protocol and archived results

9.1 Historical training-case ranking

The offline training stage evaluates the classical roster on each supplied training case with the same rolling evaluator used for case scoring. It accumulates the case MASE for each available method, omits NaN values, and sorts ascending by the unrounded arithmetic mean. A method with no recorded score sorts last. Ties follow the declared roster order. The stored scores are rounded to five decimal places only after ranking, and the first method is recorded as the default. Each training case with a score contributes one term; longer cases do not automatically receive more weight. This historical artifact depends on the caller's choice of training cases. It makes no per-prefix selection and has no independent confirmation stage. It is separate from the discovery-trained forest in Section 14.9 and does not determine the method shown as best on a reserved case.

The aggregation takes one pass over the available case scores and sorts the fixed classical roster. Repeated rolling fits dominate its cost. When two methods have missing scores on different cases, their means can summarize different subsets; record both denominators before using the ranking in a new evaluation.

9.2 Archived common final-block protocol

Each selected series supplies one forecast origin and one final block. Fourteen heavy forecast paths remain frozen; five corrected lightweight methods and three combinations are newly computed. Every admitted path is scored on the same actual values and horizon within its case. Original rolling-backtest results remain archived. Their differing origin budgets, ranging from three to more than twenty-five windows, make a direct ranking of those aggregates confounded by the evaluation schedule.

Let $y_1, \ldots, y_n$ denote the contiguous prefix, $a_1, \ldots, a_H$ the final targets and p_j a point prediction. The historical seasonal scale is

$$s_m = \frac{1}{n-m} \sum_{t=m+1}^{n} |y_t - y_{t-m}|, \tag{30}$$

with the lag bounded to the available history. MAE is $H^{-1} \sum_j |a_j - p_j|$, RMSE is the square root of mean squared error, and MASE is MAE/s_m [2]. A value below one compares a future error

with a historical scale. It does not directly show that the method beat an independently scored SeasonalNaive path on the same future block. A zero scale produces an unavailable scaled metric rather than an invented denominator.

For lower and upper columns l_j, u_j, empirical coverage is $H^{-1}\sum_j \mathbf{1}\{l_j \le a_j \le u_j\}$ and width is $H^{-1}\sum_j (u_j - l_j)$. With nominal interval miscoverage $\alpha = 0.2$, the interval score is

$$IS_j = (u_j - l_j) + \frac{2}{\alpha}(l_j - a_j)_+ + \frac{2}{\alpha}(a_j - u_j)_+. \tag{31}$$

MSIS is its block mean divided by s_m. The artifact's WQL uses only $Q = \{0.1, 0.5, 0.9\}$:

$$\mathrm{WQL}_3 = \frac{2\sum_{q\in Q}\sum_{j=1}^{H}\rho_q(a_j - \widehat{y}_j^{(q)})}{3\sum_{j=1}^{H}|a_j|}, \quad \rho_q(e) = \max(qe, (q-1)e). \tag{32}$$

This is an average of three quantile losses, not a complete predictive-density evaluation. A zero absolute-target sum makes this normalized score unavailable. Proper-score theory motivates reporting probabilistic quality, while the finite quantile grid limits what is observed [3].

9.3 Archived-path final-block results

Table 20 scores every admitted method on the same fifteen reserved blocks, without giving longer histories more weight in the case mean.

Table 20: Corrected common final-block comparison on all fifteen selected cases. Each case has equal weight; coverage and scaled error describe this finite collection rather than a population guarantee.

Method	Cases	Mean MASE	Mean coverage
Chronos-Bolt	15	1.0363	0.7563
TimesFM-2.5	15	1.0633	0.8071
TiRex-2	15	1.0809	0.7694
Chronos-2	15	1.1555	0.6905
DLinear	15	1.1600	0.5595
AutoARIMA	15	1.2104	0.7813
NLinear	15	1.2298	0.4524
NHITS (nf)	15	1.2546	0.5079
NHITS	15	1.2898	0.4857
LightGBM	15	1.2983	0.5786
SeasonalNaive	15	1.4137	0.8702
RecencyMean	15	1.4489	0.8889
ContextMemory	15	1.4504	0.8944
AutoTheta	15	1.4964	0.7329
HoltWinters	15	1.6778	0.8639
AutoETS	15	1.6940	0.7639
NLinear (nf)	15	1.7547	0.1821
EqualMean	15	1.8970	0.8611
SES	15	1.9167	0.8274
Theta	15	1.9224	0.8274
Holt	15	3.9846	0.7302
DLinear (nf)	15	10.7385	0.4996

Chronos-Bolt has the smallest mean MASE in this selected, mixed-provenance comparison, followed by TimesFM 2.5 and TiRex-2. Their paths were not freshly refitted or regenerated for

this experiment. No statistical significance, resource-normalized superiority or independence from pretraining data follows from that order. Method-specific training budgets and context limits remain part of the compared configurations.

Coverage exposes substantial differences that a point ranking conceals. Several neural paths have narrow intervals and low observed coverage. A broad interval can increase coverage at the expense of decision usefulness; coverage alone should not be optimized. Individual case outcomes and widths remain available in the artifact and workbench, including unfavorable neural and combination results.

Figure 3 retains both signs of the case-level difference, so favorable examples cannot replace the stated aggregate comparison.

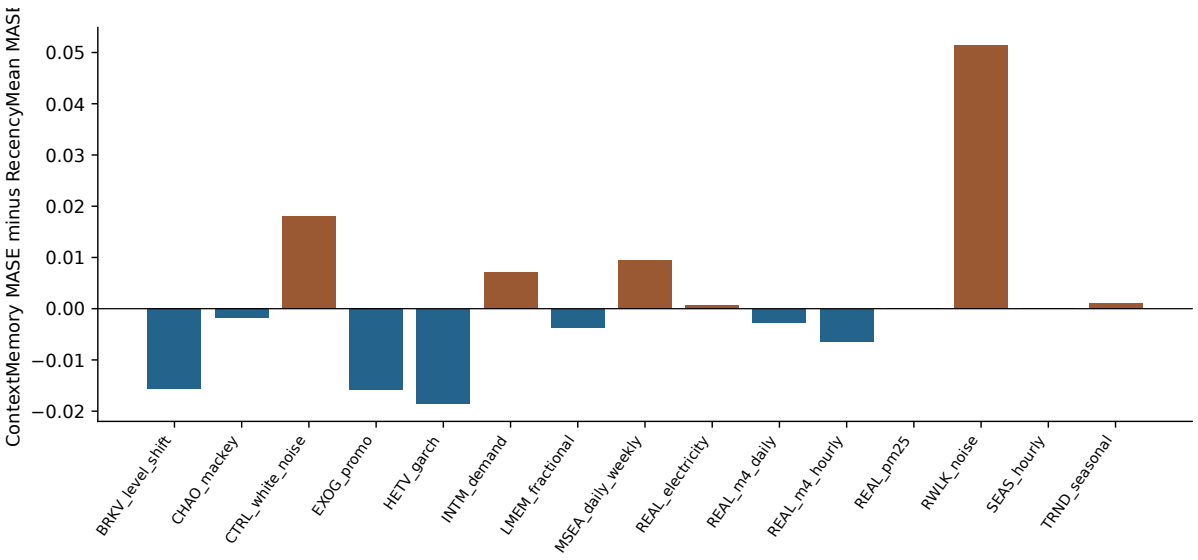


Figure 3: ContextMemory minus RecencyMean MASE for every selected case. Positive values favor the recency-only ablation. The common target and expert set isolate the contextual weighting component within each case, but the selected origins do not establish a general effect.

ContextMemory's mean MASE is 1.4504, versus 1.4489 for RecencyMean. The difference is unfavorable at the declared primary criterion, despite occasional individual improvements. EqualMean gives 1.8970. Settings are unchanged after either the invalidated display-history calculation or the corrected run. A changed configuration would define a new experiment and must not overwrite this result.

These summaries weight the selected cases equally rather than weighting every target value equally. A long horizon therefore does not acquire more influence merely because it supplies more scored leads. Conversely, an equal-case mean does not describe any known operational mixture of demand series. The reported average answers a question about this inventory, and changing the inventory could change its ordering. Per-case tables allow a reader to inspect that dependence instead of accepting the average as a universal ranking.

The archived-path result is also a comparison of recorded configurations, not a controlled assessment of architecture alone. Pretraining, context caps, optimization budgets and the distinction between an automatic reference implementation and a direct adaptation remain part of each method identity. No error bar over fifteen deliberately selected cases would remove these design differences. The separate external study supplies a larger and more explicitly accounted collection for its smaller roster; it does not retroactively turn this final-block exercise into a representative neural-model benchmark.

10 Fresh execution of the complete roster

A second experiment fits or infers all twenty-two configurations again from the same verified contiguous prefixes. The separately admitted artifact records every prediction, source-file and input hash, checkpoint revision, package version, fit/inference duration and warning category. An independent standard-library checker recomputes all seven metrics from the stored numerical arrays, verifies complete inventories and rejects trials, changed targets, frozen paths disguised as new fits and stale source bindings. These admission checks establish internal numerical consistency and recorded identity; they do not establish independence from model pretraining data.

The execution uses Python 3.12, NumPy 2.1.3, PyTorch 2.9.1 with CUDA 12.8, StatsForecast 2.0.3, LightGBM 4.6.0, MLForecast 1.0.31, NeuralForecast 3.1.9, Chronos forecasting 2.3.2, TimesFM 2.0.2 and TiRex-2 0.1.1. Its GPU is an NVIDIA GeForce RTX 4070 Laptop GPU. NumPy follows the TiRex-compatible line in this isolated Linux environment; notebook/parity computations use their separately pinned environment. Direct and reference neural models retain their declared 300-step budgets. LightGBM uses one worker; BLAS/OpenMP threads and compiler workers are bounded, and torch CPU operations use four threads. These resource choices are part of the recorded configuration.

Table 21 pairs new executions with archived paths on the same outcomes; matching values describe recomputation rather than independent statistical replication.

Table 21: Fresh execution alongside the archived-path comparison. The same fifteen final blocks are used in both; these are not independent replications.

Method	Archived MASE	Fresh MASE	Fresh coverage
Chronos-Bolt	1.0363	1.0363	0.7563
TimesFM-2.5	1.0633	1.0633	0.8071
TiRex-2	1.0809	1.0809	0.7694
Chronos-2	1.1555	1.1555	0.6905
DLinear	1.1600	1.1600	0.5595
AutoARIMA	1.2104	1.2104	0.7813
NLinear	1.2298	1.2298	0.4524
LightGBM	1.2983	1.2983	0.5786
NHITS (nf)	1.2546	1.2993	0.4968
NHITS	1.2898	1.3031	0.4746
SeasonalNaive	1.4137	1.4137	0.8702
RecencyMean	1.4489	1.4489	0.8889
ContextMemory	1.4504	1.4504	0.8944
AutoTheta	1.4964	1.4964	0.7329
HoltWinters	1.6778	1.6778	0.8639
AutoETS	1.6940	1.6940	0.7639
NLinear (nf)	1.7547	1.7547	0.1821
EqualMean	1.8970	1.8970	0.8611
SES	1.9167	1.9167	0.8274
Theta	1.9224	1.9224	0.8274
Holt	3.9846	3.9846	0.7302
DLinear (nf)	10.7385	10.7385	0.4996

The two experiments use the same final targets, so rerunning an implementation is not another independent statistical test. Differences can reflect runtime, precision or training implementation rather than a changed data population. ContextMemory still fails its aggregate comparison with

RecencyMean; the selected settings are not retuned after that outcome. Warning categories remain visible even when predictions are finite. Durations include fitting, inference, imports and first-use loading and cannot be interpreted as a normalized warm-inference leaderboard.

Exact Hugging Face revisions identify the four admitted pretrained checkpoints. The experiment retains its specified TimesFM 2.5 checkpoint and recorded licence. A newer upstream release would require separate review and execution rather than inheriting these results. This roster makes no claim to contain the latest released model of every family. The original streaming results below were not regenerated by this final-block execution.

Each admitted fresh path has the requested horizon and three ordered finite columns. Its execution record retains the package and checkpoint identities used for that run. A new successful run establishes computability for these inputs; it does not establish numerical identity across arbitrary devices or a new independent target realization.

11 Comparison with four Timestral checkpoints

Timestral provides a pretrained multivariate quantile forecaster [11]. This experiment uses only its univariate interface. Every checkpoint receives the same complete observed prefix as the fresh common-block comparison, represented in float32. The input excludes the reserved target values. All 21 quantile columns are retained, with columns .1, .5 and .9 supplying the scored interval and center. The official CPU implementation uses four threads, symmetric averaging and nonnegative clipping under its model rules. Inference code, checkpoint revisions and the immutable reference digest were specified before Timestral execution, with the comparator outcomes already known.

Table 22 summarizes dimensionless scores across the same fifteen cases. They include four authentic excerpts and eleven generated cases. The raw MAE, RMSE and interval widths remain per case because the cases have different source units. Timestral-123M has the lowest observed mean MASE and MSIS within this set of four sizes. This ranking does not establish that another model-size ordering would hold across other data or that the smaller checkpoint dominates every case. The 209M checkpoint attains 89.14 percent mean case coverage in the four authentic excerpts and 73.86 percent across the generated cases. Those subgroup means illustrate why the aggregate coverage alone is insufficient.

Table 22: Case-weighted scores for four official Timestral checkpoints on the same fifteen reserved blocks, with a nominal 80 percent interval.

Checkpoint	MASE	MSIS	Coverage
Timestral-5M	1.08491	4.80842	81.47%
Timestral-28M	1.05492	4.67321	81.94%
Timestral-123M	1.03585	4.61816	81.11%
Timestral-209M	1.04903	4.67244	77.94%

The planned 209M minus AutoTheta mean case MASE difference is -0.44740. Twelve differences are negative and three positive. Ten thousand jointly reweighted case samples give a descriptive 95 percent percentile interval of $[-0.79356, -0.15991]$. These draws preserve the pairing but cannot establish population sampling independence. The baseline outcomes are from the unchanged fresh comparison, rather than concurrent benchmark execution on the same machine. Historical reference timing therefore cannot be used to infer a speed advantage from the recorded Timestral inference durations.

Two executions matched all 60 full quantile paths and case scores exactly on the recorded environment. Separate scoring logic verified the case errors, scores and directions against the held-out observations and recomputed the seeded reweighting interval. The complete numerical

output records source, model and checkpoint-file hashes. The study does not resolve vendor pretraining overlap with the public sources. It supplies no estimate of benefits from multivariate or known-future covariates, and it does not reproduce the vendor's broad benchmark rankings. These limitations distinguish observed performance on the selected blocks from prospective use on unseen source populations.

The companion table `next/tables/fresh-by-case.csv` gives all twenty-two fresh method scores on each of the fifteen final blocks. With one block per case, the individual case scores are direct measurements on that block. Repeated lead errors within a block do not provide independent replications. A paired reweighting across selected cases measures sensitivity to their relative weighting. It cannot supply within-case uncertainty over a future forecast origin or a population confidence interval for a source collection.

12 Combinations selected by temporal validation

ContextMemoryCV and ValidationMix extend the fixed combination with parameter selection inside the observed prefix. They retain SeasonalNaive, SES, Holt, additive HoltWinters and finite-history Theta as experts. At a remembered validation origin v, a training block beginning at o is eligible only if $o + H \leq v$. At least three fully completed training blocks are required. Candidate scores average scaled absolute error over these causally held-out historical blocks. Without eligible validation, the method falls back to the original RecencyMean rule. No outer target is accepted by the fitting API.

ContextMemoryCV selects among context shrinkages $\{0, .5, 1\}$ and temperatures $\{.15, .5, 1\}$ plus the recency-only and equal-weight controls. ValidationMix additionally considers regularized convex stacking, minimizing mean squared normalized expert-combination error plus $\lambda\|w - \mathbf{1}/5\|_2^2$ over the simplex, for $\lambda \in \{.01, .1, 1, 10\}$. Five experts permit explicit active-set enumeration. A 5% uniform floor is applied after fitting. Candidate ranking uses historical absolute error; the selected rule is refit on all completed memory blocks. All families and parameter values are fixed before the external outcomes are evaluated. Established stacking and temporal-validation principles are not claimed as new general ideas.

The external sources are complete Monash M4 Weekly and M3 Monthly inventories [32, 33]. Horizons and seasonal periods are $(13, 52)$ and $(18, 12)$, respectively. Every method receives at most the final 512 contiguous observations at an origin. Up to three nonoverlapping final blocks are used, with a minimum available prefix of $\max(2m, 2H + 8, 24)$. Sixty-five weekly identities have no eligible origin, leaving 294 series and 882 origins. Six exact duplicate monthly identities are excluded, leaving 1,422 series and 3,547 origins. Every source identity and exclusion remains available. Origins are averaged within series, then series equally within each collection. This is not the original forecasting-competition protocol or an official competition ranking.

Table 23 keeps point error, coverage and interval score together, making the collection-specific interval weaknesses visible alongside the point-error gains. Figure 4 compares the two external collection means with AutoTheta; the point-error ordering remains separate from the interval-score comparison.

ValidationMix lowers mean MASE relative to RecencyMean on both admitted collections, .2855 versus .3109 weekly and .8993 versus .9085 monthly. ContextMemoryCV gives .2848 and .8996. AutoTheta's .3076 weekly result is higher, while its .8472 monthly result is lower. The point improvement therefore depends on collection and comparator. Context shrinkage, temperature and temporal selection change together; this comparison does not isolate the causal value of the context kernel alone.

Interval quality remains unfavorable relative to AutoTheta. ValidationMix gives weekly MSIS 2.5477 and coverage .9503, and monthly MSIS 6.8208 and coverage .9326. AutoTheta gives 1.7125

Table 23: Complete external comparison. W denotes 294 eligible M4 Weekly series; M denotes 1,422 unique M3 Monthly series. MASE and MSIS average origins within series and then series equally; Cov is empirical interval coverage. These are descriptive summaries, not confidence intervals.

Method	W MASE	W MSIS	W Cov	M MASE	M MSIS	M Cov
SeasonalNaive	0.8446	8.1670	0.9326	1.0864	8.8123	0.9342
SES	0.3533	2.3404	0.8926	1.0456	7.1945	0.8534
Holt	0.3806	2.4611	0.8844	1.8040	10.8472	0.8005
HoltWinters	0.5396	3.0778	0.8413	0.9375	6.7112	0.9160
Theta	0.3469	2.3237	0.9041	1.0021	7.0453	0.8700
AutoETS	0.3413	1.8490	0.8713	0.8514	4.1518	0.7324
AutoTheta	0.3076	1.7125	0.8511	0.8472	4.2994	0.7279
ContextMemory	0.3079	2.9468	0.9751	0.9086	6.8846	0.9348
RecencyMean	0.3109	2.9315	0.9742	0.9085	6.8846	0.9348
EqualMean	0.3548	3.3233	0.9702	0.9962	7.2131	0.9333
ContextMemoryCV	0.2848	2.5538	0.9511	0.8996	6.7999	0.9329
ValidationMix	0.2855	2.5477	0.9503	0.8993	6.8208	0.9326

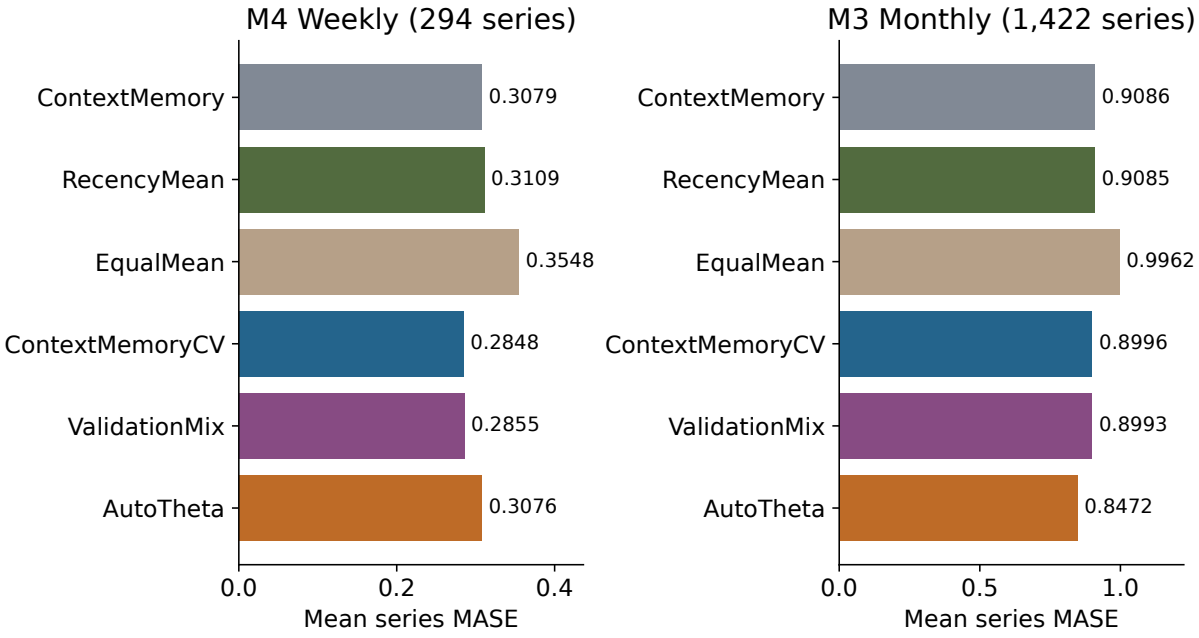


Figure 4: The new temporally selected combinations and fixed controls on each external collection, with AutoTheta as an automatic-statistical comparator. The complete twelve-method table retains the other individual experts and AutoETS. Bars are descriptive mean MASE without population confidence intervals.

and .8511 weekly, and 4.2994 and .7279 monthly. Nominal coverage is .8. These weighted Gaussian endpoints are not calibrated by minimizing point error, and wider intervals do not establish a better decision rule.

Table 24 reports a retrospective paired-series calculation on the retained forecasts. First average all admitted origins within each series and method. For each control, subtract its series loss from the corresponding ValidationMix loss. Resample the resulting paired differences with replacement within each collection, keeping the number of selected identities fixed at 294 or 1,422. Ten thousand draws use PCG64 with seed 20261105 plus the first four bytes of the SHA-256 of the collection identifier interpreted in little-endian order. The endpoints are the linear 2.5 and 97.5 percent quantiles of the resampled means. This operation does not refit an expert, choose a new parameter or treat the repeated origins as independent replications. The complete retained evidence includes 36 comparisons. The intervals were calculated after seeing the original results and remain exploratory and unadjusted for multiplicity or source dependence.

Table 24: ValidationMix minus each control after averaging available origins within each series. Unadjusted exploratory 95% paired-series intervals describe the admitted collection, not population-wide superiority.

Collection	Control	Loss	Difference	95% interval	Lower loss
Weekly	RecencyMean	MASE	−0.025337	[−0.033914, −0.017227]	190/294
Monthly	RecencyMean	MASE	−0.009154	[−0.013176, −0.005173]	589/1422
Weekly	AutoTheta	MASE	−0.022065	[−0.037521, −0.008272]	155/294
Monthly	AutoTheta	MASE	0.052154	[0.032437, 0.073112]	689/1422
Weekly	RecencyMean	MSIS	−0.383808	[−0.431160, −0.334374]	252/294
Monthly	RecencyMean	MSIS	−0.063788	[−0.088692, −0.040255]	589/1422
Weekly	AutoTheta	MSIS	0.835230	[0.650853, 1.038115]	49/294
Monthly	AutoTheta	MSIS	2.521379	[2.364504, 2.673453]	267/1422

The monthly mean MASE difference from RecencyMean is −0.009154, with an interval from −0.013176 to −0.005173. Only 589 of 1,422 monthly series have lower error than RecencyMean. These two statements describe the distribution of magnitudes and the frequency of improvement, respectively. Monthly AutoTheta still has lower point error, and its interval score remains lower in both collections. The positive ValidationMix minus AutoTheta MSIS differences in Table 24 quantify the cost of the point-selected mixture's interval construction under this scoring rule.

On the original fifteen development blocks, ContextMemoryCV and ValidationMix give mean MASE 1.3974 and 1.4031, while the original fixed ContextMemory and RecencyMean results remain 1.4504 and 1.4489. Their MSIS values, 9.5025 and 9.3428, are worse than RecencyMean's 9.1076. These already examined cases are not new external confirmation. The separate original twenty-two-variant execution and complete M4 Daily association/selection study remain unchanged.

All 53,148 method–origin forecast paths are independently rescored against their source-derived targets and seasonal scales. Final expert weights reconstruct the combined quantile paths. Temporal splits, complete historical targets, candidate losses and chosen parameters remain in the compressed numerical file. Browser delivery uses a compact index and separately fetched per-series documents, rather than downloading the entire numerical file for one chart. Every displayed path is independently checked against its stored score. Exact duplicates do not imply that all remaining series are independent or free of related-source overlap; no population or operational-superiority conclusion follows.

13 Streaming evaluation as separate evidence

The historical streaming experiment uses a predict–observe–update order [6], but the point forecaster is recomputed on each full observed prefix. Its artifact key `Theta` denotes a smoothing-plus-half-drift adaptation, not the corrected finite-history Theta equation used in the final-block comparison. ACI-style [7] and PID-style controllers adapt the effective miscoverage level of a normalized-residual FIFO buffer. The PID-style rule is not a faithful implementation of published Conformal PID [8]; its guarantees are not imported by name. No native stateful model update or framework capability comparison is evaluated.

A full numerical reconstruction exports 21,296 method-cutoff rows across fifteen cases, with targets, forecast endpoints, scores, buffer sizes and controller states. Every archived coverage trajectory and final weighted quantile loss matches at saved precision. At nominal 80%, mean rolling absolute coverage error is .0830 for the raw smoothing-plus-drift base, .0656 for its ACI-style controller and .0645 for its PID-style controller. The much smaller .0416, .0057 and .0060 values measure absolute deviations after temporal averaging, which permits local errors to cancel. These are different statistics. Scaled widths and interval scores accompany coverage.

The reconstruction also varies coverage (.7, .8, .9), two point bases and three gain multipliers, retaining all 630 case-setting rows. Eleven generated mechanisms are repeated at two additional seeds, while the four real excerpts are not assigned artificial repetitions. These are sensitivity calculations on the same selected design, not independent population confirmation. The original gains have no recovered historical tuning rationale, and original per-call hardware metadata remain unavailable. Historical latencies therefore support no normalized runtime comparison. Exact source arrays and hashes permit numerical replay without claiming to restore missing acquisition history. These streaming outcomes are separate from both the final-block experiment and the M4 Daily corpus study.

The information order is observable at the row level. A prediction uses the current prefix and the residual buffer available before its target is revealed; only after scoring does that target affect the next fit and controller state. The record therefore supports a direct audit of a potential one-step timing error. Recomputing a prefix at each cutoff has a different computational cost from incrementally updating sufficient state. The former is the procedure reconstructed here, so no measured replay duration is transferred to a native online implementation. Buffer capacity, seasonal scaling and clipping also affect the resulting intervals and remain part of the controller configuration.

13.1 Browser delivery and executable teaching artifacts

Fifteen catalog boundaries cover traces, manifests, the index, diagnostic analysis, streaming results, ONNX metadata/assets, confirmed browser calibration, the retained ridge study, archived-path and fresh comparisons, historical selector metadata, the complete daily corpus study, the validated-combination index and its per-series records, and release readiness. Normative JSON Schemas validate the committed JSON documents. An OpenAPI component inventory generates browser TypeScript types without activating a prediction API. Cross-field invariants, source hashes, temporal reconstruction and numerical parity remain executable tests.

The browser delegates chart interaction, summaries, tables, reset controls and keyboard support to the pinned shared shell. The product supplies data adapters, stable method identities and scenario state. Bilingual content uses locale-aware numbers; model and licence versions retain their identity. Request cancellation and generation identifiers prevent a slower case request or inference result from overwriting a newer selection.

Numbered local workflows check prerequisites, initialize declared environments, build an external

data candidate and start the development server. Generation binds source and payload fingerprints, refuses partial inventories and changed frozen inputs, and separates candidate creation from explicit promotion. Historical engine-version metadata is retained rather than relabeled to match the new browser release. An unused epidemic-model surrogate inherited from an older repository structure was removed.

Sixteen executed English and Spanish notebooks cover a seasonal reference, an incomplete-season experiment, actual statistical/tree adapter execution, the fixed combination with its ablations, sequence patterns, multiscale complexity and disjoint-series corpus inference. Their metadata records exact direct dependencies, sources, inputs and method coverage. Fresh-kernel execution and byte comparison check that committed outputs are current. Numerical convergence notices are retained as portable text; machine paths and incidental notebook metadata are excluded.

14 Sequence diagnostics and corpus evaluation

14.1 Diagnostic atlas and computational boundaries

The diagnostic toolkit contains ten families: stationarity, autocorrelation, seasonality, filtering and decomposition, change points, volatility, distribution and complexity, fractals, nonlinear dynamics, and causality. Methods include ADF/KPSS and related tests, ACF/PACF and Ljung–Box diagnostics, STL/MSTL and periodograms, offline change-point procedures, ARCH/GARCH diagnostics and surrogate-based nonlinear analyses. A short series or unavailable dependency produces an explicit unavailable result for that analysis; it does not silently produce a substitute measurement.

Statistical diagnostics are descriptive or model-dependent. An autocorrelation band containing the observed coefficients does not prove white noise. A unit-root test does not certify future stationarity. A fractal estimate is sensitive to sample length, detrending and the assumed stochastic class. A nonlinear surrogate rejection does not uniquely identify deterministic chaos. The workbench exposes these cautions alongside the results rather than presenting a diagnostic label as a forecasting guarantee.

The browser recomputes lightweight diagnostics and light forecasts for generated scenarios. Committed analysis artifacts retain their original scope; several describe the complete observed series, including what later becomes a forecasting holdout. They are suitable for retrospective explanation, not for feature selection at the earlier forecast origin. The proposed combination computes its own prefix-only contexts and does not consume those retrospective diagnostics.

Heavy statistical, neural and pretrained calculations run offline. GPU availability, checkpoint caches and Python environments are distinct from the served browser bundle. A local environment with only NumPy cannot be represented as a complete neural or foundation-model rerun. Conversely, rescoring a stored model path does not require pretending that its original runtime has been restored.

The procedure crosswalk in `next/tables/procedure-crosswalk.csv` connects each executable procedure to its method section, validation file and example of retained evidence where available. It distinguishes callable diagnostics from those with retained values. All fifteen archived bivariate panels have a skipped marker, while current typed dispatch is exercised by runtime tests. The shared dispatch conditions are described below, and each fitted statistic retains its separate interpretation.

Algorithm 9 shows the shared availability and input rules of the retrospective analysis stage. Its individual calculations are specified in the following subsections. A reported panel error preserves failure rather than inserting a zero-valued scientific result.

Algorithm 9 Dispatch retrospective diagnostic panels on one admitted series

Require: Observations y, declared period m, case identity, seed and typed covariates
Compute univariate panels on y, preserving exceptions as panel errors
Set $z \leftarrow y$
if an admissible robust STL fit has seasonal strength at least 0.64 **then**
    Set $z \leftarrow y - S$ using its seasonal component S
end if
Compute break and regime diagnostics on z
if y contains at least 400 observations **then**
    Derive a local case seed and run the nonlinear panel with nine surrogates
else
    Record an unavailable nonlinear panel
end if
if a typed covariate has a complete aligned history **then**
    Compute the pair diagnostics using that first covariate
else
    Record unavailable pair diagnostics
end if
Bind the original-axis sequence diagnostics to the full record
return diagnostic panels and availability records

14.2 Unit roots, stationarity and differencing

The legacy univariate adapters remove nonfinite entries before fitting. A lag then refers to the compressed sequence, so a missing observation can change the temporal meaning of a lag. The sequence-extension adapters instead require a finite contiguous history. The analysis stage applies these diagnostics to the full available case record, which can include observations later reserved for forecast scoring. Its diagnostic results cannot be used as historical predictors without recomputing them inside the relevant prefix.

The augmented Dickey–Fuller regression [15] is

$$\Delta y_t = c + \gamma y_{t-1} + \sum_{j=1}^{p} \phi_j \Delta y_{t-j} + e_t. \tag{33}$$

The adapter uses a constant and chooses augmentation order by AIC in Statsmodels 0.14.6. It tests the unit-root null using the statistic for γ, whose reference distribution differs from an ordinary regression t test. Phillips–Perron keeps a constant and uses the `arch` 8.0.0 tau statistic with a long-run covariance correction. Its bandwidth controls the correction for serial dependence and is distinct from the augmentation order. DFGLS first removes the deterministic level using GLS detrending, then estimates a Dickey–Fuller regression on that residual sequence, with lag selection delegated to the pinned backend. The Zivot–Andrews call searches admissible interior breaks with the constant-break specification. Its selected break index maximizes evidence for the test's alternative under that specification. It does not identify a known external event.

KPSS reverses the direction of the null. For residuals from the level model, with $S_t = \sum_{i \le t} e_i$ and long-run variance estimate $\widehat{\omega}^2$, the statistic is

$$K = \frac{\sum_{t=1}^{n} S_t^2}{n^2 \widehat{\omega}^2}. \tag{34}$$

For the Phillips–Perron calculation, regress y_t on an intercept and y_{t-1} for the $N = n - 1$ usable rows. Let $\widehat{\rho}$ be the lag coefficient, b its OLS standard error and u_t the regression residual. Compute $s^2 = \sum u_t^2/(N-2)$ and $g_0 = \sum u_t^2/N$. Let L^2 be the Bartlett-weighted Newey–West long-run

variance on the undemeaned residuals, using $\lceil 12(n/100)^{1/4} \rceil$ lags. The backend's default tau statistic is

$$Z_\tau = \frac{\sqrt{g_0}}{L}\frac{\widehat{\rho}-1}{b} - \frac{L^2 - g_0}{2L}\frac{Nb}{s}. \tag{35}$$

The backend converts this value with its constant-specification MacKinnon reference. A zero coefficient standard error or too few residuals for the bandwidth refuses the test. The long-run covariance sum costs $O(Nk)$ for k included lags. DFGLS uses the constant-only local coefficient $1 - 7/n$ to quasi-difference y and the constant, preserving their first rows. It fits the quasi-differenced level by least squares and subtracts that level from the original observations. The final lag count minimizes AIC on the OLS-demeaned series with no deterministic regressor, using the backend's sample-size lag cap. Fit a no-intercept augmented regression to the GLS-demeaned observations with that lag count and report its lagged-level t statistic using the DFGLS reference. Both wrappers clean the input before fitting and retain the probability and decision at .05.

Zivot–Andrews uses the Statsmodels approximation that selects lags once. Run ADF with constant and trend and choose the lag count by AIC, then hold that count fixed for all break candidates. Discard the first and last 15 percent of the record from the candidate search using the backend's integer trim. Fit the intercept-break regression at each remaining position and retain the most negative lagged-level t statistic. The library looks up the associated probability for that break specification and returns the zero-based minimizing index. This scan performs one regression per eligible candidate. Rank failure in the auxiliary design can leave the entire panel unavailable. The Statsmodels adapter selects bandwidth automatically [16]. Probability values at the ends of its tables can be bounds. At level .05 the differencing routine checks KPSS repeatedly after applying $(1 - B)$, where $By_t = y_{t-1}$. It stops at two differences, fewer than eight values, a constant, nonrejection or a numerical exception. Thus its returned order is a recommendation and may encode an early stop. Seasonal differencing uses robust STL strength $\max(0, 1 - \mathrm{Var}(R)/\mathrm{Var}(S + R))$ and applies $(1 - B^m)$ once if strength reaches .64. Short histories and zero denominators return zero strength. The serialized `stationary` flag means rejection of the unit-root null for ADF, Phillips–Perron, DFGLS and Zivot–Andrews. For KPSS it means nonrejection of level stationarity. The ADF/KPSS panel labels the two true flags stationary and the two false flags nonstationary. A true ADF flag with a false KPSS flag gives its difference-stationary label. A false ADF flag with a true KPSS flag gives its trend-stationary label. These disagreement labels are heuristics from the two level specifications. The test probability and reference critical values are dimensionless. Only the Zivot–Andrews result includes `break_index`, a zero-based position in the cleaned sequence serialized as a number. Phillips–Perron and DFGLS use the pinned `arch` defaults for their bandwidth and lag choices. Zivot–Andrews uses the pinned Statsmodels defaults for its trim and AIC lag selection. A failed backend call can leave the stationarity panel unavailable.

14.3 Correlation, periodicity and multi-seasonal decomposition

The sample autocorrelation at lag k divides the lagged centered cross-product by the centered sum of squares. Statsmodels computes it using its FFT branch at up to forty lags. The partial autocorrelation uses adjusted Levinson–Durbin recursion, so its coefficients describe the remaining association after intermediate lags enter the autoregression. The wrapper's $n - 1$ cap alone cannot satisfy the backend's stricter short-sample PACF limit. Both displays compare nonzero lags to the same pointwise white-noise band $z_{1-\alpha/2}/\sqrt{n}$ at $\alpha = .05$. A significant-looking isolated lag in that display has not been corrected for selection across lags.

$$Q_{\mathrm{BP}} = n \sum_{k=1}^{K} r_k^2, \qquad Q_{\mathrm{LB}} = n(n+2) \sum_{k=1}^{K} \frac{r_k^2}{n-k}, \tag{36}$$

$$D = \frac{\sum_{t=2}^{n} (y_t - y_{t-1})^2}{\sum_{t=1}^{n} y_t^2}. \tag{37}$$

The panel uses Box–Pierce and Ljung–Box at $K = 10$, with zero fitted-model degrees of freedom supplied to the backend. A chi-square approximation addresses the specified serial correlations. It cannot certify independence. The Durbin–Watson formula is evaluated on the cleaned levels in this panel. The familiar residual interpretation around two requires the conditions of a regression residual calculation. The separate lag-pair API returns (y_{t-k}, y_t) for $1 \le k < n$, which can be inspected in source units for direct analysis. The baked panel omits these paired values.

Both correlograms store objects ordered by integer lag, including lag zero. Their significant-lag lists exclude zero and contain the lags for which the absolute coefficient strictly exceeds the common band. The portmanteau probabilities are in $[0, 1]$, with `white_noise` true on nonrejection at .05. For the stored `identification_hint`, check Ljung–Box first. The producer takes its white-noise-like branch when `white_noise` is true. When that flag is false and at least four nonzero ACF lags are available, average the absolute coefficients separately over the later half of each nonzero-lag array. An above-band ACF tail and below-band PACF tail suggests AR. The reverse suggests MA. Two above-band tails suggest ARMA. Equality or two below-band tails give a mixed reading. A nonfinite Ljung–Box probability also sets the flag false, so the heuristic can emit the mixed reading on a constant series whose probability serializes as null. A true test rejection requires a finite probability below .05. The stage performs no model order search for this field.

Algorithms 10 and 11 give the test-specific branches and distinguish the two lagged analyses from the spectral summaries.

Algorithm 10 Compute the stationarity decisions and differencing recommendations from one finite cleaned series.

Require: Cleaned observations and an optional declared period

1: ADF: select the augmentation lag by AIC and test the lagged level in the augmented regression
2: KPSS: compute the partial-sum statistic with automatic long-run variance bandwidth
3: Phillips–Perron: fit the lagged-level regression and apply the Newey–West tau correction
4: DFGLS: quasi-difference the level design, choose lags on the OLS-demeaned series and fit the GLS variant
5: Zivot–Andrews: select one lag count, scan admissible breaks and keep the minimum statistic and index
6: Interpret the two null directions, then repeat KPSS with differences up to order two
7: When the period is eligible, evaluate robust STL strength for the seasonal recommendation

The SciPy 1.15.3 periodogram and Welch calls use one sample per observation step and choose the highest nonzero-frequency bin. A peak frequency f gives period $1/f$ in observation steps. Welch averages windowed spectra over segments of length at most 256, reducing variance while coarsening frequency resolution. The winning bin can exist even on noise or a spectrum without meaningful positive power. The stage tests the declared period and twice that period for robust STL strength. For an admissible rounded spectral period, it computes $y_t = T_t + S_t + R_t$ and reports trend and seasonal strength ratios. At least two complete periods are required. The public MSTL adapter iteratively estimates multiple seasonal components, then exports their sum with the trend and remainder. A separate phase-subseries API reshapes complete cycles and reports phase means, discarding the final incomplete cycle. Neither of those two APIs is called by the default panel.

Algorithm 11 Compute lag statistics and period summaries on the cleaned observation axis using the declared defaults.

Require: Cleaned observations and declared candidate periods

1: Compute FFT ACF and adjusted Levinson–Durbin PACF through the admissible lag cap
2: Store coefficient rows, nonzero above-band lags, portmanteau results and the Durbin–Watson ratio
3: Evaluate the identification hint from the Ljung–Box flag and absolute tail averages
4: Compute one-sided periodogram and Welch densities and select each maximum nonzero-frequency bin
5: Fit robust STL to eligible candidate periods for strength and to the admitted rounded spectral period for the summary
6: Store complete periodogram bins, compact Welch and available decomposition strengths

STL estimates the seasonal component by smoothing detrended observations within each phase with local regressions, then removes a low-pass component from the smoothed seasonal sequence. It smooths the deseasonalized sequence for the trend and repeats these steps. The single-period wrapper enables robust residual weighting. Statsmodels 0.14.6 supplies the smoothing windows and inner and outer iteration defaults. The resulting trend, seasonal and residual arrays have source units and share the cleaned observation axis in the direct API. The baked `stl_at_dominant` object exists only when the periodogram peak has period at least two and its rounded integer p satisfies $p \geq 2$ and $n \geq 2p$. It stores the one-period list and dimensionless trend and seasonal strengths in $[0, 1]$. It omits the three component arrays.

The separate MSTL wrapper admits periods at least two and refuses a cleaned record shorter than twice its largest period. The pinned backend can remove a period at or beyond half the record length. It sorts eligible periods, starts their components at zero, adds back the old component for each period, applies STL to the partly deseasonalized values, and subtracts the updated component before advancing. It makes two sweeps for multiple periods and one for a single retained period. The wrapper passes no power transform or custom smoothing windows. It returns the sum of the seasonal columns on the cleaned axis. Its period field lists the requested admissible input periods. The cost grows with the number of period passes and the underlying STL smoothing cost per pass.

The periodogram uses SciPy's one-sided density scaling, constant detrending and a boxcar window. For a non-edge positive bin k on an n-sample record, its density is twice the squared magnitude of the discrete Fourier coefficient of the centered observations, divided by $f_s n$. The zero and Nyquist bins, when present, are not doubled. Frequencies are cycles per observation step, and the density is in $U^2/(\text{cycles per step})$ for source unit U. Multiplying by bin width before summation restores squared source units. The reported peak power has density units. The corresponding period is the reciprocal of peak frequency, in observation steps. Welch uses a Hann window and half-segment overlap under the pinned defaults. Its compact output retains peak power and period rather than the aligned frequency and density arrays. If no positive-frequency bin is available, the record contains `dominant_period=null` and `dominant_power=0.0`. A numerical failure produces an error record instead of a spectrum.

14.4 Filters, intrinsic modes and wavelet energy

The Hodrick–Prescott calculation estimates the trend vector T by solving

$$\min_T \sum_t (y_t - T_t)^2 + 1600 \sum_{t=2}^{n-1} (T_{t+1} - 2T_t + T_{t-1})^2. \tag{38}$$

The fitted trend and residual cycle share the source unit. The fixed penalty is not adapted to hourly or daily sampling. Baxter–King instead truncates a symmetric band-pass filter with periods

6 to 32 and 12 coefficients on either side. Its direct result drops 12 observations at each end. The default filter panel omits that result. The Christiano–Fitzgerald panel uses the same nominal band and a drift adjustment, with asymmetric endpoint weights from its random-walk approximation. Full-length output does not make this filter causal.

EMD sifts envelope means from extrema until the backend identifies an intrinsic mode, then repeats on the residue. The panel retains at most six modes and uses the EMD-signal 1.9.0 stopping rules. Its plain EMD seed argument has no effect. The separate CEEMDAN entry point uses 50 noise-assisted trials with a local noise seed, which controls the perturbations. The default filter panel runs plain EMD. The modes depend on the algorithm and do not identify physical sources by themselves. Continuous wavelet energy uses the Morlet wavelet on a fitted linear detrend. For 64 geometrically spaced periods from two to $\max(4, n/2)$ the scale is period times the wavelet's central frequency. PyWavelets 1.8.0 supplies the coefficients. The panel sums their squared magnitudes over time, retaining one energy per scale, while the direct method can return the full time-scale array. Edge effects and significance are not corrected. Wavelet fitting requires at least eight observations and shorter arrays are refused.

For CEEMDAN, standardize by the input standard deviation and decompose the seeded normal noise realizations with EMD. Normalize each trial's noise modes by the standard deviation of its first noise mode. Add that first noise mode times .005 to the standardized record, extract one mode per trial and average the first modes. At each later step, perturb the residual with the corresponding noise mode scaled by 0.005 times the residual standard deviation when that mode is available. Apply a single-mode EMD to each perturbed residual and average its local means. Subtract that average from the current residual to obtain the next mode. The backend stops when the residual fails to support another mode, its range is below .01 or the sum of absolute residual values is below .05 on the normalized scale. It appends the final residual as the last component row, restores source scale and returns a separate reconstruction closure residual. Work scales with the number of trials and sift iterations per extracted mode. A degenerate zero-variance input cannot be interpreted through this normalization.

The baked filter object stores the cleaned-axis HP trend and cycle and the CF cycle in U. Its CF band contains the lower and upper periods in observation steps. EMD modes form an array of rows, each containing one value per cleaned observation. The residue follows the same axis. For fewer than eight observations the direct CWT call raises an error, whereas the panel omits `scalogram` if the earlier filters succeed. An error from a filter that is invoked replaces the panel with an error record. The retained scalogram aligns period and energy arrays, stores the maximum-energy period and names the wavelet. Energy is a sum of squared coefficients over the cleaned axis in squared source units.

Algorithm 12 records the distinct filters and the additional callable decomposition routes.

Algorithm 12 Apply the filter, mode and wavelet procedures to the same cleaned input with separate output axes.

Require: Cleaned observations y

1: HP: solve the penalized smooth-trend problem and subtract the trend
2: CF: apply the drift-adjusted asymmetric band and retain its full-axis cycle
3: BK when requested: use the symmetric finite band and return the shortened cycle
4: EMD: sift successive modes and return the remaining residue, capped at six modes for the panel
5: CEEMDAN when requested: decompose seeded noise and extract ensemble modes with adaptive residual scaling
6: CWT for $n \geq 8$: detrend, transform at 64 Morlet scales and sum squared coefficients by scale
7: Emit the baked HP, CF, EMD and eligible CWT summaries

14.5 Breakpoints, latent regimes and volatility

When robust STL strength reaches .64, the stage subtracts the seasonal component before the change-point panel. If the decomposition fails, the original record enters the panel. PELT with the squared-error segment cost searches for

$$\min_{\tau_1,\ldots,\tau_k} \sum_{j=0}^{k} \sum_{t=\tau_j}^{\tau_{j+1}-1} (y_t - \overline{y}_j)^2 + k\, 2\widehat{\sigma}^2 \log(\max(n, 2)). \tag{39}$$

Here $\tau_0 = 0, \tau_{k+1} = n$, each segment has at least five observations, and the default variance proxy is half the population variance of first differences with a 10^{-8} fallback in squared source units. The `ruptures` 1.1.10 implementation prunes the search under its cost conditions [17] and evaluates interior candidates on a five-observation grid. The separately callable binary segmentation uses the same default grid and greedily splits with a supplied number of breaks. The terminal index n is omitted from the exported breakpoints. Segment summaries use half-open zero-based bounds, with a mean and population standard deviation over each segment.

The CUSUM statistic is the maximum absolute scaled partial sum of residuals after removing the sample mean, using the OLS-residual reference test at .05. The two-state Markov regression estimates state-dependent means and variances and a transition matrix. Its plotted state path is the argmax of smoothed full-record probabilities. It can change retrospectively when observations arrive. The direct fit returns an $n \times 2$ array of smoothed state probabilities, one row per cleaned observation. The retained panel stores the argmax path, two fitted regime means in source units and a 2×2 transition matrix. Rows identify destination regimes and columns identify origin regimes. The fitted switching variances and smoothed probabilities are absent from the retained summary. Five of the fifteen retained analysis panels contain a state path and ten contain a nested Markov error. A returned path does not certify optimizer convergence.

For volatility, ARCH-LM regresses squared demeaned values on twelve lagged squares and tests with the effective sample size times the auxiliary R^2. A rejection triggers a Gaussian constant-mean GARCH(1,1) fit with internal rescaling:

$$\sigma_t^2 = \omega + a e_{t-1}^2 + b\sigma_{t-1}^2. \tag{40}$$

The panel retains parameters, $a + b$ and log likelihood, while the direct fit also provides conditional standard deviations. These estimates are in the backend's fitted scale when rescaling is applied. Rolling standard deviation uses sample degrees of freedom one and a trailing window $\min(20, \max(3, \lfloor n/10 \rfloor))$, with leading nulls. The Box–Cox implementation adds $1 - \min(y)$ when needed for positivity. For shifted positive x, the transform is $(x^\lambda - 1)/\lambda$ or $\log x$ at $\lambda = 0$. With a declared period of at least two, Guerrero estimates λ from seasonal-subseries relative variability; without that period, the panel uses profile likelihood. The shift and selected exponent are needed to reverse the transformation. Its additive one is in source units and can change the result under unit conversion.

The volatility record's lag orders and rolling window count observations. The ARCH probability is dimensionless and lies in $[0, 1]$. The rolling mean and standard deviation use the input units and align to the cleaned-series positions, with nulls before the first complete window. The fitted GARCH constant mean and variance intercept have units U_{fit} and U_{fit}^2, respectively. The shock coefficient, variance coefficient and their sum are dimensionless. The log likelihood is in natural log units on the fitted scale. The serializer does not retain the backend's numerical rescaling multiplier, so the stored mean and intercept cannot by themselves be converted back into physical units. The lack of a GARCH result after an ARCH-LM nonrejection records a decision to skip that fit. It does not imply zero volatility.

The pinned Guerrero routine retains the last complete blocks of length m. For each block j, compute its positive shifted mean μ_j and sample standard deviation s_j. Use bounded Brent search

over $[-0.9, 2]$ to minimize the sample coefficient of variation of $s_j/\mu_j^{1-\lambda}$ across those blocks. With at most two periods of data the backend returns $\lambda = 1$ directly. The block moments take linear work in the retained record length. Each objective evaluation then scales with the number of blocks. The wrapper passes the selected exponent to SciPy for the transform. Without a declared seasonal period of at least two, SciPy selects the exponent by its profile likelihood instead. The separate log option fixes the exponent at zero. The baked `boxcox` object holds only the method label, dimensionless exponent, nonnegative input-unit shift and reference. A failed transform produces a nested error in place of these fields.

Algorithm 13 shows which decisions govern the expensive fits and their output branches.

Algorithm 13 Dispatch structural-break and volatility computations with their separate input transformations and eligibility decisions.

Require: Cleaned observations and a declared period when available

1: Use the robust-STL adjustment for break analysis when the strength gate is met
2: Run PELT on the selected break input, retaining segment bounds and statistics
3: Compute OLS-CUSUM and fit the two-state Markov model, preserving a nested fit error
4: On the unadjusted observations, compute ARCH-LM and trailing volatility
5: If ARCH-LM rejects, fit GARCH. Otherwise record its skip
6: Shift to a positive input and estimate Box–Cox by Guerrero or profile likelihood
7: Record transform failures and GARCH failures in their individual branches

14.6 Distribution, scaling and deterministic-dynamics diagnostics

The distribution panel computes a population standard deviation, biased sample skewness and excess kurtosis. Jarque–Bera evaluates $n(g_1^2/6 + g_2^2/24)$ against its asymptotic reference. Shapiro–Wilk is run only from 3 to 5,000 observations. It evaluates the normal order-statistic pattern and neither test supplies serial-independence evidence. The baked `normal` flag requires Jarque–Bera nonrejection and, where available, Shapiro–Wilk nonrejection at .05. It retains both probabilities, with a null for unavailable Shapiro–Wilk, and omits the statistics returned by the direct summary. Kernel density uses SciPy Gaussian kernels at one hundred points spanning the sample, with bandwidth chosen by the library and density in inverse source units. A singular covariance can fail the panel even when the moments were computable. The normal Q–Q calculation returns ordered observations against theoretical standard-normal quantiles, with a fitted slope in source units.

The scalar entropy diagnostics use AntroPy 0.2.2. Sample entropy calls the default order two and tolerance .2 times population standard deviation, independently of the public ordinal pattern order. Permutation entropy uses order three, unit delay and normalization by log(3!). Welch spectral entropy divides the Shannon entropy of normalized spectral powers by the log of the bin count. These are distinct from multiscale sample entropy below. BDS compares an embedding correlation integral against the independent-product reference at dimension two, with the default radius 1.5 standard deviations and only for $n \geq 20$. The direct complexity result carries the BDS statistic and probability. The panel keeps only `bds_pvalue` and `iid`, with nulls for both when BDS is ineligible or fails. The flag means nonrejection at .05. Catch24 retains 22 backend-named features plus the original mean and standard deviation in source units. Each catch22 value has the interpretation of its keyed backend feature. If `pycatch22` cannot be imported, the record sets `available` to false and stores a reason and reference in place of the feature map.

R/S scaling fits the log rescaled range across observation scales. The first call uses the simplified random-walk estimator; on failure it uses the nolds polynomial fit. DFA estimates the slope of log root-mean-square fluctuation after linear detrending of integrated profiles. The panel maps a DFA slope above one by subtracting one, then averages the mapped value with R/S for its interpretation label. Values below .45 give the anti-persistent label, values above .55 give the persistent label and

the inclusive middle gives the random-walk-like label. The `reliable` flag only indicates $n \geq 100$. The distinct `arfima_link.d` field is $H_{\mathrm{RS}} - 1/2$ rounded to four decimal places. It is a descriptive proxy. For $n \geq 200$, MF-DFA uses linear detrending on 24 geometrically spaced scales from 8 to $\max(16, \lfloor n/4 \rfloor)$, truncated to integers and deduplicated, with q in $\{-5, -3, -2, -1, 1, 2, 3, 5\}$. From slopes $h(q)$ it calculates $\tau(q) = qh(q) - 1$, $\alpha(q) = \mathrm{d}\tau/\mathrm{d}q$ and $f(\alpha) = q\alpha - \tau(q)$. The panel stores aligned q, $h(q)$, $\alpha(q)$ and $f(\alpha)$ arrays and omits $\tau(q)$. Width above .15 is a heuristic flag. A short input or failed fit yields a nested error. A nonfinite value from a completed fit is represented by null at its array position. Higuchi uses graph lengths up to lag ten, Katz uses path length and extent, and Petrosian counts sign changes of first differences. Their empirical graph estimates do not identify an attractor dimension. The public DCCA coefficient normalizes the summed detrended profile covariance by the geometric mean of the two detrended variance sums at one scale. It uses $s = \max(8, \lfloor m/8 \rfloor)$ by default on the shared length m and linearly detrends the two integrated profiles within each of the $m - s$ overlapping windows of length $s + 1$. It refuses $s \geq m$ and returns a nonfinite coefficient for a zero denominator. The implementation cleans the series separately before truncating to shared length, so callers must align missing observations before this call. The default panel omits this direct result.

The AntroPy 0.2.2 Higuchi calculation forms k offset subsequences for each $k = 1, \ldots, 10$. For offset j, let $M = \lfloor (n - 1 - j)/k \rfloor$. It sums the M absolute increments at step k and multiplies the sum by $(n - 1)/(k^2 M)$. Average the normalized lengths across offsets and regress their logarithms on $\log(1/k)$. This needs $O(nk_{\max})$ work for the lengths. A constant path or offset with no increments can make the slope unavailable. Katz uses $L = \sum_{t=2}^{n} |y_t - y_{t-1}|$, $a = L/(n - 1)$ and $d = \max_t |y_t - y_1|$. Its value is $\log(L/a)/\log(d/a)$. Petrosian counts the sign-bit transitions Z in the first-difference array and evaluates $\log n/[\log n + \log(n/(n + 0.4Z))]$. Both take linear work in the record length. The Katz calculation is undefined on a constant path and can exceed two on finite samples.

Delay embedding forms $(y_t, y_{t+\ell}, \ldots, y_{t+(d-1)\ell})$, requiring a nonempty vector roster. The canonical nonlinear panel uses $d = 4$, $\ell = 1$ and $n \geq 400$. Correlation dimension estimates a slope of $\log C(r)$ against $\log r$, with $C(r)$ the fraction of close pairs. The Lyapunov adapter fits average log-neighbor divergence versus observation step, with temporal exclusion $\max(5, \lfloor n/200 \rfloor)$. Both slope fits use seeded robust regression and a least-squares fallback. Recurrence quantification caps the embedding at 800 vectors, constructs Euclidean pair distances, chooses a threshold at the .1 quantile, removes the diagonal and counts runs of length at least two. The threshold uses all pair distances before diagonal removal. If C is the off-diagonal recurrence count across both triangles and v the capped vector count, the rate is $C/[v(v - 1)]$. Determinism and laminarity divide the points in qualifying diagonal and vertical runs, respectively, by C. Both ratios are zero when $C = 0$, and the longest diagonal run is then zero. The direct result includes the threshold in source units. The baked record retains the three ratios and the longest diagonal run. Its pair matrix has quadratic memory in the capped count. The strict distance inequality and ties can yield density below the target.

IAAFT alternates Fourier amplitude replacement and remapping to the observed ranks for one hundred iterations. The final rank mapping preserves the sorted sample values, with only approximate agreement of spectral amplitudes. The stage requests nine surrogates, comparing the case's dimension estimate with the surrogate mean minus two population standard deviations when that deviation is positive. The reported count is the number of completed calls, which can include a nonfinite dimension estimate. For the 0–1 statistic, demean the series and draw 100 phases uniformly on $[\pi/5, 4\pi/5]$ from the case-seeded generator. For phase c, form cumulative sums of the centered observations times $\cos(tc)$ and $\sin(tc)$ at positions $t = 1, \ldots, n$. At each lag $k = 1, \ldots, \lfloor n/10 \rfloor - 1$, average the squared displacement of this two-dimensional path across all eligible pairs. Correlate the lag vector with those averages when they vary, and take the median correlation across usable phases. The statistic is nonfinite if none are usable. The joint Boolean requires a Lyapunov slope above .01, a 0–1 statistic above .5 and a positive surrogate flag. The

retained explanation string uses a zero cutoff for Lyapunov; the computation uses .01. These finite-sample criteria do not provide a calibrated test of deterministic chaos.

Algorithm 14 describes the report assembly across the distribution, scaling and nonlinear calculations.

Algorithm 14 Compute distribution, scaling and nonlinear diagnostics while retaining unavailable branches and finite-sample thresholds.

Require: Cleaned observations and case seed

Compute moments, available normality tests, KDE and ordered normal Q–Q points
Evaluate sample, permutation and spectral entropies, and attempt BDS when $n \geq 20$
Append the keyed Catch24 features if the backend is available
Fit R/S and DFA slopes and evaluate Higuchi, Katz and Petrosian dimensions
Attempt MF-DFA on the scale and moment grids when $n \geq 200$
if the nonlinear panel is eligible **then**
    Embed the sequence and fit dimension and Lyapunov slopes using the case seed
    Compute the RQA fractions and the phase-median 0–1 statistic
    Form IAAFT surrogates and apply the dimension comparison and joint flag
end if

14.7 Bivariate diagnostics and refitted ridge forecasts

Public bivariate calls truncate to the shorter series, then take a joint finite mask on paired values. The typed analysis stage requires equal aligned input lengths before dispatching. For the direct CCF, let $x'_t = x_t - \bar{x}$ and $y'_t = y_t - \bar{y}$. At signed lag k the numerator is $\sum_{t \in I_k} x'_t y'_{t+k}$ over the overlapping indices I_k. The denominator is $\sqrt{\sum_t x'^2_t \sum_t y'^2_t}$ over the two complete cleaned arrays, so it is fixed across lags. A positive peak lag means that x leads y under this definition. The public cap is 40 lags and the panel uses the fixed pointwise band $1.96/\sqrt{n}$.

Granger comparisons [18] run the SSR F test at lags one through six in both directions, passing the tested response as the first Statsmodels column and the candidate predictor as the second. The smallest probability is compared to .05 without multiplicity correction. Engle–Granger [19] fits the constant specification and tests the residual for a unit root. Johansen [20] uses deterministic order zero and one lag of first differences, comparing its rank-zero trace statistic to the corresponding 95 percent critical value. The separate integration diagnostic asks ADF not to reject in levels and to reject in first differences for both series. It qualifies interpretation without suppressing the returned cointegration statistics. These tests concern the specified lagged relation and integration assumptions. They do not identify an intervention effect. The archived exogenous analysis artifact has a skipped bivariate panel. The current typed dispatch accepts the declared first covariate, a separate runtime correction.

The successful bivariate record stores the paired sample count `n`. The cross-correlation block stores the signed peak lag in observation steps, the dimensionless correlation in $[-1, 1]$ up to floating-point error, a nonnegative pointwise band and a lead label determined by the lag sign. Each Granger direction stores the tested lag selected from $1, \ldots, 6$, the dimensionless probability in $[0, 1]$ and the Boolean decision at .05. Here x denotes the target supplied to the bivariate panel and y the declared first covariate. The cointegration block carries the Engle–Granger probability and decision, the Johansen rank-zero decision and the joint integration flag; its `valid` field repeats that flag. A false flag qualifies the reported tests without discarding their values. Reference strings identify the underlying procedures. A skipped panel or failed computation does not supply the successful branch's fields.

The dispatch appends the target case identifier, first covariate name and fixed descriptive scope

string to aligned pairs, including when a backend returns an error. A length mismatch returns an error before fitting and without those identifiers. A univariate case carries a skipped reason. A finite paired sample may still be too short or otherwise unsuitable for the numerical routines; a failed fit supplies an error instead of test values.

Algorithm 15 gives the paired diagnostic and refitted forecast operations. They use different information windows.

Algorithm 15 Compute retrospective paired diagnostics and construct the separately scored blind and covariate-aware ridge forecasts.

Require: Aligned paired record for diagnostics, observed prefix and covariate availability for forecasting

For diagnostics, jointly drop nonfinite pairs and compute centered signed-lag cross-correlations
Use a 10^{-12} denominator when the centered scale is zero
Choose the first maximum absolute correlation and retain its signed coefficient and lag
Run both directions of the Granger lag scan and store each minimum-probability lag and decision
Compute Engle–Granger, Johansen and the levels-versus-differences integration flags
Preserve the skip or error branch if dispatch or fitting fails
for each forecast cutoff and ridge variant **do**
    Construct the lagged design from the prefix and the permitted covariate column
    Solve the penalized normal equations, or use the last value if the prefix is too short
    Recurse over the requested leads with the declared covariate availability and form residual intervals
end for

The streaming experiment fits two local ridge references. For target y_t, the row of X contains an intercept, y_{t-1} and y_{t-m}, with the first covariate optionally appended. With $D = \operatorname{diag}(0, 1, \ldots, 1)$, the fit solves

$$(X^{\mathsf{T}}X + D)\widehat{\beta} = X^{\mathsf{T}}y. \tag{41}$$

At prefixes of length at most $\max(1, m) + 3$, it falls back to the last value. At longer prefixes it forecasts recursively, inserting the first future covariate column when supplied and zero otherwise. Dispersion is the population residual standard deviation with a 10^{-6} fallback. A Gaussian quantile at lead j adds its normal coefficient times that dispersion times $\sqrt{j}$. The normal-equation solve costs $O(np^2 + p^3)$ for p columns and n available regression rows. At each streaming cutoff it refits on the prefix, so it is not a native state-update timing test. On the one generated scheduled-driver case, aware and blind versions yield MASE .49322 and .75956 and coverage .7682 and .7909. This comparison supports the usefulness of the declared driver in that construction. The current streaming Theta resolves to the corrected finite-history equation. The older retained replay uses the archived smoothing-plus-half-drift formula under the same method key.

14.8 Sequence diagnostics on original observations

The diagnostic extension operates on every original observation rather than the display-decimated history. For window length m, the exact self-join Matrix Profile is

$$P_i = \min_{j:\,|i-j|>\lceil m/4 \rceil} \|z(x_{i:i+m}) - z(x_{j:j+m})\|_2, \tag{42}$$

where each window receives its own population-standard-deviation normalization. STUMPY supplies the exact join and the constant-subsequence convention. A completely constant sequence is reported as unavailable because it cannot discriminate local shapes. The motif is the lowest-distance reported nearest-neighbor pair whose two windows do not overlap. The three discord

candidates have large finite profile distances and non-overlapping query windows. Their ranks are not calibrated anomaly probabilities; normalization deliberately removes local amplitude and mean [21].

FLUSS consumes nearest-neighbor indices and constructs a corrected arc curve [22]. The case diagnostics request three candidate regimes with exclusion factor one and $L = m$. No known generator boundary enters this calculation. The requested regime count is a parameter, not an estimated count of true changes. App overlays retain original integer window positions, inclusive interval edges and candidate boundaries. These retrospective annotations use the complete sequence and cannot establish an online detection time.

Figure 5 places the estimated windows and boundaries on original observation positions. The annotations are algorithm outputs, not supplied ground-truth regime labels.

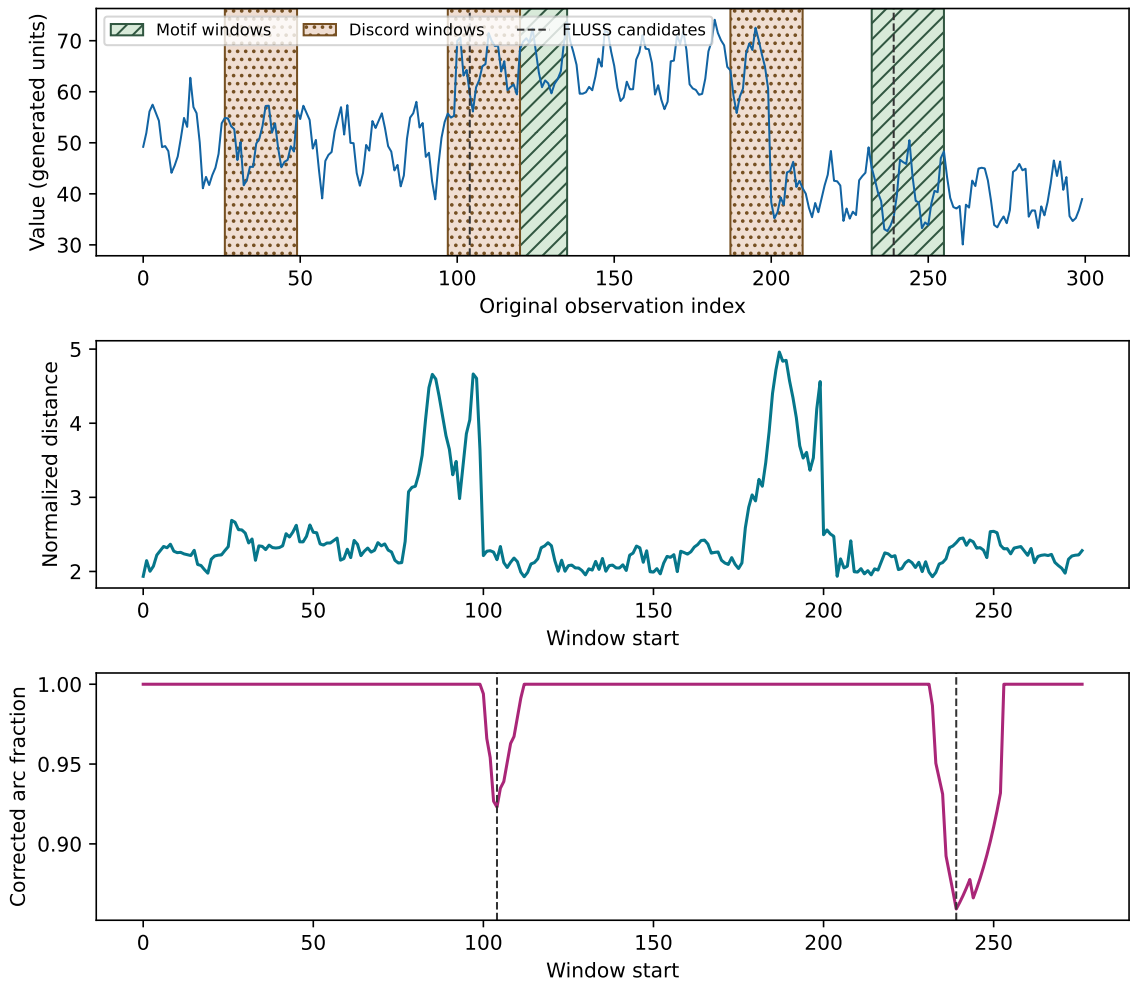


Figure 5: Original-axis sequence annotations for the structural-break case. Hatched spans label the motif and discord windows, and dashed boundaries mark FLUSS candidates. Neither the window ranks nor the requested segmentation count establishes ground-truth anomalies.

Costa multiscale entropy averages non-overlapping blocks at each scale τ [23]. Sample entropy uses order two, tolerance $r = 0.15s(x)$ fixed to the original population standard deviation, and the same $N_\tau - 2$ template starts for dimensions two and three. Distinct unordered pairs match at Chebyshev distance at most r, excluding self matches. If B_τ pairs match at dimension two and A_τ extensions match, the estimate is

$$\mathrm{SampEn}_\tau = -\log(A_\tau / B_\tau). \tag{43}$$

Fewer than twenty coarse observations, or no matching extensions, yield an unavailable estimate with a reason and exported counts. An unavailable value is not zero complexity. Constant sequences give zero where all eligible templates match.

Binary Lempel–Ziv complexity uses $1[x_t > \text{median}(x)]$, assigning ties to zero. Both phrase count $c(n)$ and $c(n)\log_2(n)/n$ are retained [24]. The asymptotic normalization is not clipped to one. Time-reversal asymmetry reports the mean of $x_{t+\ell}(x_{t+2\ell}^2 - x_t^2)$ at lags one, two and seven, where available, on raw and standardized observations. Raw units are cubed source units. Reversal changes the sign, but a finite nonzero statistic alone is not a significance test or an irreversibility test.

Algorithm 16 follows the retained original-axis record from input admission to the four kinds of output.

Algorithm 16 Compute original-axis sequence annotations and complexity measures from an admitted contiguous observation record.

Require: Finite contiguous observations x and seasonal lag m
1: Select the profile window from $\min(\max(8, 2m), 64, \lfloor n/3 \rfloor)$
2: **if** the profile is eligible and the record is nonconstant **then**
3: Compute the self join, nonoverlapping ranked windows and the FLUSS arc summary
4: **else**
5: Record why the profile is unavailable
6: **end if**
7: For each coarse scale, count template matches and compute available sample entropy
8: Threshold at the median and retain raw and normalized Lempel–Ziv phrase measures
9: For each admissible lag in $\{1, 2, 7\}$, evaluate raw and standardized time-reversal asymmetry
10: Bind all retained arrays and window positions to the original observation axis

Figure 6 shows the entropy calculation as coarsening reduces available template counts; its curve is not a calibrated measure of forecasting ability.

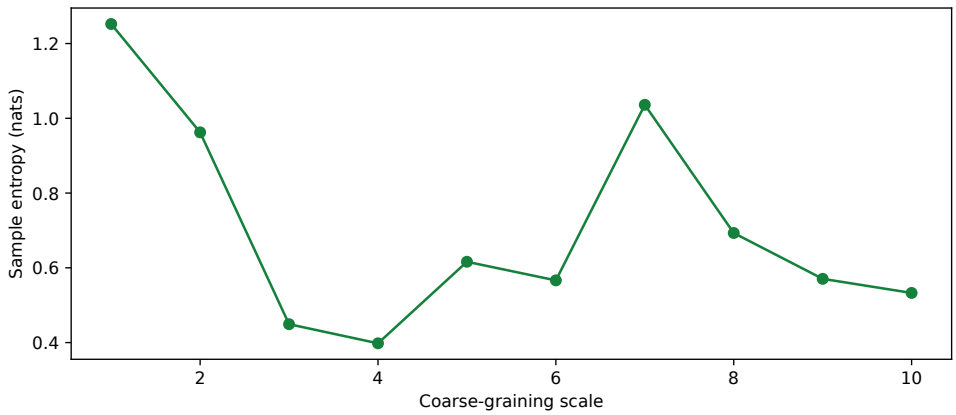


Figure 6: Multiscale entropy on the structural-break case with the original tolerance fixed across scales. Coarse sequence lengths decrease with scale; finite-sample availability and matching counts remain part of the interpretation.

The previous nolds correlation-dimension and Lyapunov defaults used randomized RANSAC fits without an explicit random state. The adapter now obtains deterministic log-distance fitting data, then fits the same robust polynomial-feature linear model with an explicit local seed. Least squares is the fallback when consensus fails. A case seed is derived from SHA-256 of case identity and the supplied pipeline seed; surrogate and 0–1 calculations receive derived local seeds. Process-global

random state is not changed. DFA and fallback R/S slopes explicitly request least squares. These fit choices can move historical diagnostics even when the source observations are unchanged. Two complete analyzer calls for every case must serialize identically before promotion; independent-process and caller-RNG tests additionally protect the estimator boundary. The existing surrogate gate remains necessary for a chaos classification.

14.9 Corpus forecastability and disjoint-series confirmation

A separate fixed-protocol experiment processes the complete Monash M4 Daily source, 4,227 series under CC BY 4.0 [30]. Every identity is admitted or assigned an explicit exclusion. Exact whole-sequence duplicates retain the lexicographically first identity for inference. Nonfinite observations, insufficient prefixes and unavailable MASE denominators are excluded with reasons. Source ZIP and extracted TSF hashes identify the input bytes; a full-sequence identity hash is used only for duplicate grouping and partition allocation, never as a predictor.

Each admitted series contributes three non-overlapping fourteen-step future blocks. Each model uses at most the preceding 1,024 contiguous observations at an origin. Features use only the earliest fitting prefix. Forecast seasonality is one, matching the M4 Daily setting; weekly phase strength is a separately named diagnostic. The fixed roster is Naive, SeasonalNaive, SES, Holt, HoltWinters, Theta, AutoETS and AutoTheta. Some configurations coincide when the period is one; these explicit identities do not imply eight distinct future paths. No deep or pretrained model participates in this corpus study, which does not replace the separate twenty-two-variant case experiment.

Model MASE is averaged over the three origins within each series. The association target is $\log(1 + \min_k \overline{\mathrm{MASE}}_k)$ over the fixed roster. This is an optimistic within-roster oracle, not inherent predictability. Forty prefix diagnostics include catch24, location/scale and ordering summaries, multiscale entropy, binary complexity, reversal asymmetry and profile distances. Associations may reflect a scaled-error denominator or source grouping rather than a causal forecasting mechanism.

Exact sequence identities allocate approximately seventy percent of series to discovery and the rest to confirmation. Kendall tests use available feature values and aggregate series targets. Benjamini–Yekutieli correction treats the entire feature family separately in each partition at $q = 0.05$ [31]. In sorted order, adjusted probabilities use

$$q_{(i)} = \min_{j \geq i} \left\{ 1, \frac{K H_K}{j} p_{(j)} \right\}, \qquad H_K = \sum_{k=1}^{K} k^{-1}. \tag{44}$$

Confirmation requires rejection in both partitions and matching effect direction. All feature counts, signed effects, raw probabilities, adjusted probabilities and negative decisions are retained. Series are the statistical units; origins are not treated as independent replications. Related industries or latent sources can still violate observation-independence assumptions after exact deduplication.

A fixed random forest predicts the best model identity from discovery features, with 200 trees, maximum depth eight, minimum leaf size ten and random state 42. Missing-value medians and the constant comparator are fitted on discovery only. Confirmation outcomes choose neither parameters nor feature thresholds. The selected rule is compared once on confirmation series with that constant method and the future-informed per-series oracle. Scores are descriptive held-out results, not population-superiority or operational guarantees. Negative selection outcomes remain in the report.

Algorithm 17 fixes where the partition enters the feature and selector experiment.

For the partition calculation, hash the ASCII string `chronoscope-atlas-v1:` followed by the lower-case hexadecimal digest of the little-endian float64 sequence. Read the first eight hex digits as the unsigned integer used in Algorithm 17.

Algorithm 17 Compare the discovery-trained selector and constant method on a separate set of admitted series.

Require: Declared M4 Daily source, fixed methods, prefix features and series partitions

1: Validate identities, retain the first exact duplicate identity and record exclusions
2: Assign discovery when the first 32 bits of the salted sequence hash modulo 100 is below 70
3: Compute features from the earliest fitting prefix, and MASE from the three future blocks for each method
4: Set the association target to $\log(1 + \min_j \overline{\mathrm{MASE}}_j)$ for each series
5: Use Kendall on at least 20 finite nonconstant feature pairs, with $(0, 1)$ for unavailable effects and probabilities
6: Apply Benjamini–Yekutieli correction within each partition
7: Retain a confirmation flag only for concordant adjusted discoveries in both partitions
8: Impute from discovery medians, using zero for an entirely missing discovery feature
9: Fit the selector and constant comparator on discovery rows, with model roster order resolving ties
10: Score both rules on confirmation rows and retain the future-informed oracle separately

The complete source contains 4,227 identities; 4,226 are admitted and 1 is explicitly excluded. The deterministic partitions contain 2,946 discovery and 1,280 confirmation series. Of forty tested features, 21 meet the fixed two-partition confirmation rule. The discovery-selected constant is AutoETS. On confirmation series, the fixed selector has mean MASE 3.3501, the constant 3.2570, and the future-informed oracle 3.0005. The selector's error is 0.0931 higher than the constant on this fixed split. This descriptive outcome is not a significance test of selection benefit. The oracle uses future outcomes and cannot be deployed as a selection policy.

Exclusions by reason: zero or unavailable MASE scale: 1.

Confirmation method	Mean MASE
AutoTheta	3.2195
Naive	3.2216
SeasonalNaive	3.2216
Theta	3.2228
SES	3.2263
AutoETS	3.2570
Holt	3.6217
HoltWinters	3.6217

Table 25 retains all forty tested prefix features and the adjusted probabilities in both partitions, rather than showing only the twenty-one confirmations.

Table 25: All forty prespecified features. D/C denote discovery/confirmation. Signed Kendall effects, BY-adjusted probabilities and the fixed confirmation decision are retained, including negative outcomes. Available counts and raw probabilities are in the public artifact.

Feature	n_D	τ_D	q_D	n_C	τ_C	q_C
acf_1	2946	0.012	1	1280	0.032	0.5
acf_7	2946	-0.004	1	1280	0.017	1
catch24_CO_Embed2_Dist_tau_d_expfit_meandiff*	2946	0.126	4.8e-23	1280	0.154	7.4e-15
catch24_CO_FirstMin_ac	2946	-0.063	3.4e-06	1280	-0.049	0.068
catch24_CO_HistogramAMI_even_2_5	2946	0.006	1	1280	0.004	1
catch24_CO_f1ecac*	2946	-0.094	4e-13	1280	-0.079	0.00023
catch24_CO_trev_1_num*	2946	0.145	4.7e-30	1280	0.173	3.9e-18
catch24_DN_HistogramMode_10	2946	-0.021	0.5	1280	-0.047	0.086
catch24_DN_HistogramMode_5	2946	-0.029	0.12	1280	-0.044	0.12
catch24_DN_Mean	2946	-0.035	0.03	1280	-0.045	0.1
catch24_DN_OutlierInclude_n_001_mdrmd*	2946	-0.042	0.0044	1280	-0.059	0.014
catch24_DN_OutlierInclude_p_001_mdrmd*	2946	0.136	1.6e-26	1280	0.167	2.6e-17
catch24_DN_Spread_Std	2946	-0.034	0.034	1280	-0.044	0.12
catch24_FC_LocalSimple_mean1_tauresrat*	2946	0.117	3.7e-20	1280	0.120	3.5e-09
catch24_FC_LocalSimple_mean3_stderr	2946	-0.003	1	1280	-0.023	1
catch24_IN_AutoMutualInfoStats_40_gaussian_fmmi*	2946	-0.088	1.2e-08	1280	-0.106	1.7e-05
catch24_MD_hrv_classic_pnn40	2946	0.010	1	1280	-0.005	1
catch24_PD_PeriodicityWang_th0_01*	2946	-0.077	3.2e-08	1280	-0.098	9.9e-06
catch24_SB_BinaryStats_diff_longstretch0*	2946	-0.074	2.2e-07	1280	-0.099	1e-05
catch24_SB_BinaryStats_mean_longstretch1*	2946	-0.110	1e-17	1280	-0.098	3.8e-06
catch24_SB_MotifThree_quantile_hh	2946	0.006	1	1280	-0.000	1
catch24_SB_TransitionMatrix_3ac_sumdiagcov*	2946	0.046	0.0031	1280	0.085	0.00017
catch24_SC_FluctAnal_2_dfa_50_1_2_logi_prop_r1*	2946	-0.080	3.6e-09	1280	-0.098	8.1e-06
catch24_SC_FluctAnal_2_rsrangefit_50_1_logi_prop_r1	2946	0.001	1	1280	-0.035	0.44
catch24_SP_Summaries_welch_rect_area_5_1*	2946	-0.064	2.2e-06	1280	-0.054	0.029
catch24_SP_Summaries_welch_rect_centroid*	2946	0.097	1.8e-11	1280	0.076	0.0025
history_length*	2946	-0.149	7.8e-26	1280	-0.156	1.4e-12
kurtosis	2946	0.029	0.11	1280	0.046	0.093
linear_trend_r*	2946	0.114	3.8e-19	1280	0.147	1.4e-13
log_std	2946	-0.035	0.034	1280	-0.044	0.12
lz_binary*	2946	0.041	0.007	1280	0.057	0.02
mse_1	2946	0.002	1	1280	-0.006	1
mse_10	2476	0.020	0.76	1071	-0.017	1
mse_2	2941	0.009	1	1278	0.003	1
mse_5	2834	0.010	1	1223	0.020	1
profile_max	2946	0.018	0.87	1280	0.014	1
profile_median*	2946	-0.050	0.00041	1280	-0.056	0.022
reversal_1*	2946	0.130	2e-24	1280	0.141	1.3e-12
skewness*	2946	0.083	1.6e-10	1280	0.094	8.1e-06
weekly_phase_strength*	2946	0.090	3.3e-12	1280	0.086	5.5e-05

* Rejected at BY $q \leq 0.05$ in both partitions with the same effect direction.

The evaluator retains external per-origin forecast-path audits and independently rescores every exported MASE before compact export. Its resumable cache binds source bytes, fixed protocol, implementation and package identities. Admission independently checks inventory, duplicate separation, temporal schedules, discovery-only imputation, aggregate scores, corrected probabilities and Kendall effects. The public notebook retrains the selector from admitted discovery rows and repeats reference statistical calculations without claiming a new fit of the eight external-corpus forecasting models. These layers distinguish full forecasting reproduction, compact-table statistical reproduction and browser inspection.

Figure 7 puts the discovery-trained selector beside the fixed comparison and the future-informed oracle, retaining the unfavorable held-out selection result.

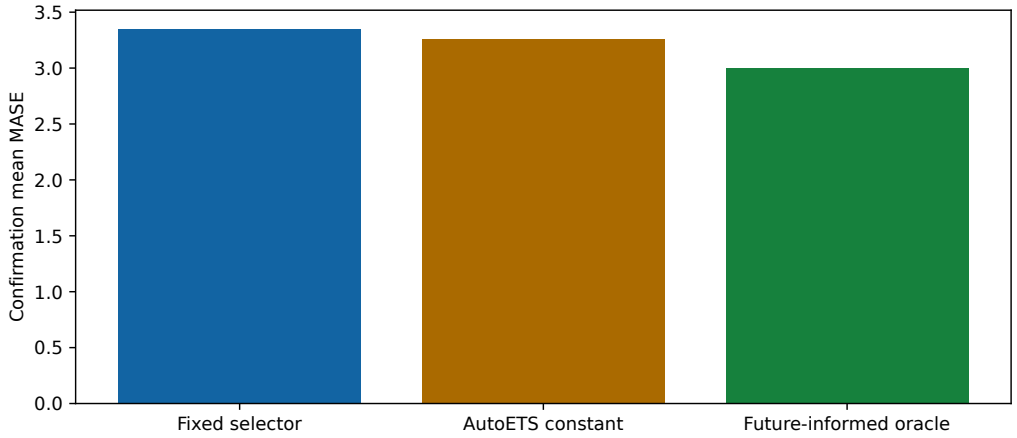


Figure 7: Mean MASE on confirmation series for the fixed discovery-trained selector, the discovery-selected constant method and the future-informed oracle. Bars are descriptive averages over the same admitted confirmation rows; they are not confidence intervals.

15 Held-out calibration of the browser linear forecast

The browser forecast is a global linear map fitted to synthetic windows, not the locally trained neural implementation that shares the NLinear name in the offline comparison. This distinction matters because a name does not identify a loss function, a training sample or an uncertainty procedure. The original browser matrix has 48 input coordinates and 12 output coordinates. It is estimated by least squares on 52,144 overlapping windows drawn from 400 generated series. Its point forecasts run through an actual ONNX matrix multiplication in the browser. Neither this small representation nor its execution speed establishes that its forecast distribution is calibrated.

In the previous implementation, a pooled training-residual standard deviation was multiplied by the observed window's scale and by the square root of the forecast lead. This rule produces finite bands, but no held-out experiment had established their nominal coverage. Training residuals reuse outcomes that helped estimate the map; pooling leads also discards the error distribution's dependence on forecast distance. Multiplying that pooled scale by another lead-dependent factor is an engineering approximation rather than a consequence of the fitted global regression. The following experiments evaluate that specific implemented rule. They do not treat Gaussian intervals as a uniformly weak baseline or extrapolate their behavior to every model with Gaussian errors.

The first experiment tested a fixed regularization grid and independent calibration. Its point-error

result was unfavorable. A second, separately specified experiment therefore retained the original point matrix and tested prefix-stratified interval calibration on fresh generated series. This sequence separates an observed failure from a new confirmation. It does not permit the first test set to become the second experiment's tuning set while still being described as independent confirmation. The complete first result remains in the record, including its validation decision and family-specific coverage.

15.1 Observed-window normalization and a fixed point map

Let $x = (x_1, \dots, x_L)$ be the last $L = 48$ observations and let a_h denote the target at lead $h \in \{1, \dots, 12\}$. All normalization quantities are computed from x, before any target is revealed. Define

$$\bar{x} = L^{-1} \sum_{i=1}^{L} x_i, \qquad s(x) = \left[L^{-1} \sum_{i=1}^{L} (x_i - \bar{x})^2 \right]^{1/2}, \qquad z_i = \frac{x_i - x_L}{s(x)}. \tag{45}$$

For a matrix $W \in \mathbb{R}^{L \times H}$, the normalized prediction is $\hat{u} = z^\top W$ and the prediction in the observation's units is

$$\hat{a}_h = x_L + s(x)\hat{u}_h. \tag{46}$$

The matrix contains no intercept. Subtraction of the last observation fixes the origin used by the exported model; subtraction of the sample mean would define a different predictor. The denominator is the population standard deviation of the observed window, not the residual standard deviation of a trend fit. The training recipe and browser implementation use this same convention.

If $s(x) = 0$, the normalized model is unavailable. Substituting a raw-unit scale of one would change the computation when the same constant sequence is expressed in different units. A constant sequence can be forecast by a separately declared constant predictor, but that is not evidence for assigning a calibrated uncertainty scale to this normalized map. Nonfinite observations, unsupported horizons and malformed matrix outputs are also refused. The 12-lead model does not generate additional leads by repeating its last output.

The deployed matrix is preserved byte-for-byte throughout both experiments. Its SHA-256 is recorded with the calibration metadata, and the browser verifies the actual downloaded graph against that digest before inference. This prevents a valid calibration array from being accidentally paired with a different point map. Numerical parity is checked independently using the float32 graph and a NumPy multiplication. Agreement of those computations is a numerical identity check; it is not a forecasting-performance claim.

15.2 Regularization, selection and the first held-out experiment

The training collection uses the original generator and random sequence. Each series is assigned one of four equally probable mechanisms: sinusoidal seasonality, trend plus seasonality, trend, or independent noise. Periods are selected from six, twelve and twenty-four observations; series lengths are sampled from the integer interval $[120, 260)$. These are controlled mechanisms, not observations from the electricity, air-quality or competition datasets. The overlapping windows within one training series share observations. They therefore cannot be used as independent calibration or test replications.

For normalized training rows Z and targets U, the candidate family minimizes

$$\widehat{W}_\lambda = \operatorname*{argmin}_W \left\{ \frac{1}{N} \|U - ZW\|_F^2 + \lambda \|W\|_F^2 \right\}. \tag{47}$$

The fixed grid is $\{0, 10^{-6}, 10^{-4}, 10^{-2}, 10^{-1}, 1, 10\}$. The zero entry uses a least-squares solver, including when the Gram matrix is singular. Positive penalties solve the regularized normal equations. Candidate matrices are converted to float32 before validation. Selection minimizes mean normalized squared error on the final complete window of each of 200 independent validation series. Exact ties select the earlier grid entry. The selected matrix is not refitted using the validation or calibration series.

Four hundred additional series supply calibration errors and another four hundred supply test errors. A series contributes only its final complete 48-to-12 window outside training. The training, validation, calibration and test generators use local PCG64 seeds 20260704, 20261105, 20261106 and 20261107, respectively. These integers identify generator streams; they are not dates of data collection. Full-sequence hashes verify that no series is duplicated across any partition. Family labels are retained for evaluation, but do not enter the fitted linear map.

Validation selected $\lambda = 0.1$. On the independent test sample, normalized mean squared error was 0.550561, compared with 0.548742 for the frozen original map. The paired difference was +0.001819, with an exploratory 95% series bootstrap interval $[0.000145, 0.003465]$. Only 150 of 400 series had lower squared loss. Thus the validation-selected ridge candidate did not improve the point forecast in this experiment. This outcome is retained rather than replaced by the favorable validation score or a new penalty chosen from test outcomes. A repeat-last control had normalized squared error 1.795089, which locates both linear maps relative to a simpler forecast without making either map an unconditional winner outside this generator.

15.3 Per-lead split calibration and its conditional limits

For a fixed point model and independent calibration series indexed by j, define the absolute normalized error at lead h as

$$R_{jh} = \left| \frac{a_{jh} - \hat{a}_{jh}}{s(x_j)} \right|. \tag{48}$$

With n calibration series and miscoverage $\alpha = 0.2$, the radius is the kth order statistic,

$$k = \lceil (n+1)(1-\alpha) \rceil, \qquad q_h = R_{(k)h}, \qquad I_h(x) = [\hat{a}_h - s(x)q_h, \hat{a}_h + s(x)q_h]. \tag{49}$$

If $k > n$, a finite empirical radius is not available under this convention. Interpolation of the nominal 80th percentile is not substituted for the specified finite-sample rank. For $n = 400$, the selected one-based rank is 321. Calibration uses the actual runtime output of the exported point model, so calibration and testing apply the same numerical predictor.

For independent exchangeable calibration and test series and a point model fixed before calibration, the rank of the new residual among the $n+1$ residuals is uniform in the absence of ties; conservative tie handling gives the usual lower coverage bound. Consequently

$$\Pr\{a_h \in I_h(x)\} \geq 1 - \alpha. \tag{50}$$

This is a marginal statement over new series from the same population, separately for each lead [34]. It does not say that all twelve future observations are covered together with probability 0.8. Nor does it guarantee 0.8 conditional coverage for each observed prefix, each latent generator family or an arbitrary authentic series. Internal serial dependence within one generated series does not by itself invalidate exchangeability across independently generated series; pooling dependent windows as separate calibration observations would be a different design.

In the first test sample, the calibrated ridge intervals had 79.46% overall coverage, normalized mean width 1.659786 and normalized interval score 2.781262. The original Gaussian rule had 96.60% coverage, width 4.593046 and score 4.764406. However, the calibrated candidate's white-noise coverage was only 53.06%, while seasonal and trend-plus-seasonal coverage exceeded 93%. The

aggregate result therefore could not support a statement that all mechanisms were calibrated. It motivated a new prefix-stratified procedure and a fresh confirmation population, while leaving the original result intact.

15.4 Prefix-roughness strata and independent confirmation

The second experiment changes intervals while preserving exactly the original point predictions. Its observable grouping statistic is

$$\rho(x) = \frac{(L-1)^{-1}\sum_{i=1}^{L-1}(x_{i+1}-x_i)^2}{L^{-1}\sum_{i=1}^{L}(x_i-\bar{x})^2}. \tag{51}$$

The three fixed bins are $[0, 0.5)$, $[0.5, 1.25)$ and $[1.25, \infty)$. This ratio compares local changes with total observed variation. Independent noise has population ratio near two; a well-sampled sinusoid has ratio approximately $2\{1-\cos(2\pi/m)\}$. These relationships motivate separating smooth, oscillatory and rough prefixes without using a hidden mechanism label. The boundaries are engineering choices fixed before the new confirmation outcomes, rather than estimated cut points optimized on those outcomes.

For a bin b, the calibration radius uses only series whose observed prefixes belong to b:

$$q_{bh} = R^{(b)}_{(k_b)h}, \qquad k_b = \lceil (n_b+1)(1-\alpha) \rceil. \tag{52}$$

At least twenty independent calibration series are required in every bin. Conditional on a fixed prefix partition and exchangeable series within a bin, the rank argument applies separately in that bin. It does not acquire validity within every latent family simply because the bin was computed from a prefix. The number of calibration series per bin affects the granularity and sampling variability of each radius; finer partitions can therefore reduce efficiency or leave a bin without an admissible radius.

The confirmation uses 400 new calibration series from seed 20261109 and 800 new test series from seed 20261110. Their full-sequence hashes are disjoint from all 1,400 series in the first experiment. The point map is not trained, selected or updated in this confirmation. Its Gaussian, globally calibrated and prefix-stratified interval arms use identical points at every test origin. Calibration bin counts were 115, 120 and 165. This yields finite-sample ranks 93, 97 and 133, respectively. The global arm uses all 400 calibration series and provides a matching ablation of the prefix partition.

Algorithm 18 Fixed-point forecast with prefix-stratified marginal intervals. Every quantity used at prediction time is determined before observing the next target.

Require: Frozen point matrix W, independently calibrated radii q_{bh}, observed prefix x, supported horizon $H = 12$

Refuse nonfinite observations, an incomplete lookback or $s(x) \le 0$
Compute z from Equation 45
Select the fixed bin b containing $\rho(x)$
Compute $\hat{u} = z^\top W$ using the verified float32 ONNX graph
for $h = 1, \ldots, H$ **do**
    Set $\hat{a}_h = x_L + s(x)\hat{u}_h$
    Store $I_h = [\hat{a}_h - s(x)q_{bh}, \hat{a}_h + s(x)q_{bh}]$
end for
return $(I_1, \ldots, I_H)$

Algorithm 18 requires $O(LH + L)$ arithmetic after the small fixed graph has been loaded. It carries no recurrent hidden state and performs no online refit. Hash verification binds the graph to the radii; it does not identify whether a new series belongs to the calibration population. The browser

therefore describes the intervals as nominal marginal bands calibrated on synthetic prefixes, with authentic-data coverage unconfirmed. Changing the requested horizon or nominal level requires new calibration metadata instead of silently extrapolating the existing arrays.

15.5 Scores, uncertainty and confirmation outcomes

Each test series receives equal weight. Point errors are divided by the observed lookback standard deviation and then squared or taken in absolute value. These are normalized errors, not MASE. For lower and upper bounds ℓ_{jh} and u_{jh}, the normalized interval score is

$$\mathrm{NIS}_j = \frac{1}{Hs(x_j)} \sum_{h=1}^{H} \left[(u_{jh} - \ell_{jh}) + \frac{2}{\alpha}(\ell_{jh} - a_{jh})_+ + \frac{2}{\alpha}(a_{jh} - u_{jh})_+ \right]. \tag{53}$$

It penalizes both width and misses, rather than rewarding coverage without regard to efficiency. The normalization is not the seasonal-naive denominator used by MSIS elsewhere in this report. Comparisons are made within this experiment and are not pooled with differently scaled losses from the external forecast-combination study.

Table 26: Independent synthetic confirmation on 800 new series. All three interval arms use identical frozen point forecasts and the same observed-prefix normalization; smaller interval scores represent more efficient predictions.

Quantity	Gaussian	Global	Stratified
Normalized squared error	0.548553	0.548553	0.548553
Marginal coverage	0.970521	0.834583	0.827292
Normalized width	4.593046	1.873393	1.738557
Normalized interval score	4.749003	2.803800	2.393376

Table 26 shows a 49.60% reduction in mean interval score relative to the current Gaussian rule and a 14.64% reduction relative to global calibration. These improvements concern intervals around unchanged point forecasts. Figure 8 separates interval score from width: a narrower band is not automatically better if its miss penalties increase. In this confirmation, stratification reduces both the mean width and the mean proper interval score relative to global calibration.

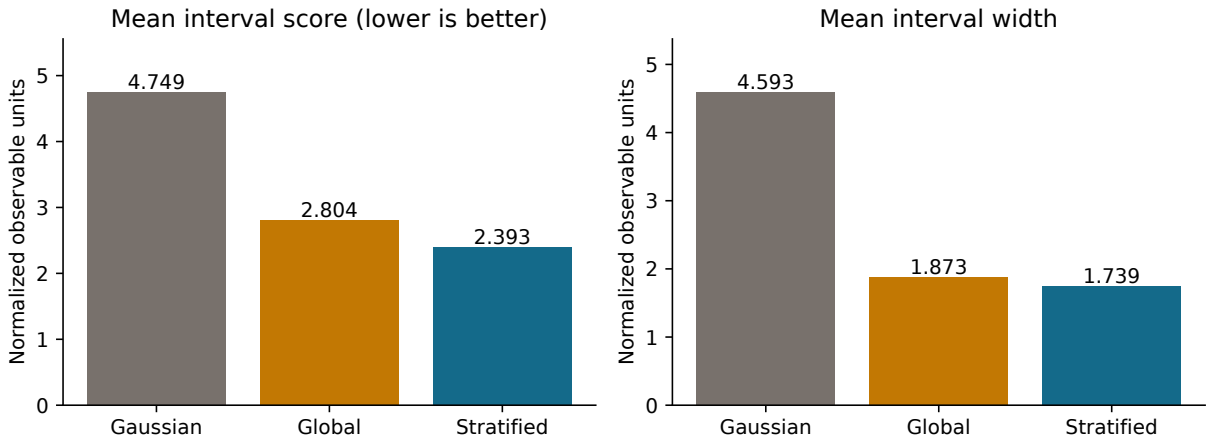


Figure 8: Mean interval score and width from all 800 independent confirmation series. The three procedures share point forecasts; these plotted means do not represent independently estimated confidence intervals.

Uncertainty is summarized by drawing 10,000 paired bootstrap samples of the 800 complete series with local PCG64 seed 20261108. Within a draw, the same series indices are used for

both interval procedures. Forecast leads are not resampled as independent observations. The stratified-minus-Gaussian mean interval-score difference is -2.355627, with exploratory 95% percentile interval $[-2.437258, -2.273077]$. The stratified-minus-global difference is -0.410424, with interval $[-0.451789, -0.368795]$. These are unadjusted exploratory intervals under the experiment's independent-series design, not multiplicity-controlled evidence about every reported subgroup. Respectively, 778 and 606 of 800 series have strictly lower interval score, so the mean advantage does not conceal a majority of series with larger losses.

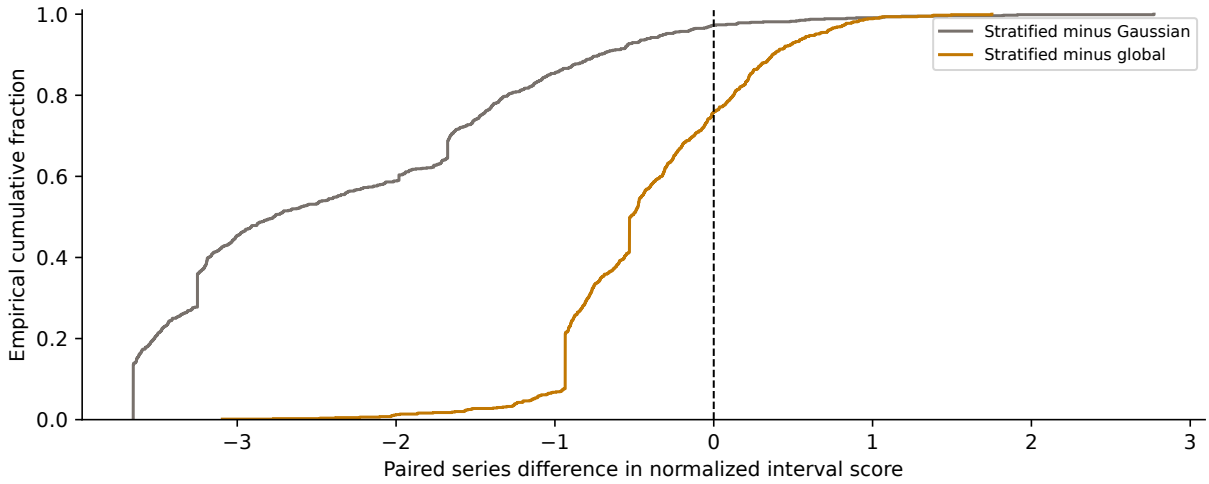


Figure 9: Empirical distributions of paired interval-score differences for individual confirmation series. Negative values favor prefix stratification; the positive tails retain series on which the comparison is unfavorable.

Figure 9 makes the individual-loss distribution visible alongside its aggregate uncertainty. Some prefixes benefit little and some have larger interval penalties after stratification. No series is removed because its interval score worsens. The empirical distribution is a description of these 800 generated series; its tails cannot establish the frequency of rare failures in an operational population absent from the generator.

Table 27: Prefix-bin calibration support and held-out coverage. Fixed bins use observed histories alone; their coverage summaries do not guarantee validity for every hidden generating mechanism or individual prefix.

Observed-prefix roughness	Calibration series	Test coverage
$[0, 0.5)$	115	0.843377
$[0.5, 1.25)$	120	0.804773
$[1.25, \infty)$	165	0.826786

Table 27 reports coverage in the observable bins targeted by the stratified construction. Family-specific results remain distinct. Coverage is 80.46% for noise, 74.57% for trend, 86.71% for seasonality and 88.92% for trend plus seasonality. Figure 10 shows that stratification addresses the large noise-family shortfall of global calibration without eliminating all family differences. In particular, the trend family remains below nominal coverage. These family outcomes were not used to redraw the bins after the confirmation was scored.

Figure 11 shows lead-specific coverage, ranging from 79.50% to 85.13% for the stratified rule. Separate marginal radii allow different error distributions at different leads. They do not force a monotone square-root growth law and do not imply a joint prediction region. The experiment's prospectively fixed engineering criteria require identical point forecasts, adequate calibration

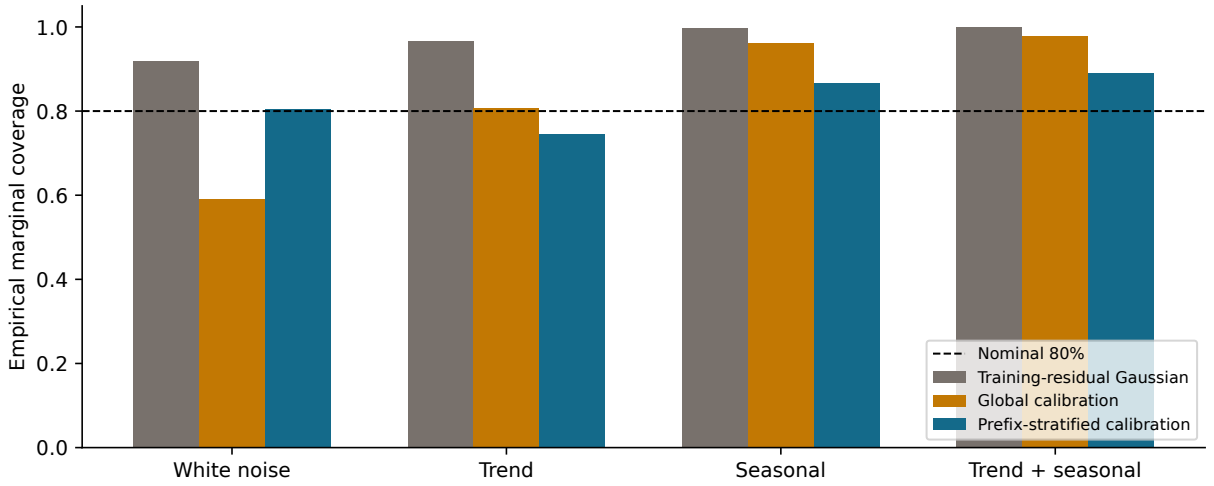


Figure 10: Confirmation coverage by latent generator family for identical point forecasts. Prefix stratification improves the noise-family shortfall, while the trend family's remaining undercoverage prevents a universal calibration claim.

support, lower mean interval score, overall coverage between 75% and 85%, bin coverage between 70% and 90%, and normalized runtime error below 10^{-4}. All six passed. This admission decision is explicitly weaker than demonstrating calibration on real data.

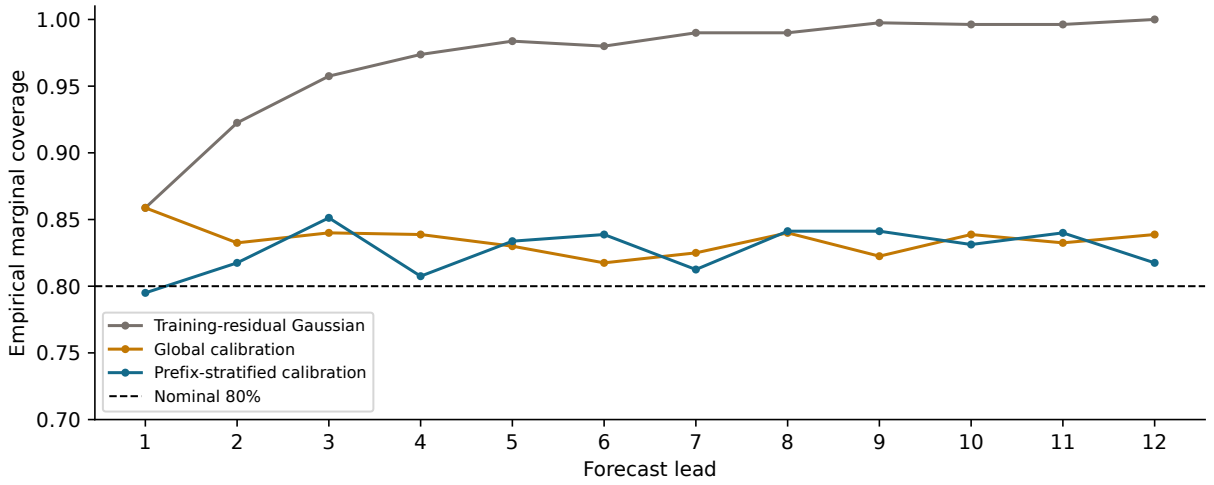


Figure 11: Empirical coverage at each of twelve future leads. The dashed line marks nominal marginal coverage; the graph does not report simultaneous coverage of an entire forecast trajectory.

15.6 Unit invariance, reproducibility and applicability

Consider a positive change of units and location, $x'_i = ax_i + b$, $a > 0$. Then $s(x') = as(x)$ and $z'_i = z_i$. The frozen matrix therefore returns the same normalized point vector. Equation 46 gives $\hat{a}'_h = a\hat{a}_h + b$. Both the numerator and denominator of Equation 51 are multiplied by a^2, so the selected bin is unchanged. The calibrated endpoints transform as

$$I'_h(x') = aI_h(x) + b. \tag{54}$$

Normalized errors, widths and interval scores are consequently unchanged in exact arithmetic. Float32 execution and finite-precision computation of the observed scale introduce small numerical

differences, which are tested with explicit tolerances. This invariance explains why an observed-data scale is required and why a fallback measured in arbitrary raw units is inadmissible. It does not prove correctness of every other method in the product.

All 2,600 generated series across the two experiments are retained, together with their full-sequence hashes and every test target, point prediction and interval endpoint. The first experiment was executed in two separate processes and every generated output byte matched. An independent checker imports no training, calibration or scoring producer: it recomputes all stored losses, prefix groups, partition disjointness and source bindings. Its runtime mode also reconstructs every calibration radius and test prediction directly from the stored graph weights and actual ONNX execution. The independently measured maximum normalized arithmetic discrepancy is below 10^{-6}.

These checks support an auditable computational conclusion: within the specified synthetic mixture, the stratified interval rule improves interval score around the preserved point forecast. They do not establish an advantage for the discarded ridge point model, generalize the generator to authentic industrial observations, or validate an action policy based on the bands. Before such use, the relevant population, observation revisions, forecast horizon, loss asymmetry and calibration/update schedule must be specified and tested with independent data. The browser retains these limits in its description instead of presenting the nominal level as an unconditional promise for every uploaded series.

16 Validation and scientific limitations

The validation layers are distinct. Structural schema checks establish artifact shape; source bindings establish input identity; independent equation tests establish selected numerical behaviours; browser traversal measures rendered routes and interactions; local notebook reproduction measures executable teaching artifacts. A successful build alone establishes none of the other layers. Computational records distinguish local numerical checks from remote CI, deployment and owner acceptance.

The fifteen-case forecasting experiment has these scientific limitations: one origin per selected series; a small, non-random collection; configuration-specific runtime and historical evidence retained in a separate experiment; unverified pretrained-data overlap; bounded historical training budgets; approximate lightweight uncertainty; possible numerical convergence issues in freshly executed reference adapters; and retrospective diagnostic artifacts that may include the final block. No operational acceptance or economic benefit is implied. For those twenty-two configurations, a broader study still needs predeclared datasets, repeated origins, runtime and memory budgets, dependence-aware comparisons and independent pretraining-overlap checks. The M4 Daily study below extends series-level evidence for its separate eight-method roster.

For the contextual proposal, a stronger follow-up also needs faithful FFORMA and arbitrated-ensemble comparators rather than only internal ablations. Such work could reject or improve the proposed recurrence rule. The fixed-context negative outcome remains informative without being promoted to a general impossibility result.

Numerical checks use different reference constructions where feasible. Seasonal continuation is checked on an incomplete cycle so that restarting phase would fail. Finite-history Theta is compared with a two-line derivation rather than a second transcription of its final formula. Exact profile distances are checked against exhaustive distances on short sequences. Entropy calculations retain independent matching-count examples, and corrected probabilities are recomputed without relying only on the plotting artifact. These checks target concrete mistakes; passing them does not prove every input regime correct.

Browser evidence adds another boundary. A late asynchronous result must not acquire a newer

case identity, a replay-decimated history must not become a contiguous fit input, and an optional combination record must not join a different prefix. Tests submit these mismatched inputs to verify each refusal. Frontend build success, pointer traversal, keyboard interaction and idle behavior are measured separately. The latest walkthrough increment still requires its complete final browser matrix, and the artifact admission result is not a substitute for that rendered check. Independent reader review and complete dimensional review remain separate requirements before a new publication or release can be accepted.

17 Discussion

The combined evidence supports a disciplined distinction between improving a component and establishing a general solution. The fixed contextual combination does not beat its matching recency control on the selected development blocks. Temporally validated combinations improve the external collection mean point error against that control, while AutoTheta remains stronger on monthly point error and both interval-score comparisons. These outcomes suggest that candidate selection and regularization deserve attention, but they do not isolate the context kernel as the reason for the gain. A faithful comparison with established feature-dependent and arbitrated combination methods remains necessary before making a broader comparative claim.

The browser experiments illustrate a different improvement route. The validation-selected ridge candidate slightly worsens independent point error, so the served point graph remains unchanged. Prefix-stratified residual calibration then improves interval score on an independently generated confirmation collection using that same graph. This gain addresses the uncertainty procedure rather than the point predictor. Its value lies in the paired comparison, fixed evaluation protocol and retained subgroup failures. A stronger point predictor could still be useful, but this experiment supplies no basis for saying that adding regularization automatically creates one.

The daily corpus result similarly separates descriptive association from useful model selection. Features can have reproducible associations with a within-roster error target while a discovery-trained selector performs worse than a simple automatic method on confirmation series. The association target is itself an optimistic oracle over a finite roster, and it can depend on the normalization used by the loss. A feature plot should therefore help formulate a hypothesis rather than trigger an automatic operational choice. The workbench retains both the association table and the unsuccessful selection result so that readers can inspect this distinction.

For the intended research and teaching use, the practical recommendation is to begin with the source, information cutoff and score definition before interpreting a displayed winner. A user should compare point and interval behavior together, inspect individual series as well as aggregate means, and verify whether a displayed result is a fresh fit or an archived replay. Subsequent development can test better-supported alternatives on prospectively specified authentic data and matching resource budgets. The present evidence supports those next experiments and an inspectable evaluation instrument; it does not identify a universally preferred forecast method or an accepted business policy.

18 Limitations and threats to validity

The first limitation is sampling. Fifteen deliberately selected cases demonstrate behavior and permit detailed source reconstruction, but they do not estimate accuracy for an identified deployment population. Eleven are generated mechanisms and four are real excerpts. A case with a known generator can reveal leakage, phase errors or an unstable scale; its controlled simplicity can also exclude the missingness, changing definitions and interventions encountered in practice. The larger external studies improve inventory accounting for their specified collections without turning those

collections into a random sample of future customer demand.

The second limitation is method comparability. The twenty-two configurations have different context caps, training budgets and historical origins. A direct adaptation may omit an originating paper's architectural components. A maintained package can change default behavior between versions, while a pretrained model can contain related data in an undocumented training corpus. The report addresses implementation identity and source-bound execution, but does not establish that all model families received equally effective tuning or equal computational resources. A common target block is necessary for the stated comparison and insufficient for a general architectural conclusion.

The third limitation concerns statistical interpretation. Origins within a series can be dependent; different series can share a latent source after exact deduplication. The paired-series bootstrap for the external combination study resamples series and is exploratory, with no multiple-comparison adjustment. The disjoint-series feature study controls its declared feature-test family but cannot convert association into causation. The browser confirmation tests generated histories under a fixed mixture of mechanisms. Its marginal coverage and interval-score improvement do not establish authentic-data coverage, simultaneous path coverage or validity within every conditioning group. A remaining trend-family undercoverage is materially relevant to interpretation.

The fourth limitation is engineering and reproducibility scope. Historical streaming hardware details and the original controller-gain rationale were not recovered. Reconstructed arrays can reproduce numerical trajectories without restoring acquisition history or a normalized latency experiment. The internal Python distribution has not yet completed the required reusable-library boundary work, and the existing lesson executions do not establish compliance with the newer teaching-content requirements. Complete observable-unit review and independent reader review remain open. The present document is a working revision, so it is not evidence of a newly accepted publication or completed application release.

19 Reproduction and availability

Implementations and admitted numerical outputs are available in the source snapshot for the archived studies.

The source tree contains the corrected computation in `chronoscopelab.reevaluation`, the fixed proposal in `model/context_memory.py`, original released traces, admitted source CSVs and the generated comparison. Running `scripts/reevaluate.py` regenerates the common final-block artifact while preserving the original traces. The staged local workflow provides complete candidate admission and explicit promotion. The original fresh-study implementation bytes are retained in a 56-source archive with a verified inventory. The historical execution commit was unavailable. The reproducibility instructions beside this report identify the distinct commands for numerical checks, figure generation and the clean LaTeX build.

The numerical tables and figures are generated directly from the two admitted forecast comparisons, full-sequence diagnostics and complete M4 study. The research report generator checks independent corpus statistics before producing its tables. The new lesson authoring script records the exact admitted corpus results; fresh-kernel execution remains a separate step. Code and compact admitted data use the repository's software/data licences; prose uses the licence stated in the header. Numerical reproduction does not establish representative population sampling, operational adoption or calibrated uncertainty for a future deployment.

20 Conclusions

ChronoScope supports source-bound sequence diagnostics and finite forecast evaluation rather than a general claim of forecasting superiority. The corrected selected-case comparison aligns original contiguous prefixes and reserved targets while retaining each implementation's training and context policy. The fixed contextual rule does not improve its matching recency control. Temporal validation yields lower mean MASE for ValidationMix than RecencyMean on the admitted weekly and monthly collections, .2855 versus .3109 and .8993 versus .9085, while AutoTheta remains better on monthly point error and both collections' interval score. These differences identify bounded empirical findings and unresolved comparative questions.

The four Timestral sizes were run on those same fifteen blocks. The planned 209M minus AutoTheta mean case MASE difference is -0.44740. The paired case-reweighting interval runs from -0.79356 to -0.15991. These outcomes describe the selected univariate comparisons and leave pretraining overlap and transfer to new source populations unresolved.

A separate browser experiment improves uncertainty without changing its point graph. On eight hundred fresh generated series, prefix-stratified calibration lowers mean normalized interval score from 4.749003 to 2.393376. The paired difference is -2.355627, with an exploratory 95 percent interval [-2.437258,-2.273077]. Marginal coverage is 82.73 percent, and subgroup differences remain. Neither this synthetic result nor successful authentic-history inference establishes authentic-data calibration.

The next scientific steps are matching comparisons with established combination methods, prospectively specified authentic-data interval evaluation, and information-arrival-aware testing of any native stateful forecaster. The next engineering steps include complete method-level teaching coverage, reusable-library separation, dimensional review and independent manuscript review. Those obligations remain explicit rather than being inferred from compilation, artifact admission or a responsive browser.

A Nested data fields and scientific admission

Table 28 enumerates nested fields expressed in the normative JSON Schemas. A requirement marker refers to the immediate parent or alternative branch. An empty constraint cell records the absence of an additional explicit schema constraint. In particular, an open object admits a family of records without enumerating all fields. The open diagnostic objects below retain their producer semantics even when those semantics have no corresponding nested structural restriction. The values U, U^2 and U^3 denote the source measurement unit, its square and its cube. A dimensionless ratio is comparable across source scales only under its stated normalization and eligibility rules.

Trace and case membership. The case index lists each case identifier, category and relative manifest path. Its count, unique identifiers and one-to-one subject binding are checked against the actual manifest inventory. A trace's method paths contain one lower, central and upper value in U per reserved lead. The backtest record contains dimensionless MASE, coverage and per-lead scaled errors, MAE and RMSE in U, and a count of completed windows. Null scores represent unavailable measurements, including a missing seasonal scale. Redaction can omit paths and empty display histories under the permission flags. A reserved lead is not an index into the decimated display history.

Manifest references and permissions. The nested engine identifies package, version and model role. Provenance carries source, licence and redistribution permission. The series record counts observations, seasonal lag and horizon, and reports level and standard deviation in U,

fitted slope in U per observation and dimensionless correlation, seasonal strength and zero fraction. Artifact references pair a relative path and declared schema with an integer byte count. Optional analysis and streaming references can be absent without changing the selected trace identity. Lane gates carry eligibility, required wheels, byte and millisecond budgets and refusal reasons. The pipeline checks path identity, size and licence separately from JSON validation.

Streaming. Each method's rolling score vectors align with its retained cutoff positions. Coverage is a fraction and the scaled loss columns are dimensionless. The trailing finite window of up to 25 retained forecast outcomes controls the displayed local statistic. Cumulative time is measured in milliseconds. The final summary reduces the evaluated sequence and has a different denominator from the local trajectory. A covariate record identifies whether the first series is past-only or known in advance, its allowed lag and the supplied values in the covariate's own units. A producer exception can yield a method object containing an error message. The success-row schema does not admit that object as a measured trajectory.

Analysis. The open stationarity records contain test statistics, probabilities, rejection decisions, recommended difference counts and labels. Autocorrelation arrays align with lag counts and use dimensionless values. Seasonality combines spectral frequencies in inverse observation steps, candidate periods in observations, strength ratios and additive component values in U. Filter trends and cycles are in U, while energy sums have squared coefficient units. Break records contain segment boundaries, level means and standard deviations, and optional latent-state and transition information. Volatility includes rolling sample statistics and fitted-transform settings, with leading nulls for incomplete windows. Distribution fields distinguish probability values, moment ratios, ordered sample values and density in inverse U. Fractal and nonlinear arrays use dimensionless fitted slopes or inverse-step rates as specified by the estimator. They retain skip and error states when a backend or input is unavailable. The sequence record separately binds the original observation indices, normalized profile distances and neighbor indices, corrected arc fractions, entropy tolerances in U, match and phrase counts, and reversal statistics in U^3 or standardized units. A missing or nonfinite estimate remains unavailable. The bivariate panel records a skipped state when an eligible paired input is absent.

The companion `next/tables/observed-analysis-fields.csv` records the actual nested keys and JSON types observed across the fifteen admitted analysis documents. Array lengths and counts of cases with each field expose optional, missing and mixed-shape records. This inventory describes the released observations, so it cannot expand the normative schema or admit a new shape in a later release. The method section beside each field gives its calculation and units.

The Catch24 dictionary in `next/tables/catch24-fields.csv` identifies the retained feature keys and their units or observation axes. The twenty-two shape and dynamics features use the backend's standardized input. The two additional location and scale features use the original values. A nonfinite feature is serialized as null. An unavailable backend omits the feature map and carries a reason. The keyed dictionary follows the original definitions and the pinned implementation [25].

Diagnostic field conventions. The panel count `n` measures cleaned observations. The root count precedes that cleaning. A position in a cleaned array is zero-based unless it is expressed as a positive lag or window length. Each `reference` identifies the procedure that produced its enclosing result. A panel-level `error` replaces that panel's success fields. Nested error objects replace only the affected Markov, multifractal, Box–Cox or GARCH result. Skip strings record eligibility decisions. Null is the serialization of an unavailable scalar or a nonfinite numerical result. In a retained vector it preserves the element position.

Stationarity and correlation fields. The ordered stationarity test array contains ADF, KPSS, Phillips–Perron, DFGLS and Zivot–Andrews. Its names identify the procedure and its null strings state the tested hypothesis. Statistics and their critical values are dimensionless and can have either sign. Critical-value keys identify nominal levels and differ by procedure. Probabilities lie in $[0, 1]$. The `stationary` and `combined_verdict` fields encode the test-specific rules in Section 14.2. The nonseasonal recommendation is an integer in $0, 1, 2$. The seasonal recommendation is zero or one. The seasonal recommendation and strength appear only when the declared period is at least two. The strength is a ratio in $[0, 1]$. ACF and PACF entries pair the coefficient with its lag in observations. ACF magnitudes are at most one up to rounding. An adjusted finite-sample PACF can exceed that bound. Box–Pierce and Ljung–Box store nonnegative statistics and their tested lag. The Durbin–Watson ratio lies in $[0, 4]$ for a nonzero denominator. The identification hint follows Section 14.3.

Seasonality, filters and changes. Candidate strength map keys are periods in observation steps. Values are the seasonal-strength ratios for the declared period and its double. A zero may represent an ineligible short input or a zero-variance denominator. The spectral bin arrays share an ordered frequency axis. The period and density units and the conditional STL summary are specified in Section 14.3. In the filter panel, HP lambda is the dimensionless coefficient of the second-difference penalty. EMD stores an IMF count and that many cleaned-axis rows, together with the residual. The scalogram stores aligned period and energy summaries under the conditions in Section 14.4. The change-point flag and note say whether the input was STL-deseasonalized. PELT stores its cost model, penalty in U^2, break positions and half-open segment statistics in U. CUSUM stores a nonnegative dimensionless statistic, probability and nonrejection flag. Markov states are zero-based. The transition matrix entries are probabilities whose columns sum to one up to rounding for a successful fit. The state path can be absent when the backend produces a nested error.

Volatility and distribution fields. The ARCH-LM decision records whether its probability is below .05. The lag count, rolling window and array alignment are specified in Section 14.5. The GARCH model label and two orders identify the fit. The fitted parameters and scale are interpreted there. Its nested skip indicates ARCH-LM nonrejection. The Box–Cox method label identifies Guerrero or the profile likelihood. The accompanying input-unit shift is needed alongside the dimensionless exponent. Distribution mean and standard deviation use U. Skewness and excess kurtosis are dimensionless ratios. The KDE grid and density align by grid index, with axes U and $1/U$. Q–Q theoretical and sample arrays align by rank. The fitted slope and intercept use U and the line correlation is dimensionless. Sample entropy is a natural logarithm ratio. Permutation and spectral entropy divide by their reference log cardinalities. The BDS and normality decisions and their null behavior are defined in Section 14.6.

Fractal and nonlinear fields. The R/S, DFA, graph dimensions, fractional-differencing proxy and MF-DFA slopes are dimensionless estimates. The implementation does not clip graph dimensions into a theoretical interval. The multifractal arrays share the ordered q axis and the width is nonnegative for a finite fit. The stored Boolean tests width strictly above .15. The Hurst label and length-only flag are specified in Section 14.6. In the nonlinear panel, the seed is a case-derived unsigned 32-bit integer. The correlation dimension is dimensionless and the Lyapunov estimate is per observation step. The 0–1 statistic is a correlation in $[-1, 1]$ when finite. RQA stores three fractions and a maximum diagonal length in steps. The surrogate result stores a completed-call count and a nullable decision. The archived diagnostic note is explanatory text. Its numerical criterion is specified in the method section. Below 400 input observations, the stage emits a skip in place of the nonlinear values.

Model and interval metadata. The ONNX file has float32 input shape $[1, 48]$ and output shape $[1, 12]$. The companion metadata identifies the input normalization, horizon and historical residual scale. Shape, graph digest and numerical parity are separate from the JSON check. The browser calibration record contains two roughness bin edges, a minimum count for each bin, three rows of twelve dimensionless radii and a twelve-lead global vector. All admitted radii are finite and nonnegative. Wrong graph identity, nonfinite normalization or a missing required radius refuses the interval. The separate experimental ridge record carries the seven penalty and normalized mean-square-error pairs, validation selection and twelve absolute-residual radii for its own graph. It depends on its own disjoint selection, calibration and test partitions.

Common comparisons. Each case holds the contiguous prefix, common reserved targets, seasonal lag and horizon. A method record has aligned forecasts, dimensionless MASE, MSIS, weighted quantile loss and coverage, plus MAE, RMSE and width in U. Scales come from the fitting prefix or the declared absolute target denominator. Optional expert allocations align with the declared order and historical origins. Fresh runs add seconds, warning categories, runtime versions and checkpoint identities. The fresh document binds its original implementation archive and source bytes. A valid JSON vector is still refused if the prefix, target, method roster, scaling or recomputed metric disagrees with the reference. The selector export uses method and mean-MASE pairs, a training-case count and a default. An empty-score fallback that serializes as null fails the numeric ranking constraint.

Forecastability and combinations. The corpus record stores three origin indices per included series, a split identity and complete-sequence hash, forty named nullable features and eight method scores. The features form a keyed object. The separate `selector_features` array fixes the order of the discovery imputation medians. Association records carry valid-pair count, Kendall coefficient, raw and adjusted probabilities and the confirmation decision. Exclusion records preserve the rejected identity and reason. The combination index distinguishes cohort inventory from admitted counts, summarizes scores after averaging within each series, and binds its source and audit digests. Each case document fixes prefix start, length and little-endian float64 digest, the horizon-length target, an ordered H by 3 array and five ordered weights where the method uses them. Memory origins and validation counts must satisfy the historical availability rule. Full candidate losses and validation splits live in the compressed archive; the small served case record retains the selected configuration and can expose an exclusion instead of an admissible path.

Table 28: Explicit nested JSON fields and schema constraints across the ChronoScope data interfaces. Requirement markers apply within the immediate parent record or branch.

Field path	JSON type	Explicit constraint	Req.
trace			
`quantile_levels[]`	number	No explicit range	item
`history_index[]`	integer	minimum=0	item
`history[]`	number	No explicit range	item
`actual[]`	number	No explicit range	item
`methods[]`	object		item
`methods[].name`	string	No explicit range	yes
`methods[].family`	string	No explicit range	yes
`methods[].backtest`	object		yes
`methods[].backtest.mase`	number/null	No explicit range	yes
`methods[].backtest.wql`	number/null	No explicit range	yes
`methods[].backtest.coverage`	number/null	No explicit range	yes
`methods[].backtest.mae`	number/null	No explicit range	yes
`methods[].backtest.rmse`	number/null	No explicit range	yes

Field path	JSON type	Explicit constraint	Req.
methods[].backtest.smape	number/null	No explicit range	yes
methods[].backtest.msis	number/null	No explicit range	yes
methods[].backtest.per_horizon_scaled	array		yes
methods[].backtest.per_horizon_scaled[]	number/null	No explicit range	item
methods[].backtest.n_windows	integer/null	minimum=0	yes
methods[].point	array		no
methods[].point[]	number	No explicit range	item
methods[].lower	array		no
methods[].lower[]	number	No explicit range	item
methods[].upper	array		no
methods[].upper[]	number	No explicit range	item
summary.best_method	string/null	No explicit range	yes
summary.best_mase	number/null	No explicit range	yes
summary.nominal_coverage	number/null	No explicit range	yes
manifest			
engine.package	string	No explicit range	yes
engine.version	string	No explicit range	yes
engine.model	string	No explicit range	yes
provenance.source	string	No explicit range	yes
provenance.license	string	No explicit range	yes
provenance.citation	string	No explicit range	yes
provenance.public_artifact_ok	boolean	No explicit range	yes
analysis_artifact[anyOf 1]	null	No explicit range	branch
analysis_artifact[anyOf 2]	object		branch
analysis_artifact[anyOf 2].path	string	No explicit range	yes
analysis_artifact[anyOf 2].format	string	No explicit range	yes
analysis_artifact[anyOf 2].analysis_schema	string	No explicit range	yes
analysis_artifact[anyOf 2].bytes	integer	minimum=0	yes
streaming_artifact[anyOf 1]	null	No explicit range	branch
streaming_artifact[anyOf 2]	object		branch
streaming_artifact[anyOf 2].path	string	No explicit range	yes
streaming_artifact[anyOf 2].format	string	No explicit range	yes
streaming_artifact[anyOf 2].streaming_schema	string	No explicit range	yes
streaming_artifact[anyOf 2].bytes	integer	minimum=0	yes
series.n_obs	integer	minimum=1	yes
series.seasonality	integer	minimum=1	yes
series.horizon	integer	minimum=1	yes
series.source	string	No explicit range	yes
series.mean	number	No explicit range	yes
series.std	number	No explicit range	yes
series.trend_slope	number	No explicit range	yes
series.seasonal_strength	number	No explicit range	yes
series.acf1	number	No explicit range	yes
series.pct_zeros	number	No explicit range	yes
artifact.path	string	No explicit range	yes
artifact.format	string	No explicit range	yes
artifact.trace_schema	string	No explicit range	yes
artifact.bytes	integer	minimum=0	yes
gate.lane	string	No explicit range	yes
gate.pure_python	boolean	No explicit range	yes
gate.wheels	array		yes
gate.wheels[]	string	No explicit range	item

Field path	JSON type	Explicit constraint	Req.
gate.trace_bytes	integer	minimum=0	yes
gate.run_ms_budget	number	No explicit range	yes
gate.trace_bytes_budget	integer	minimum=0	yes
gate.reasons	array		yes
gate.reasons[]	string	No explicit range	item
flags[]	object		item
flags[]{key}	string	No explicit range	keyed
metrics{key}	number/null	No explicit range	keyed
streaming			
methods{key}	object		keyed
methods{key}.cutoffs	array		yes
methods{key}.cutoffs[]	integer	minimum=0	item
methods{key}.rolling_mase	array		yes
methods{key}.rolling_mase[]	number/null	No explicit range	item
methods{key}.rolling_coverage	array		yes
methods{key}.rolling_coverage[]	number/null	No explicit range	item
methods{key}.rolling_wql	array		yes
methods{key}.rolling_wql[]	number/null	No explicit range	item
methods{key}.cumulative_cost_ms	array		yes
methods{key}.cumulative_cost_ms[]	number/null	No explicit range	item
methods{key}.n_steps	integer	minimum=0	yes
methods{key}.final	object		yes
methods{key}.final.mase	number/null	No explicit range	yes
methods{key}.final.coverage	number/null	No explicit range	yes
methods{key}.final.wql	number/null	No explicit range	yes
references{key}	string	No explicit range	keyed
analysis			
sequence[oneOf 1]	object		branch
sequence[oneOf 1].n	integer	minimum=0	yes
sequence[oneOf 1].observation_indices	array		yes
sequence[oneOf 1].observation_indices[]	integer	minimum=0	item
sequence[oneOf 1].values	array		yes
sequence[oneOf 1].values[]	number	No explicit range	item
sequence[oneOf 1].matrix_profile	declared union	No explicit range	yes
sequence[oneOf 1].matrix_profile[oneOf 1]	object		branch
sequence[oneOf 1].matrix_profile[oneOf 1].available	fixed value	const=true	yes
sequence[oneOf 1].matrix_profile[oneOf 1].n	integer	minimum=0	yes
sequence[oneOf 1].matrix_profile[oneOf 1].window	integer	minimum=0	yes
sequence[oneOf 1].matrix_profile[oneOf 1].exclusion_zone	integer	minimum=0	yes
sequence[oneOf 1].matrix_profile[oneOf 1].profile	array		yes
sequence[oneOf 1].matrix_profile[oneOf 1].profile[]	number/null	No explicit range	item
sequence[oneOf 1].matrix_profile[oneOf 1].nearest_neighbor	array		yes
sequence[oneOf 1].matrix_profile[oneOf 1].nearest_neighbor[]	integer	No explicit range	item

Field path	JSON type	Explicit constraint	Req.
sequence[oneOf 1].matrix_profile[oneOf 1].motifs	array		yes
sequence[oneOf 1].matrix_profile[oneOf 1].motifs[]	object		item
sequence[oneOf 1].matrix_profile[oneOf 1].motifs[].start	integer	minimum=0	yes
sequence[oneOf 1].matrix_profile[oneOf 1].motifs[].match	integer	minimum=0	yes
sequence[oneOf 1].matrix_profile[oneOf 1].motifs[].distance	number	No explicit range	yes
sequence[oneOf 1].matrix_profile[oneOf 1].discords	array		yes
sequence[oneOf 1].matrix_profile[oneOf 1].discords[]	object		item
sequence[oneOf 1].matrix_profile[oneOf 1].discords[].start	integer	minimum=0	yes
sequence[oneOf 1].matrix_profile[oneOf 1].discords[].distance	number	No explicit range	yes
sequence[oneOf 1].matrix_profile[oneOf 1].corrected_arc_curve	array		yes
sequence[oneOf 1].matrix_profile[oneOf 1].corrected_arc_curve[]	number	No explicit range	item
sequence[oneOf 1].matrix_profile[oneOf 1].regime_boundaries	array		yes
sequence[oneOf 1].matrix_profile[oneOf 1].regime_boundaries[]	integer	minimum=0	item
sequence[oneOf 1].matrix_profile[oneOf 1].regimes_requested	integer	minimum=0	yes
sequence[oneOf 1].matrix_profile[oneOf 1].normalization	string	No explicit range	yes
sequence[oneOf 1].matrix_profile[oneOf 1].note	string	No explicit range	yes
sequence[oneOf 1].matrix_profile[oneOf 2]	object		branch
sequence[oneOf 1].matrix_profile[oneOf 2].available	fixed value	const=false	yes
sequence[oneOf 1].matrix_profile[oneOf 2].reason	string	No explicit range	yes
sequence[oneOf 1].matrix_profile[oneOf 2].n	integer	minimum=0	no
sequence[oneOf 1].matrix_profile[oneOf 2].window	integer	minimum=0	no
sequence[oneOf 1].multiscale_entropy	object		yes
sequence[oneOf 1].multiscale_entropy.order	integer	minimum=0	yes
sequence[oneOf 1].multiscale_entropy.tolerance_ratio	number	No explicit range	yes
sequence[oneOf 1].multiscale_entropy.tolerance_basis	string	No explicit range	yes

Field path	JSON type	Explicit constraint	Req.
sequence[oneOf 1].multiscale_entropy.scales	array		yes
sequence[oneOf 1].multiscale_entropy.scales[]	object		item
sequence[oneOf 1].multiscale_entropy.scales[].scale	integer	minimum=0	yes
sequence[oneOf 1].multiscale_entropy.scales[].n	integer	minimum=0	yes
sequence[oneOf 1].multiscale_entropy.scales[].tolerance	number	No explicit range	yes
sequence[oneOf 1].multiscale_entropy.scales[].entropy	number/null	No explicit range	yes
sequence[oneOf 1].multiscale_entropy.scales[].matching_m	integer	minimum=0	yes
sequence[oneOf 1].multiscale_entropy.scales[].matching_m_plus_one	integer	minimum=0	yes
sequence[oneOf 1].multiscale_entropy.scales[].reason	string	No explicit range	no
sequence[oneOf 1].lempel_ziv	object		yes
sequence[oneOf 1].lempel_ziv.raw	integer	minimum=0	yes
sequence[oneOf 1].lempel_ziv.normalized	number	No explicit range	yes
sequence[oneOf 1].lempel_ziv.threshold	number	No explicit range	yes
sequence[oneOf 1].lempel_ziv.alphabet	string	No explicit range	yes
sequence[oneOf 1].lempel_ziv.normalization	string	No explicit range	yes
sequence[oneOf 1].time_reversal	array		yes
sequence[oneOf 1].time_reversal[]	object		item
sequence[oneOf 1].time_reversal[].lag	integer	minimum=0	yes
sequence[oneOf 1].time_reversal[].raw	number	No explicit range	yes
sequence[oneOf 1].time_reversal[].standardized	number	No explicit range	yes
sequence[oneOf 1].time_reversal[].note	string	No explicit range	yes
sequence[oneOf 1].interpretation	string	No explicit range	yes
sequence[oneOf 2]	object		branch
sequence[oneOf 2].available	fixed value	const=false	yes
sequence[oneOf 2].reason	string	No explicit range	yes
sequence[oneOf 2].n	integer	minimum=0	no
sequence[oneOf 2].window	integer	minimum=0	no
sequence[oneOf 3]	object		branch
sequence[oneOf 3].error	string	No explicit range	yes
browser-calibration			
calibration_bin_counts[]	integer	minimum=20	item
absolute_residual_radii[]	array	minItems=12; maxItems=12	item
absolute_residual_radii[][]	number	minimum=0	item
global_radii[]	number	minimum=0	item
browser-ridge-study			
absolute_residual_radii[]	number	minimum=0	item
selection[]	object		item

Field path	JSON type	Explicit constraint	Req.
selection[].penalty	fixed value	enum=[0,1e-06,0.0001,0.01,0.1,1,10]	yes
selection[].normalized_mse	number	minimum=0	yes
selection[].selected	boolean	No explicit range	no
reevaluation			
protocol.name	string	No explicit range	yes
protocol.forecast_origins_per_case	integer	No explicit range	yes
protocol.quantile_levels	array		yes
protocol.quantile_levels[]	number	No explicit range	item
protocol.heavy_forecasts	string	No explicit range	yes
protocol.light_forecasts	string	No explicit range	yes
protocol.history	string	No explicit range	yes
protocol.limits	string	No explicit range	yes
protocol.context_memory	object		yes
protocol.context_memory{key}	number	No explicit range	keyed
trace_sha256{key}	string	pattern="^[0-9a-f]{64}$"	keyed
code_sha256{key}	string	pattern="^[0-9a-f]{64}$"	keyed
cases[]	object		item
cases[].case_id	string	No explicit range	yes
cases[].seasonality	integer	minimum=1	yes
cases[].horizon	integer	minimum=1	yes
cases[].history_length	integer	minimum=2	yes
cases[].actual	array	minItems=1	yes
cases[].actual[]	number	No explicit range	item
cases[].methods	array	minItems=1	yes
cases[].methods[]	object		item
cases[].methods[].name	string	No explicit range	yes
cases[].methods[].family	string	No explicit range	yes
cases[].methods[].source	fixed value	enum=["frozen released forecast","recomputed from current source"]	yes
cases[].methods[].point	array	minItems=1	yes
cases[].methods[].point[]	number	No explicit range	item
cases[].methods[].lower	array	minItems=1	yes
cases[].methods[].lower[]	number	No explicit range	item
cases[].methods[].upper	array	minItems=1	yes
cases[].methods[].upper[]	number	No explicit range	item
cases[].methods[].metrics	object		yes
cases[].methods[].metrics.mae	number	minimum=0	yes
cases[].methods[].metrics.rmse	number	minimum=0	yes
cases[].methods[].metrics.mase	number/null	minimum=0	yes
cases[].methods[].metrics.msis	number/null	minimum=0	yes
cases[].methods[].metrics.wql	number/null	minimum=0	yes
cases[].methods[].metrics.coverage	number	minimum=0; maximum=1	yes
cases[].methods[].metrics.mean_width	number	minimum=0	yes
cases[].methods[].fit_inference_seconds	number	minimum=0	no
cases[].methods[].warning_categories	array		no
cases[].methods[].warning_categories[]	string	No explicit range	item
cases[].methods[].expert_weights	object		no
cases[].methods[].expert_weights{key}	number	minimum=0; maximum=1	keyed

Field path	JSON type	Explicit constraint	Req.
cases[].methods[].training_origins	array		no
cases[].methods[].training_origins[]	integer	minimum=1	item
cases[].history	array	minItems=1	yes
cases[].history[]	number	No explicit range	item
cases[].category	string	No explicit range	yes
cases[].legacy_backtest_window_counts	object		yes
cases[].legacy_backtest_window_counts{key}	integer/null	No explicit range	keyed
summary[]	object		item
summary[].name	string	No explicit range	yes
summary[].cases	integer	minimum=1	yes
summary[].mean_mase	number	No explicit range	yes
summary[].mean_coverage	number	minimum=0; maximum=1	yes
input_sha256{key}	string	pattern="^[0-9a-f]{64}$"	keyed
runtime.python	string	No explicit range	yes
runtime.platform	string	No explicit range	yes
runtime.packages	object		yes
runtime.packages{key}	string	No explicit range	keyed
runtime.checkpoint_revisions	object		yes
runtime.checkpoint_revisions{key}	string	pattern="^[0-9a-f]{40}$"	keyed
runtime.checkpoint_sha256	object		yes
runtime.checkpoint_sha256{key}	object		keyed
runtime.checkpoint_sha256{key}{key}	string	pattern="^[0-9a-f]{64}$"	keyed
runtime.cuda_available	boolean	No explicit range	yes
runtime.torch_cuda	string/null	No explicit range	yes
runtime.gpu	string/null	No explicit range	yes
runtime.torch_cpu_threads	integer	minimum=1	yes
fresh-reevaluation			
protocol.name	string	No explicit range	yes
protocol.forecast_origins_per_case	integer	No explicit range	yes
protocol.quantile_levels	array		yes
protocol.quantile_levels[]	number	No explicit range	item
protocol.heavy_forecasts	string	No explicit range	yes
protocol.light_forecasts	string	No explicit range	yes
protocol.history	string	No explicit range	yes
protocol.limits	string	No explicit range	yes
protocol.context_memory	object		yes
protocol.context_memory{key}	number	No explicit range	keyed
trace_sha256{key}	string	pattern="^[0-9a-f]{64}$"	keyed
code_sha256{key}	string	pattern="^[0-9a-f]{64}$"	keyed
cases[]	object		item
cases[].case_id	string	No explicit range	yes
cases[].seasonality	integer	minimum=1	yes
cases[].horizon	integer	minimum=1	yes
cases[].history_length	integer	minimum=2	yes
cases[].actual	array	minItems=1	yes
cases[].actual[]	number	No explicit range	item
cases[].methods	array	minItems=1	yes
cases[].methods[]	object		item
cases[].methods[].name	string	No explicit range	yes
cases[].methods[].family	string	No explicit range	yes

Field path	JSON type	Explicit constraint	Req.
cases[].methods[].source	fixed value	enum=["frozen released forecast","recomputed from current source"]	yes
cases[].methods[].point	array	minItems=1	yes
cases[].methods[].point[]	number	No explicit range	item
cases[].methods[].lower	array	minItems=1	yes
cases[].methods[].lower[]	number	No explicit range	item
cases[].methods[].upper	array	minItems=1	yes
cases[].methods[].upper[]	number	No explicit range	item
cases[].methods[].metrics	object		yes
cases[].methods[].metrics.mae	number	minimum=0	yes
cases[].methods[].metrics.rmse	number	minimum=0	yes
cases[].methods[].metrics.mase	number/null	minimum=0	yes
cases[].methods[].metrics.msis	number/null	minimum=0	yes
cases[].methods[].metrics.wql	number/null	minimum=0	yes
cases[].methods[].metrics.coverage	number	minimum=0; maximum=1	yes
cases[].methods[].metrics.mean_width	number	minimum=0	yes
cases[].methods[].fit_inference_seconds	number	minimum=0	no
cases[].methods[].warning_categories	array		no
cases[].methods[].warning_categories[]	string	No explicit range	item
cases[].methods[].expert_weights	object		no
cases[].methods[].expert_weights{key}	number	minimum=0; maximum=1	keyed
cases[].methods[].training_origins	array		no
cases[].methods[].training_origins[]	integer	minimum=1	item
cases[].history	array	minItems=1	yes
cases[].history[]	number	No explicit range	item
cases[].category	string	No explicit range	yes
cases[].legacy_backtest_window_counts	object		yes
cases[].legacy_backtest_window_counts{key}	integer/null	No explicit range	keyed
summary[]	object		item
summary[].name	string	No explicit range	yes
summary[].cases	integer	minimum=1	yes
summary[].mean_mase	number	No explicit range	yes
summary[].mean_coverage	number	minimum=0; maximum=1	yes
input_sha256{key}	string	pattern="^[0-9a-f]{64}$"	keyed
runtime.python	string	No explicit range	yes
runtime.platform	string	No explicit range	yes
runtime.packages	object		yes
runtime.packages{key}	string	No explicit range	keyed
runtime.checkpoint_revisions	object		yes
runtime.checkpoint_revisions{key}	string	pattern="^[0-9a-f]{40}$"	keyed
runtime.checkpoint_sha256	object		yes
runtime.checkpoint_sha256{key}	object		keyed
runtime.checkpoint_sha256{key}{key}	string	pattern="^[0-9a-f]{64}$"	keyed
runtime.cuda_available	boolean	No explicit range	yes
runtime.torch_cuda	string/null	No explicit range	yes

Field path	JSON type	Explicit constraint	Req.
runtime.gpu	string/null	No explicit range	yes
runtime.torch_cpu_threads	integer	minimum=1	yes
index			
cases[]	object		item
cases[].case_id	string	No explicit range	yes
cases[].category	string	No explicit range	yes
cases[].manifest_path	string	No explicit range	yes
selector			
ranking[]	array	minItems=2; maxItems=2	item
ranking[][0]	string	No explicit range	tuple
ranking[][1]	number	No explicit range	tuple
forecastability			
packages{key}	string	No explicit range	keyed
code_sha256{key}	string	pattern="^[a-f0-9]{64}$"	keyed
excluded[]	object		item
excluded[].id	string	No explicit range	yes
excluded[].excluded	string	No explicit range	yes
excluded[].sequence_sha256	string	No explicit range	yes
model_names[]	string	No explicit range	item
summary.discovery_n	integer	minimum=0	yes
summary.confirmation_n	integer	minimum=0	yes
summary.associations	array		yes
summary.associations[]	object		item
summary.associations[].feature	string	No explicit range	yes
summary.associations[].discovery	object		yes
summary.associations[].discovery.feature	string	No explicit range	yes
summary.associations[].discovery.n	integer	minimum=0	yes
summary.associations[].discovery.tau	number	minimum=-1; maximum=1	yes
summary.associations[].discovery.p	number	minimum=0; maximum=1	yes
summary.associations[].discovery.q	number	minimum=0; maximum=1	yes
summary.associations[].discovery.rejected	boolean	No explicit range	yes
summary.associations[].confirmation	object		yes
summary.associations[].confirmation.feature	string	No explicit range	yes
summary.associations[].confirmation.n	integer	minimum=0	yes
summary.associations[].confirmation.tau	number	minimum=-1; maximum=1	yes
summary.associations[].confirmation.p	number	minimum=0; maximum=1	yes
summary.associations[].confirmation.q	number	minimum=0; maximum=1	yes
summary.associations[].confirmation.rejected	boolean	No explicit range	yes
summary.associations[].confirmed	boolean	No explicit range	yes
summary.constant_method	string	No explicit range	yes
summary.selection_mean_mase	object		yes
summary.selection_mean_mase.selected	number	No explicit range	yes

Field path	JSON type	Explicit constraint	Req.
summary.selection_mean_mase.discovery_constant	number	No explicit range	yes
summary.selection_mean_mase.oracle	number	No explicit range	yes
summary.model_mean_mase	object		yes
summary.model_mean_mase{key}	number	No explicit range	keyed
summary.selector_features	array		yes
summary.selector_features[]	string	No explicit range	item
summary.discovery_medians	array		yes
summary.discovery_medians[]	number	No explicit range	item
summary.limitations	array		yes
summary.limitations[]	string	No explicit range	item
series[]	object		item
series[].id	string	No explicit range	yes
series[].start_timestamp	string	No explicit range	yes
series[].source_n	integer	minimum=0	yes
series[].sequence_sha256	string	pattern="^[a-f0-9]{64}$"	yes
series[].partition	fixed value	enum=["discovery","confirmation"]	yes
series[].features	object		yes
series[].features{key}	number/null	No explicit range	keyed
series[].mase	object		yes
series[].mase{key}	number	No explicit range	keyed
series[].origins	array	minItems=3; maxItems=3	yes
series[].origins[]	integer	minimum=0	item
series[].selected_method	string	No explicit range	no
audit.verified_origin_scores	integer	minimum=0	yes
audit.selector_sha256	string	No explicit range	yes
audit.cache_identity	string	No explicit range	yes
combination-index			
code_sha256{key}	string	pattern="^[a-f0-9]{64}$"	keyed
packages{key}	string	No explicit range	keyed
cohorts[]	object		item
cohorts[].id	string	No explicit range	yes
cohorts[].name	string	No explicit range	yes
cohorts[].inventory	integer	minimum=0	yes
cohorts[].admitted	integer	minimum=0	yes
cohorts[].excluded	integer	minimum=0	yes
cohorts[].exclusion_reasons	object		yes
cohorts[].exclusion_reasons{key}	integer	minimum=0	keyed
cohorts[].origins	integer	minimum=0	yes
cohorts[].summary	array		yes
cohorts[].summary[]	object		item
cohorts[].summary[].name	string	No explicit range	yes
cohorts[].summary[].series	integer	minimum=0	yes
cohorts[].summary[].mase	number	No explicit range	yes
cohorts[].summary[].mae	number	No explicit range	yes
cohorts[].summary[].coverage	number	No explicit range	yes
cohorts[].summary[].mean_width	number	No explicit range	yes
cohorts[].summary[].msis	number	No explicit range	yes
cohorts[].selection_counts	object		yes
cohorts[].selection_counts{key}	integer	minimum=0	keyed
cases[]	object		item
cases[].id	string	No explicit range	yes
cases[].dataset	string	No explicit range	yes
cases[].source_n	integer	minimum=0	yes

Field path	JSON type	Explicit constraint	Req.
cases[].path	string	pattern="^combinations/(m4_weekly|m3_monthly|selected_cases)/[A-Za-z0-9_]+\\.json$"	yes
cases[].origins	integer	minimum=0	yes
cases[].excluded	string	No explicit range	no
sources{key}	object		keyed
sources{key}.doi	string	No explicit range	yes
sources{key}.license	fixed value	const="CC-BY-4.0"	yes
sources{key}.tsf_sha256	string	pattern="^[a-f0-9]{64}$"	yes
sources{key}.zip_sha256	string	pattern="^[a-f0-9]{64}$"	yes
sources{key}.horizon	integer	minimum=0	yes
sources{key}.seasonality	integer	minimum=0	yes
audit.path	string	No explicit range	yes
audit.sha256	string	pattern="^[a-f0-9]{64}$"	yes
audit.uncompressed_sha256	string	pattern="^[a-f0-9]{64}$"	yes
audit.scope	string	No explicit range	yes
combination-case			
origins[]	object		item
origins[].origin	integer	minimum=0	yes
origins[].prefix_start	integer	minimum=0	yes
origins[].prefix_length	integer	minimum=0	yes
origins[].prefix_sha256	string	pattern="^[a-f0-9]{64}$"	yes
origins[].scale	number	exclusiveMinimum=0	yes
origins[].actual	array	minItems=1	yes
origins[].actual[]	number	No explicit range	item
origins[].methods	array	minItems=1	yes
origins[].methods[]	object		item
origins[].methods[].name	string	No explicit range	yes
origins[].methods[].quantiles	array	minItems=1	yes
origins[].methods[].quantiles[]	array	minItems=3; maxItems=3	item
origins[].methods[].quantiles[][]	number	No explicit range	item
origins[].methods[].metrics	object		yes
origins[].methods[].metrics.mae	number	No explicit range	yes
origins[].methods[].metrics.rmse	number	No explicit range	yes
origins[].methods[].metrics.mase	number/null	No explicit range	yes
origins[].methods[].metrics.coverage	number	No explicit range	yes
origins[].methods[].metrics.mean_width	number	No explicit range	yes
origins[].methods[].metrics.msis	number/null	No explicit range	yes
origins[].methods[].metrics.wql	number/null	No explicit range	yes
origins[].methods[].weights	array	minItems=5; maxItems=5	no
origins[].methods[].weights[]	number	No explicit range	item
origins[].methods[].memory_origins	array		no
origins[].methods[].memory_origins[]	integer	minimum=0	item
origins[].methods[].selected	array	minItems=3; maxItems=3	no
origins[].methods[].selected[]	string/number	No explicit range	item
origins[].methods[].validation_blocks	integer	minimum=0	no
readiness			
contracts[]	object		item
contracts[].id	string	No explicit range	yes
contracts[].version	string	pattern="^[0-9]+\\.[0-9]+\\.[0-9]+$"	yes

Field path	JSON type	Explicit constraint	Req.
`contracts[].status`	fixed value	enum=["real","active"]	yes
`contracts[].state`	fixed value	enum=["available","missing"]	yes
`contracts[].missing`	array		yes
`contracts[].missing[]`	string	No explicit range	item

References

[1] R. J. Hyndman and B. Billah, Unmasking the Theta method. *International Journal of Forecasting*, 2003. https://robjhyndman.com/papers/Theta.pdf.

[2] R. J. Hyndman and A. B. Koehler, Another look at measures of forecast accuracy. *International Journal of Forecasting*, 2006. https://doi.org/10.1016/j.ijforecast.2006.03.001.

[3] T. Gneiting and A. E. Raftery, Strictly proper scoring rules, prediction, and estimation. *JASA*, 2007. https://doi.org/10.1198/016214506000001437.

[4] P. Montero-Manso et al., FFORMA: Feature-based forecast model averaging. *International Journal of Forecasting*, 2020. https://doi.org/10.1016/j.ijforecast.2019.02.011.

[5] V. Cerqueira et al., Arbitrage of forecasting experts. *Machine Learning*, 108, 913–944, 2019. https://doi.org/10.1007/s10994-018-05774-y.

[6] A. P. Dawid, Statistical theory: The prequential approach. *JRSS A*, 1984. https://doi.org/10.2307/2981683.

[7] I. Gibbs and E. Candès, Adaptive conformal inference under distribution shift. *NeurIPS*, 2021. https://arxiv.org/abs/2106.00170.

[8] A. N. Angelopoulos, E. Candès and R. J. Tibshirani, Conformal PID control for time series prediction. https://arxiv.org/abs/2307.16895.

[9] R. J. Hyndman and G. Athanasopoulos, Forecasting: Principles and Practice. Holt–Winters seasonal method. https://otexts.com/fpp3/holt-winters.html.

[10] Nixtla, StatsForecast model implementation documentation. https://nixtlaverse.nixtla.io/statsforecast/src/core/models.html.

[11] A. Petralia et al., Timestral. arXiv:2610.37585, 2026. https://arxiv.org/abs/2610.37585. Inference source: https://github.com/mistralai/timestral.

[12] Amazon Science, Chronos forecasting implementation and releases. https://github.com/amazon-science/chronos-forecasting.

[13] Google Research, TimesFM model API. https://github.com/google-research/timesfm.

[14] P. Podest et al., TiRex-2: Generalizing TiRex to Multivariate Data and Streaming, 2026. https://arxiv.org/abs/2607.01204.

[15] D. A. Dickey and W. A. Fuller, Distribution of the estimators for autoregressive time series with a unit root. JASA, 1979. https://doi.org/10.1080/01621459.1979.10482531.

[16] D. Kwiatkowski, P. C. B. Phillips, P. Schmidt and Y. Shin, Testing the null hypothesis of stationarity against the alternative of a unit root. Journal of Econometrics, 1992. https://doi.org/10.1016/0304-4076(92)90104-Y.

[17] R. Killick, P. Fearnhead and I. A. Eckley, Optimal detection of changepoints with a linear computational cost. JASA, 2012. https://doi.org/10.1080/01621459.2012.737745.

[18] C. W. J. Granger, Investigating causal relations by econometric models and cross-spectral methods. Econometrica, 1969. https://doi.org/10.2307/1912791.

[19] R. F. Engle and C. W. J. Granger, Co-integration and error correction: representation, estimation, and testing. Econometrica, 1987. https://doi.org/10.2307/1913236.

[20] S. Johansen, Estimation and hypothesis testing of cointegration vectors in Gaussian vector autoregressive models. Econometrica, 1991. https://doi.org/10.2307/2938278.

[21] C.-C. M. Yeh et al., Matrix Profile I: All Pairs Similarity Joins for Time Series. ICDM, 2016. https://doi.org/10.1109/ICDM.2016.0179.

[22] S. Gharghabi et al., Matrix Profile VIII: Domain Agnostic Online Semantic Segmentation at Superhuman Performance Levels. ICDM, 2017. https://doi.org/10.1109/ICDM.2017.21.

[23] M. Costa, A. L. Goldberger and C.-K. Peng, Multiscale Entropy Analysis of Complex Physiologic Time Series. Physical Review Letters 89, 068102, 2002. https://doi.org/10.1103/PhysRevLett.89.068102.

[24] A. Lempel and J. Ziv, On the Complexity of Finite Sequences. IEEE Transactions on Information Theory 22, 75–81, 1976. https://doi.org/10.1109/TIT.1976.1055501.

[25] C. H. Lubba et al., catch22: CAnonical Time-series CHaracteristics. Data Mining and Knowledge Discovery 33, 1821–1852, 2019. https://doi.org/10.1007/s10618-019-00647-x.

[26] A. Trindade, ElectricityLoadDiagrams20112014. UCI Machine Learning Repository, 2015. https://doi.org/10.24432/C58C86.

[27] S. Chen, Beijing PM2.5 Data. UCI Machine Learning Repository, 2017. https://doi.org/10.24432/C5JS49.

[28] R. Godahewa et al., M4 Hourly Dataset, Monash Forecasting Repository. https://doi.org/10.5281/zenodo.4656589.

[29] R. Fildes, P. Goodwin and S. De Baets, Forecast value added in demand planning, *International Journal of Forecasting* 41(2), 649–669, 2025. https://doi.org/10.1016/j.ijforecast.2024.07.006.

[30] R. Godahewa et al., M4 Daily Dataset, Monash Forecasting Repository. https://doi.org/10.5281/zenodo.4656548.

[31] tsfresh, Feature filtering: dependent-test significance and Benjamini–Yekutieli correction. https://tsfresh.readthedocs.io/en/v0.21.0/text/feature_filtering.html.

[32] R. Godahewa et al., M4 Weekly Dataset, Monash Forecasting Repository, version 3. https://doi.org/10.5281/zenodo.4656522.

[33] R. Godahewa et al., M3 Monthly Dataset, Monash Forecasting Repository, version 3. https://doi.org/10.5281/zenodo.4656298.

[34] A. N. Angelopoulos and S. Bates, A Gentle Introduction to Conformal Prediction and Distribution-Free Uncertainty Quantification. https://arxiv.org/abs/2107.07511.

[35] R. J. Hyndman and Y. Khandakar, Automatic time series forecasting: The forecast package for R. *Journal of Statistical Software*, 27, 1–22, 2008. https://doi.org/10.18637/jss.v027.i03.

[36] R. J. Hyndman, A. B. Koehler, R. D. Snyder and S. Grose, A state space framework for automatic forecasting using exponential smoothing methods. *International Journal of Forecasting*, 18, 439–454, 2002. https://doi.org/10.1016/S0169-2070(01)00110-8.

[37] G. Ke et al., LightGBM: A highly efficient gradient boosting decision tree. *Advances in Neural Information Processing Systems*, 30, 2017. https://papers.nips.cc/paper/2017/hash/6449f44a102fde848669bdd9eb6b76fa-Abstract.html.

[38] Nixtla, MLForecast documentation and lag-feature forecasting API. https://nixtlaverse.nixtla.io/mlforecast/.

[39] A. Zeng, M. Chen, L. Zhang and Q. Xu, Are transformers effective for time series forecasting? AAAI, 2023; arXiv:2205.13504. https://arxiv.org/abs/2205.13504.

[40] C. Challu et al., N-HiTS: Neural hierarchical interpolation for time series forecasting. AAAI, 2023; arXiv:2201.12886. https://arxiv.org/abs/2201.12886.

[41] Nixtla, NeuralForecast model and multi-quantile loss documentation. https://nixtlaverse.nixtla.io/neuralforecast/.

[42] A. F. Ansari et al., Chronos: Learning the language of time series. arXiv:2403.07815, 2024. https://arxiv.org/abs/2403.07815.

[43] A. Das, W. Kong, R. Sen and Y. Zhou, A decoder-only foundation model for time-series forecasting. ICML, 2024; arXiv:2310.10688. https://arxiv.org/abs/2310.10688.

[44] S. Boyd and L. Vandenberghe, *Convex Optimization*. Cambridge University Press, 2004. https://web.stanford.edu/~boyd/cvxbook/.